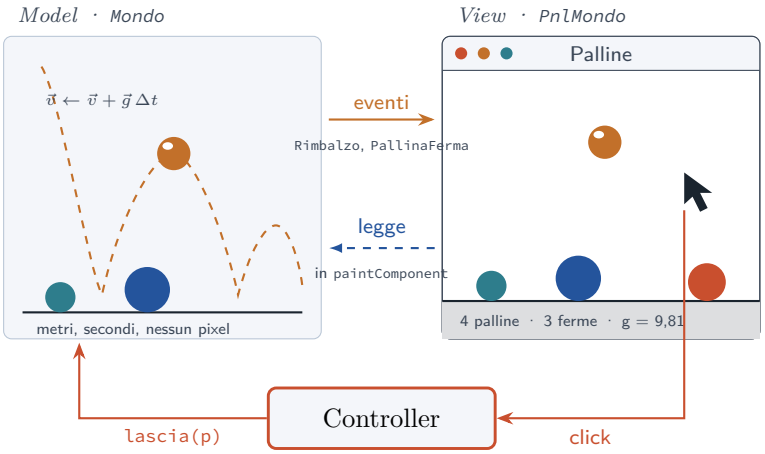
PAJC · Programmazione Avanzata in Java e C · 2026/27

Dispensa extra 2

Pattern in gioco

Eventi, Model-View-Controller, un mondo fisico che rimbalza bene e i pattern che tengono in piedi un gioco in Java e Swing



Un mondo che evolve da solo, una finestra che lo guarda.

prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1.3

Quest'opera è distribuita con licenza Creative Commons "Attribuzione – Non commerciale – Condividi allo stesso modo 4.0 Internazionale".

Indice

1 Introduzione: un giocattolo che non regge **1**
- 1.1 Il problema . . . 1
- 1.2 L'immagine guida: un mondo che non sa di Swing . . . 4
- 1.3 Che cosa serve . . . 5
- 1.4 La strada che percorreremo . . . 6
- Esercizi . . . 7
- Risposte . . . 7

2 Il mondo e la sua immagine: MVC **9**
- 2.1 Il mondo come oggetti . . . 9
- 2.2 La View: disegnare ciò che c'è . . . 12
- 2.3 Il Controller e il tempo . . . 14
- 2.4 Il triangolo: chi parla con chi . . . 15
- 2.5 Il premio: un mondo senza finestra . . . 17
- 2.6 L'esempio guida: Palline, prima versione . . . 20
- Esercizi . . . 21
- Risposte . . . 23

3 Il Model che avvisa: programmazione a eventi **25**
- 3.1 Chiedere o farsi avvisare . . . 25
- 3.2 Scrivere una sorgente di eventi . . . 26
- 3.3 Le proprietà: `PropertyChangeSupport` . . . 28
- 3.4 Le regole . . . 29
- 3.5 La View che ascolta, la View che annuncia . . . 31
- 3.6 L'esempio guida: Palline, seconda versione . . . 33
- Esercizi . . . 37
- Risposte . . . 39

4 Un mondo che rimbalza bene: il modello fisico **41**
- 4.1 Il tempo a passi fissi . . . 41
- 4.2 Integrare il moto . . . 43
- 4.3 Il rimbalzo, fatto bene . . . 44
- 4.4 Le palline fra loro . . . 46
- 4.5 Ostacoli e tunneling . . . 47
- 4.6 Misurare . . . 49
- 4.7 L'esempio guida: Palline, terza versione . . . 51
- Esercizi . . . 52
- Risposte . . . 55

5 Comportamenti, stati e comandi **56**
- 5.1 Comportamenti intercambiabili: Strategy . . . 56
- 5.2 Uno scheletro da completare: Template Method . . . 59
- 5.3 Gli stati del gioco: State . . . 62

5.4 L'input come oggetto: Command . . . 65
5.5 L'esempio guida: Palline, quarta versione . . . 68
Esercizi . . . 69
Risposte . . . 71

6 Creare e condividere: fabbriche, risorse e singleton 73
6.1 Le risorse, e dove stanno . . . 73
6.2 Chi crea: Factory . . . 77
6.3 Uno per tutti: Flyweight . . . 80
6.4 Riusare invece di creare: Object Pool . . . 81
6.5 La tabella dei pattern . . . 82
6.6 L'esempio guida: Palline, quinta versione . . . 83
Esercizi . . . 84
Risposte . . . 86

7 Palestra 88
Problema 1: chi parla con chi . . . 88
Problema 2: il contatore in tre versioni . . . 89
Problema 3: Pong . . . 89
Problema 4: il gioco della vita, completo . . . 90
Problema 5: il biliardo con le buche . . . 90
Problema 6: Mattoni . . . 91
Problema 7: Asteroidi . . . 92
Problema 8: Palline completa . . . 92

8 Dove andare adesso 94
8.1 Quando la gerarchia non basta: Entity-Component-System . . . 94
8.2 Il ciclo fuori dal thread degli eventi . . . 94
8.3 Motori fisici e strutture spaziali . . . 95
8.4 Il gioco in rete . . . 95
8.5 I test del Model, come abitudine . . . 95
8.6 Oltre Swing . . . 95
Nelle altre dispense . . . 96
Riferimenti . . . 96

Capitolo 1

Introduzione: un giocattolo che non regge

1.1 Il problema

Il programma che accompagna tutta la dispensa si chiama *Palline*, ed è un giocattolo più che un gioco. Una finestra scura con un pavimento in basso: a ogni click si lascia cadere una pallina dal punto in cui si è cliccato, la pallina cade, tocca il pavimento, rimbalza, risale un po' meno di prima, e dopo qualche rimbalzo si ferma. Le palline sono di tre materiali (gomma, piombo, bolla di sapone) che rimbalzano in modo diverso; uno slider regola la gravità; un pulsante mette in pausa; *Ctrl+Z* toglie l'ultima pallina lasciata e *Riproduci* rifaà da capo tutta la scena, con i tempi veri; a ogni urto si sente un *toc*, più forte se l'urto è più forte; un menu carica scenari pronti (la pioggia di palline, la piramide, il biliardo) e una riga di stato in basso dice quante palline ci sono e quante sono ferme.

Nessuna di queste cose è difficile da descrivere, e per il primo pezzo, le palline che cadono e rimbalzano, c'è un punto di partenza che molti conoscono: un pannello che disegna le palline e un timer che le sposta. Palline è nata da lì, aggiungendo una funzione alla volta, sempre nello stesso posto. Il risultato è una classe `PnlPalline` di settecento righe, che tiene la lista delle palline, le muove, le disegna, ascolta il mouse e la tastiera, aggiorna la riga di stato, suona e carica gli scenari. Il cuore è il metodo che disegna:

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    if (!inPausa && !nelMenu) {
        for (Pallina p : palline) {
            p.vy += 0.3;                // gravita, in pixel per fotogramma
            p.x += p.vx;
            p.y += p.vy;
            if (p.y + p.r > getHeight())     // il pavimento: inverti e basta
                p.vy = -p.vy * restituzione(p);
        }
    }
    for (Pallina p : palline) {
        g.setColor(p.colore);
        g.fillOval((int) (p.x - p.r), (int) (p.y - p.r), (int) (2 * p.r),
                   (int) (2 * p.r));
    }
    lblStato.setText(palline.size() + " palline");
    if (!inPausa && !inReplay)
        repaint();                          // e si ricomincia
}
```

Si legge senza fatica. A ogni chiamata il metodo fa avanzare la fisica di un passo (la velocità verticale cresce di 0,3 pixel per fotogramma, che è la gravità; la posizione cresce della velocità; chi è sotto il pavimento inverte la velocità, ridotta di un fattore che dipende dal materiale), poi disegna tutte le palline, aggiorna l'etichetta della riga di stato e, con `repaint()`, chiede a Swing di essere richiamato: ogni disegno è un fotogramma e il fotogramma dopo parte da solo. Il materiale decide quanto si perde a ogni rimbalzo con una catena di confronti sul nome:

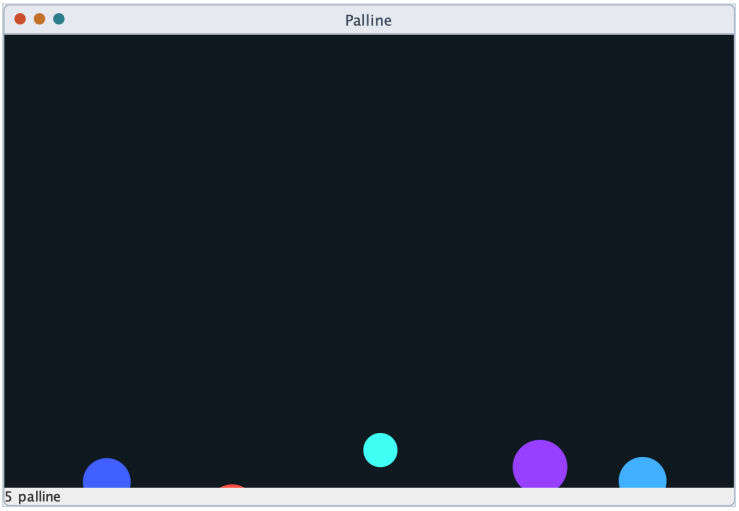
```
double restituzione(Pallina p) {
    if (p.materiale.equals("gomma"))
        return 0.8;
    else if (p.materiale.equals("piombo"))
        return 0.2;
    else if (p.materiale.equals("bolla"))
        return 0.0;
    return 0.5;
}
```

Lanciato, il programma funziona: le palline cadono, rimbalzano, si accumulano in basso, l'etichetta conta. Per qualche minuto sembra che manchi solo di rifinire. Poi lo si usa un po' più a lungo, o lo si dà a qualcun altro.

Si cambia computer. Sul portatile di chi l'ha scritto le palline cadono con una lentezza credibile. Su un computer più veloce precipitano: la gravità è "0,3 pixel per fotogramma", e i fotogrammi sono tanti quanti Swing riesce a disegnarne, cioè quanti il computer ne regge. Un computer due volte più veloce disegna due volte più fotogrammi al secondo e le palline cadono con un'accelerazione quattro volte più grande, perché la gravità si somma a ogni fotogramma e lo spostamento pure. Lo stesso programma, su macchine diverse, simula mondi diversi.

Si copre la finestra. Si porta davanti un'altra finestra, o si riduce Palline a icona: quando la si riapre, le palline sono esattamente dove le avevamo lasciate. Il mondo si è fermato, perché Swing non disegna ciò che non si vede, e in questo programma "disegnare" e "far passare il tempo" sono la stessa cosa. In un gioco vero, un avversario che si ferma quando non lo guardiamo è un errore; qui è soltanto strano, ma la causa è la stessa.

Si aspetta. Dopo i primi rimbalzi le palline dovrebbero fermarsi sul pavimento. Invece *tremolano*, e poi affondano: a ogni fotogramma la gravità le spinge un po' sotto il pavimento, il rimbalzo inverte la velocità ma non le riporta sopra, e il fattore di restituzione fa sì che il colpo verso l'alto sia sempre più debole di quello verso il basso. Dopo qualche secondo le palline sono mezze dentro il pavimento (la schermata qui sotto), e un po' dopo scompaiono sotto il bordo. Nessun errore, nessuna eccezione: semplicemente la fisica è sbagliata in un modo che si vede solo aspettando.



Palline dopo qualche secondo: le palline non si fermano sul pavimento, ci affondano.

Si preme pausa durante la riproduzione. Pausa mette `inPausa` a vero e la fisica si ferma; *Riproduci* mette `inReplay` a vero e rilegge gli eventi registrati. Premendo pausa durante la riproduzione le palline si fermano ma la riproduzione continua a inserirne di nuove, perché il codice del replay controlla `inReplay` e non `inPausa`; e se nel frattempo si apre il menu, `nelMenu` ferma la fisica ma non il replay, e alla chiusura del menu si ritrovano venti palline comparse nello stesso istante. Tre booleani fanno otto combinazioni, di cui cinque non dovrebbero esistere, e ogni `if` del programma ne considera una o due.

Si ascolta. A ogni rimbalzo il programma apre il file del suono, lo legge dal disco e lo suona. Con venti palline che tremolano sul pavimento sono centinaia di rimbalzi al secondo, centinaia di letture del disco, e un crepitio continuo che non somiglia a nessun *toc*. Gli scenari, infine, sono sei metodi quasi identici, copiati l'uno dall'altro e corretti a mano; il settimo, la "cascata", è stato rimandato perché nessuno aveva voglia di copiarne un altro.

Ognuno di questi difetti ha una causa precisa, e nessuna è un errore di battitura. Sono tutte conseguenze di un'unica scelta: tenere tutto in un posto solo, e far fare a quel posto cose che non c'entrano l'una con l'altra. Ne nascono le tre domande della dispensa.

- **Dove vive il mondo, e chi lo disegna?** Le palline con le loro posizioni, le velocità, il pavimento sono una cosa; il modo in cui compaiono sullo schermo è un'altra. Finché stanno

nella stessa classe, non si può provare la fisica senza una finestra, né cambiare la finestra senza toccare la fisica.

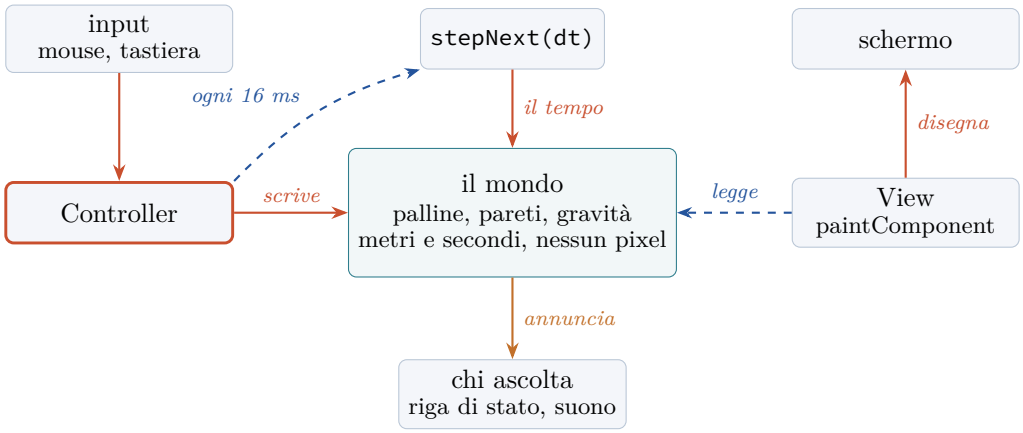
- **Come scorre il tempo?** Il mondo ha un suo tempo, in secondi, in cui una pallina cade di quasi cinque metri nel primo secondo; lo schermo ha i suoi fotogrammi, tanti quanti il computer ne regge. I due tempi vanno tenuti distinti, e il primo non deve dipendere dal secondo.
- **Come si parlano le parti?** La riga di stato vuole sapere quante palline sono ferme, il suono vuole sapere quando c'è un urto, la pausa vuole fermare tutto: senza che ogni parte conosca tutte le altre, e senza che un cambiamento in una obblighi a riscrivere le altre.

Il capitolo 7 riprende Palline e la porta fino in fondo, con tutte le funzioni descritte all'inizio e nessuno dei difetti visti qui. In mezzo c'è tutto ciò che serve per arrivarci, e soprattutto un modo di pensare il programma per parti che hanno ciascuna un compito solo.

1.2 L'immagine guida: un mondo che non sa di Swing

Un programma interattivo, e un gioco in particolare, è un ciclo. A ogni giro raccoglie l'*input* (il mouse, la tastiera), fa avanzare il *mondo* di un passo di tempo, e *disegna* il mondo com'è adesso. Poi ricomincia, decine di volte al secondo. In Swing il ciclo lo scandisce un `javax.swing.Timer`, e ogni giro è un evento come gli altri, eseguito dal thread degli eventi: il mouse e la tastiera arrivano dalla stessa coda, il disegno lo fa `paintComponent` dello stesso thread (*Thread e concorrenza* spiega la coda; *Interfacce grafiche con Swing* il disegno). Qui basta sapere che è così, e che quindi nessuna delle parti ha bisogno di proteggersi dalle altre.

Dentro questo ciclo c'è un'immagine che tornerà in ogni capitolo: **al centro sta il mondo, e il mondo non sa di Swing**. È un insieme di oggetti Java e basta, con le palline, le pareti, la gravità, misurati in metri e secondi, e un metodo che lo fa avanzare di un intervallo di tempo. Attorno ci sono tre cose che lo toccano in tre modi diversi. Il *Controller* gli scrive: traduce un click in "lascia una pallina qui", un tasto in "pausa", e gli dice quanto tempo è passato. La *View* lo legge: a ogni fotogramma guarda com'è il mondo e lo disegna, senza tenere niente per sé. E il mondo, quando gli succede qualcosa che qualcuno potrebbe voler sapere (un rimbalzo, una pallina che si ferma), lo *annuncia* a chi si è messo in ascolto, senza sapere chi sia.



Il ciclo di ogni programma di questa dispensa: il Controller scrive nel mondo, il mondo avanza, la View lo legge e lo disegna, il mondo annuncia ciò che gli succede.

La Palline del paragrafo precedente non ha un centro. Il mondo è sparso nel pannello, la fisica sta nel metodo che disegna, il tempo è il numero dei fotogrammi, la riga di stato è scritta

dal pannello che conosce l'etichetta per nome. Ogni difetto visto corrisponde a una freccia della figura che manca o che va nel verso sbagliato: la fisica in `paintComponent` è la View che scrive nel mondo; la gravità in pixel per fotogramma è il tempo del mondo appeso al tempo dello schermo; i tre booleani sono il mondo e il Controller mescolati; il suono che si carica a ogni urto è chi ascolta che non esiste, e il mondo che fa tutto da sé.

Le soluzioni che vedremo hanno un nome, e il nome è importante quanto la soluzione. Separare il mondo dalla View e da chi lo comanda si chiama *Model-View-Controller*; annunciare senza sapere a chi si chiama *Observer*; dare a ogni materiale il suo comportamento, invece della catena di `if`, si chiama *Strategy*; sostituire i tre booleani con un oggetto che sa in che stato siamo si chiama *State*; trasformare i tasti in oggetti che si possono registrare, annullare e rieseguire si chiama *Command*; caricare i suoni una volta sola e tenerli a disposizione è il problema che il *Singleton* risolve, con dei costi che vedremo. Sono *pattern*, schemi di soluzione che qualcuno ha riconosciuto in molti programmi diversi e ha descritto con un nome, un problema, una struttura e delle conseguenze.

Definizione 1.1: Pattern

Un *pattern* (in italiano *schema di progettazione*) è una soluzione ricorrente a un problema ricorrente nella struttura di un programma, descritta in modo che si possa riconoscere e riapplicare: un nome, il problema che risolve, le classi e gli oggetti coinvolti con i loro ruoli, e ciò che si guadagna e che si paga usandola. Un pattern non è una libreria né un pezzo di codice da copiare: è un'idea, che in ogni programma prende una forma un po' diversa.

I giochi sono il posto giusto per impararli, per una ragione semplice: ogni gioco ha un mondo che evolve, un tempo che scorre, un'interfaccia che lo mostra e lo comanda, degli stati (menu, in corso, pausa, finito), molti oggetti con comportamenti diversi, risorse da caricare. Sono gli stessi problemi di un'applicazione gestionale, di un editor o di un simulatore, ma in un gioco si vedono subito, si toccano con il mouse, e quando la soluzione è sbagliata lo si nota senza leggere il codice: le palline affondano.

Osservazione 1.1

Il salto è nel modo di pensare il programma: non più come una finestra che fa delle cose, ma come un mondo che esiste per conto suo, e attorno al quale la finestra è solo uno dei modi di guardarlo e di toccarlo. Il primo test di questa idea è banale e spietato: il mondo deve poter girare, ed essere provato, *senza nessuna finestra*. Se non si può, il mondo non è ancora separato.

1.3 Che cosa serve

Le classi, le interfacce, i record, le enumerazioni e le collezioni (*Java Fast Forward*): il mondo di Palline è fatto di record e di liste, e i pattern sono quasi tutti un modo di usare le interfacce. Le lambda e i method reference (*Dal nome alla funzione*): i listener e i comandi sono lambda. Le interfacce con i metodi `default`, le gerarchie `sealed` e lo `switch` con i pattern (*Le interfacce in Java moderno*): serviranno per confrontare una Strategy con uno `switch` sul tipo e capire quando conviene l'una e quando l'altro. Di Swing serve ciò che tratta *Interfacce grafiche con Swing*: un pannello che si disegna con `paintComponent`, le coordinate del mondo e quelle dei pixel, i listener del mouse e della tastiera, `javax.swing.Timer`, e l'idea, già vista lì, che un componente Swing tiene i suoi dati in un *Model* separato; qui la si ritroverà in grande. Del

thread degli eventi basta ciò che si è detto nella sezione precedente; i dettagli sono in *Thread e concorrenza*.

Il codice è Java 21 e usa solo la libreria standard, più JUnit 5 per i test del Model, che gli ambienti di sviluppo includono. Le poche schermate mostrano le finestre con l'aspetto *Metal*, quello predefinito di Swing.

Restano fuori, di proposito, il gioco in rete e il gioco a più giocatori; il ciclo del gioco su un thread diverso da quello degli eventi; i motori fisici completi, con i corpi rigidi e i vincoli; l'architettura a componenti (*Entity-Component-System*) dei giochi grandi; la grafica per immagini (*sprite*) e l'audio oltre il minimo per sentire un *toc*. Di ciascuno il capitolo 8 dà un assaggio e dice dove continuare. Restano fuori anche i pattern che un gioco non usa quasi mai (Visitor, Interpreter, Bridge): sono nominati in una tabella alla fine del capitolo 6, con una riga ciascuno.

1.4 La strada che percorreremo

Il capitolo 2 risponde alla prima domanda, e in parte alla seconda. Il mondo di Palline diventa oggetti Java puri, in metri e secondi, con un metodo che lo fa avanzare; la View lo disegna e basta; il Controller traduce il mouse e fa scorrere il tempo, misurandolo. È il *Model-View-Controller*, e il capitolo dice che cosa vuol dire in Swing, chi può parlare con chi, e qual è il premio: un test del mondo senza finestra.

Il capitolo 3 risponde alla terza. Il mondo annuncia ciò che gli succede (un rimbalzo, una pallina ferma) a chi si è messo in ascolto: il pattern *Observer*, il modo in cui Java lo realizza con gli eventi e i listener, gli strumenti della libreria per scriverne di propri, e le regole per non sbagliare. La riga di stato e il suono diventano listener; la gravità e la pausa diventano proprietà a cui lo slider e il pulsante si legano.

Il capitolo 4 completa la seconda domanda e sistema la fisica. Il passo di tempo fisso, che rende la simulazione uguale su ogni computer e riproducibile; il modo giusto di integrare il moto; il rimbalzo che riporta la pallina sopra il pavimento e la ferma quando deve fermarsi; gli urti fra palline, le pareti e gli ostacoli; le palline veloci che attraversano le cose sottili e come impedirlo. La View non cambia di una riga: è il premio della separazione.

Il capitolo 5 dà ai materiali il loro comportamento (*Strategy*), agli oggetti del mondo uno scheletro comune da completare (*Template Method*), al gioco i suoi stati come oggetti (*State*) e all'input la forma di comandi che si possono annullare e rieseguire (*Command*). Palline ottiene i tre materiali, il menu, la pausa, *Ctrl+Z* e *Riproduci*.

Il capitolo 6 si occupa di creare e condividere: le risorse caricate una volta sola (il *Singleton*, come si scrive bene e perché spesso conviene non usarlo), gli scenari costruiti da una fabbrica a partire da un file (*Factory*), i materiali condivisi fra mille palline (*Flyweight*), gli oggetti riusati invece che ricreati (*Object Pool*). In chiusura, la tabella dei pattern.

Il capitolo 7 è la palestra: otto programmi completi, in difficoltà crescente, da un contatore in tre versioni fino a Palline finita. Il capitolo 8 indica dove continuare.

Esercizi

Esercizio 1.1: Quattro volte più veloce (⋆)

Nella Palline del paragrafo 1.1 la gravità aggiunge 0,3 pixel per fotogramma alla velocità, e la velocità si somma alla posizione a ogni fotogramma. Il computer A disegna 60 fotogrammi al secondo, il computer B ne disegna 120. Di quanti pixel cade una pallina lasciata ferma nel primo secondo su A e su B (trascurare i rimbalzi)? Di quante volte è più grande l'accelerazione su B? Perché non basta dimezzare la costante 0,3 su B?

Esercizio 1.2: Che cosa si ferma (⋆)

Nella stessa Palline, per ciascuna situazione dire se le palline continuano a muoversi, e perché: (a) la finestra è ridotta a icona; (b) la finestra è coperta per metà da un'altra; (c) è aperto il menu degli scenari (`nelMenu` vero); (d) è in corso la riproduzione e si preme pausa; (e) si tiene premuto il pulsante del mouse sulla barra del titolo e si trascina la finestra.

Esercizio 1.3: Otto combinazioni (⋆)

I tre booleani `inPausa`, `inReplay` e `nelMenu` ammettono otto combinazioni. Elencare quelle che hanno senso per il giocattolo descritto all'inizio del capitolo, e per ognuna delle altre dire che cosa farebbe il programma se ci finisse dentro. Che cosa dovrebbe garantire il programma perché le combinazioni senza senso non si presentino mai?

Risposte

Risposta 1.1. Dopo n fotogrammi la velocità è $0{,}3\,n$ pixel per fotogramma e la posizione è scesa di $0{,}3 \cdot (1 + 2 + \cdots + n) = 0{,}3\,n(n+1)/2$ pixel. Su A, in un secondo, $n = 60$: circa 549 pixel. Su B, $n = 120$: circa 2178 pixel, quasi quattro volte tanto. L'accelerazione è 0,3 pixel per fotogramma al quadrato: in pixel al secondo al quadrato vale $0{,}3 \cdot 60^2 = 1080$ su A e $0{,}3 \cdot 120^2 = 4320$ su B, quattro volte. Dimezzare la costante su B dà 2160: ancora il doppio, perché la costante entra una volta nella velocità e la velocità entra a ogni fotogramma nella posizione; bisognerebbe dividerla per quattro, e comunque solo se B facesse *sempre* 120 fotogrammi al secondo, cosa che nessun computer garantisce. La soluzione è misurare il tempo, non contare i fotogrammi (capitolo 2).

Risposta 1.2. (a) Si fermano: Swing non chiama `paintComponent` di una finestra ridotta a icona, e senza disegno non c'è passo di fisica. (b) Continuano: la finestra è in parte visibile, `repaint()` viene servito e il metodo avanza tutte le palline, anche quelle nella metà coperta. (c) Si fermano, perché il blocco della fisica controlla `nelMenu`; ma la riproduzione, se è in corso, continua a inserire palline. (d) Le palline si fermano, perché il blocco della fisica controlla `inPausa`, ma la riproduzione continua: è il caso visto nel testo. (e) Dipende dal sistema: durante il trascinamento alcuni sistemi non chiedono ridisegni e il mondo si ferma, altri sì e continua. Un mondo il cui tempo dipende da che cosa fa il sistema con la finestra è il sintomo che il tempo è nel posto sbagliato.

Risposta 1.3. Hanno senso quattro stati: in corso (tutti falsi), in pausa, nel menu, in riproduzione. Le altre quattro combinazioni mescolano cose che non dovrebbero coesistere: pausa e riproduzione insieme (le palline ferme, le nuove che compaiono), menu e riproduzione (idem, con il menu davanti), pausa e menu (innocua per caso, ma all'uscita dal menu chi decide se si resta in pausa?), tutti e tre veri. Il programma dovrebbe garantire che in ogni istante *uno solo* dei quattro stati sia attivo, e che ogni passaggio dall'uno all'altro sia scritto in un posto solo: è esattamente ciò che fa un `enum` con quattro valori, o, quando ogni stato ha un comportamento suo, il pattern State del capitolo 5.

Capitolo 2

Il mondo e la sua immagine: MVC

La Palline del capitolo 1 teneva tutto in un pannello: le palline, la fisica, il disegno, il tempo, la riga di stato. Questo capitolo risponde alla prima delle tre domande, dove vive il mondo e chi lo disegna, e comincia a rispondere alla seconda, come scorre il tempo. Si parte dal mondo come insieme di oggetti Java che non sanno niente di finestre, si costruisce una View che lo disegna e basta, e un Controller che traduce il mouse e fa avanzare il tempo misurandolo. Alla fine Palline cade con la stessa gravità su ogni computer, continua a cadere anche se la finestra è coperta, e si può provare con un test, senza una finestra. Non fa ancora molto altro: la riga di stato e il suono arrivano con gli eventi del capitolo 3, la fisica fatta bene con il capitolo 4.

2.1 Il mondo come oggetti

Che cos'è Palline, se si toglie la finestra? Una scatola larga e alta, con un pavimento; dentro, delle palline, ciascuna con una posizione, una velocità e un raggio; una gravità che tira in basso; e il tempo che passa. Nessuna di queste cose è un pixel. Una pallina non è larga "trenta pixel" più di quanto un tavolo sia largo trenta pixel: è larga mezzo metro, e quanti pixel occupi dipende dalla finestra in cui la si guarda, che potrebbe essere grande o piccola, o non esserci affatto. La prima decisione è quindi di misura: il mondo si misura in *metri* e *secondi*, con l'origine in basso a sinistra e l'asse y verso l'alto, come in fisica. I pixel li vedremo nella View.

La seconda decisione è che le posizioni e le velocità sono *vettori*, coppie di numeri che si sommano fra loro e si moltiplicano per uno scalare. Scriverle come due `double` separati porta a duplicare ogni formula; usare `java.awt.Point` porta nel mondo una classe di AWT, in pixel interi, con i campi pubblici mutabili. Un record di due `double`, con i quattro metodi che servono, è la scelta giusta:

```
public record Vec2(double x, double y) {

    public static final Vec2 ZERO = new Vec2(0, 0);

    public Vec2 piu(Vec2 v)   { return new Vec2(x + v.x, y + v.y); }
    public Vec2 meno(Vec2 v)  { return new Vec2(x - v.x, y - v.y); }
    public Vec2 per(double k) { return new Vec2(x * k, y * k); }
    public double modulo()    { return Math.hypot(x, y); }
}
```

Un record è immutabile: `piu` non cambia il vettore su cui è chiamato, restituisce un vettore nuovo. Nel codice della fisica questo fa sì che la posizione di una pallina si possa passare in giro, confrontare e salvare senza timore che qualcuno la modifichi di nascosto; il costo, un oggetto nuovo a ogni somma, è irrilevante per il collettore di Java, e per un gioco con qualche centinaio di palline non si misura.

Una pallina è una posizione, una velocità e un raggio. La posizione e la velocità cambiano nel tempo, il raggio no. La classe espone lo stato in lettura e due operazioni che lo cambiano, visibili solo nel package, perché a muovere una pallina è il mondo e nessun altro:

```
public class Pallina {

    private Vec2 posizione;        // origine in basso a sinistra, y in su
    private Vec2 velocita = Vec2.ZERO;
    private final double raggio;

    public Pallina(Vec2 posizione, double raggio) {
        this.posizione = posizione;
        this.raggio = raggio;
    }

    public Vec2 posizione() { return posizione; }
    public Vec2 velocita()  { return velocita; }
    public double raggio()  { return raggio; }

    /** Un passo di dt secondi: prima la velocita, poi la posizione. */
    void stepNext(double dt, Vec2 g) {
        velocita = velocita.piu(g.per(dt));
        posizione = posizione.piu(velocita.per(dt));
    }

    /** Rimbalzo ingenuo: a terra, e velocita verticale invertita. */
    void rimbalzaSulPavimento(double restituzione) {
        posizione = new Vec2(posizione.x(), raggio);
        velocita = new Vec2(velocita.x(), -velocita.y() * restituzione);
    }
}
```

Il metodo `stepNext` è la fisica di questo capitolo, tutta in due righe: in `dt` secondi la gravità $\vec{g}$ cambia la velocità di $\vec{g}\,dt$, e la velocità cambia la posizione di $\vec{v}\,dt$. L'ordine, prima la velocità e poi la posizione con la velocità appena aggiornata, non è casuale: il capitolo 4 spiega perché l'ordine inverso fa crescere l'energia da sola. Il rimbalzo riporta la pallina appoggiata al pavimento (il centro a un raggio da terra) e inverte la componente verticale della velocità, ridotta del fattore di *restituzione*: 1 sarebbe un rimbalzo perfetto, 0 una pallina che si spiaccica. È ingenuo, e lo diciamo subito: una pallina ferma a terra riceve comunque la gravità a ogni passo, scende sotto il pavimento di una frazione di millimetro, viene riportata su e rilanciata con una velocità minuscola. Non affonda, come quella del capitolo 1, ma non si ferma mai davvero. Lo si vedrà, e lo si sistemerà, nel capitolo 4; per ora basta che sia nel posto giusto.

Il mondo tiene le palline e le dimensioni della scatola, sa aggiungere una pallina e sa far passare il tempo:

```java
public class Mondo {

    public static final Vec2 G = new Vec2(0, -9.81);     // m/s^2

    private final double larghezza, altezza;             // metri
    private final List<Pallina> palline = new ArrayList<>();

    public Mondo(double larghezza, double altezza) {
        this.larghezza = larghezza;
        this.altezza = altezza;
    }

    public double larghezza() { return larghezza; }
    public double altezza()   { return altezza; }

    public List<Pallina> palline() {
        return Collections.unmodifiableList(palline);
    }

    /** Lascia una pallina ferma in un punto: da qui in poi cade da sola. */
    public void lascia(Vec2 punto, double raggio) {
        palline.add(new Pallina(punto, raggio));
    }

    /** Fa passare dt secondi: le palline cadono e rimbalzano. */
    public void stepNext(double dt) {
        for (Pallina p : palline) {
            p.stepNext(dt, G);
            if (p.posizione().y() < p.raggio())
                p.rimbalzaSulPavimento(0.8);
        }
    }
}
```

Tre cose da notare. La lista delle palline esce in una vista non modificabile: chi guarda il mondo può scorrerla, non può aggiungere o togliere palline; per farlo deve passare da `lascia`, e il mondo resta padrone del proprio stato. Le operazioni hanno i nomi del dominio (`lascia`, `stepNext`) e non quelli dell'interfaccia (`mousePressed`, `tick`): il mondo non sa che esiste un mouse. E in tutto il file non compare nessun `import` da `java.awt` o `javax.swing`.

Definizione 2.1: Model

Il *Model* di un programma interattivo è l'insieme degli oggetti che rappresentano ciò che il programma *è* (il mondo, i dati, le regole), con le operazioni che li fanno cambiare, e nient'altro: non come viene mostrato, non come l'utente lo comanda. Un Model ben fatto non dipende da nessuna libreria grafica, si può costruire e far evolvere in un `main` senza finestre, e si può provare con un test automatico.

Attenzione 2.1: Il `Point` nel Model

La tentazione è usare le classi che ci sono: `java.awt.Point` per la posizione, `java.awt.Color` per il materiale, `Rectangle` per la scatola. Il Model compila e funziona, ma è legato ad AWT in tre modi che si pagano dopo. `Point` ha coordinate intere, e una pallina che si muove di 0,4 pixel per fotogramma non si muove mai; i campi `x` e `y` sono pubblici e mutabili, e chiunque riceva la posizione può cambiarla; e un Model che importa AWT non gira più su un server senza schermo, non si prova senza una finestra e non si può mostrare con un'altra libreria. La regola è secca: **nel Model non si importa né `java.awt` né `javax.swing`**. Il colore non è una proprietà della pallina ma del modo di disegnarla: la pallina avrà un *materiale*, e sarà la View a decidere che la gomma è arancione.

2.2 La View: disegnare ciò che c'è

La View è un pannello che riceve il mondo nel costruttore e, quando Swing glielo chiede, lo disegna com'è in quel momento. Ha due compiti soli: tradurre i metri in pixel e disegnare. Il primo è la trasformazione di coordinate di *Interfacce grafiche con Swing*, con una scelta in più: la scala si ricalcola a ogni disegno dalle dimensioni correnti del pannello, in modo che il mondo intero stia dentro senza deformarsi, qualunque sia la forma della finestra.

```
public class PnlMondo extends JPanel {

    private static final int BORDO = 10;          // pixel attorno al mondo
    private final Mondo mondo;

    public PnlMondo(Mondo mondo) {
        this.mondo = mondo;
        setBackground(new Color(0x101820));
    }

    /** Pixel per metro: il mondo intero sta nel pannello, senza deformarlo. */
    private double scala() {
        return Math.min((getWidth() - 2 * BORDO) / mondo.larghezza(),
                        (getHeight() - 2 * BORDO) / mondo.altezza());
    }

    private double worldToScreenX(double x) { return BORDO + x * scala(); }
    private double worldToScreenY(double y) { return getHeight() - BORDO - y *
    scala(); }

    /** Dal pixel al punto del mondo: serve a chi riceve il mouse. */
    public Vec2 screenToWorld(Point p) {
        return new Vec2((p.x - BORDO) / scala(),
                        (getHeight() - BORDO - p.y) / scala());
    }
```

I due metodi `worldToScreenX` e `worldToScreenY` portano un punto del mondo sullo schermo, e il secondo inverte l'asse, perché nel mondo y cresce verso l'alto e nei pixel verso il basso. `screenToWorld` fa la strada inversa, e sarà l'unica cosa che il Controller chiede alla View: dove ha cliccato l'utente, in metri. Il disegno è la seconda metà della classe:

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                        RenderingHints.VALUE_ANTIALIAS_ON);
    g2.setColor(Color.GRAY);
    g2.draw(new Line2D.Double(worldToScreenX(0), worldToScreenY(0),
    worldToScreenX(mondo.larghezza()), worldToScreenY(0)));
    g2.setColor(new Color(0xE0A020));
    for (Pallina p : mondo.palline()) {
        double r = p.raggio() * scala();
        g2.fill(new Ellipse2D.Double(worldToScreenX(p.posizione().x()) - r,
                                     worldToScreenY(p.posizione().y()) - r, 2 *
    r, 2 * r));
    }
}
```

Il pavimento è una linea a $y = 0$; ogni pallina è un'ellisse centrata sulla sua posizione, con il raggio portato in pixel. Il metodo *legge* il mondo (la lista, le posizioni, i raggi) e non vi scrive mai; non chiama `stepNext`, non tiene una copia delle palline, non sa quanto tempo è passato. Se Swing lo chiama dieci volte di seguito senza che il mondo sia cambiato, disegna dieci volte la stessa immagine, ed è giusto così.

Definizione 2.2: View

La *View* è la parte del programma che mostra il Model: lo legge e lo traduce in ciò che l'utente vede (pixel, testo, suono). Non tiene i dati che mostra, non li modifica, e non decide che cosa significano le azioni dell'utente. In Swing una View è tipicamente un componente che ridefinisce `paintComponent`, oppure un componente standard (un'etichetta, una barra) che qualcuno aggiorna dal Model.

Attenzione 2.2: La fisica in `paintComponent`

È la tentazione opposta a quella del `Point`: "tanto `paintComponent` viene chiamato a ogni fotogramma, facciamo avanzare il mondo lì". È la Palline del capitolo 1, con tutti i suoi difetti: il mondo si ferma quando la finestra è coperta, perché Swing non disegna ciò che non si vede; va a scatti quando Swing fonde due richieste di ridisegno in una, perché si perde un passo; e con due View sullo stesso mondo il tempo passa due volte. `paintComponent` è un metodo che Swing chiama *quando vuole*, anche zero volte, anche cento: può solo leggere.

2.3 Il Controller e il tempo

Resta da collegare le due parti, e da far passare il tempo. Lo fa il Controller, la terza classe, che conosce sia il mondo sia la View: registra sulla View un listener del mouse che traduce il click in un'operazione sul mondo, e possiede il `javax.swing.Timer` che scandisce il ciclo.

```
public class Controller {

    private final Mondo mondo;
    private final PnlMondo view;
    private final Timer timer = new Timer(16, e -> stepNext());
    private long ultimo;                          // nanoTime() del passo prima

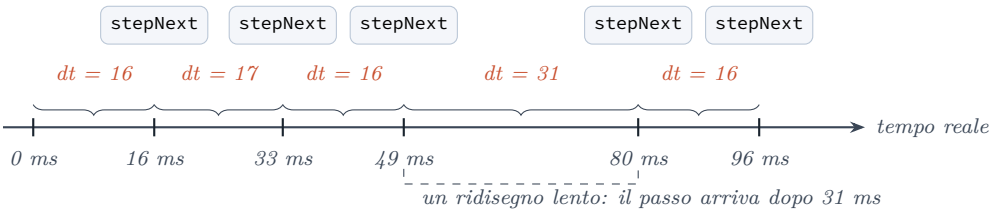
    public Controller(Mondo mondo, PnlMondo view) {
        this.mondo = mondo;
        this.view = view;
        view.addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                mondo.lascia(view.screenToWorld(e.getPoint()), 0.25);
            }
        });
    }

    public void avvia() {
        ultimo = System.nanoTime();
        timer.start();
    }

    /** Un passo del ciclo: il tempo vero, il mondo avanza, la View si rifa. */
    private void stepNext() {
        long adesso = System.nanoTime();
        double dt = (adesso - ultimo) / 1e9;             // secondi
        ultimo = adesso;
        mondo.stepNext(dt);
        view.repaint();
    }
}
```

Il listener del mouse è di tre righe e non disegna niente: chiede alla View di tradurre il pixel in metri, e dice al mondo di lasciare lì una pallina di 25 centimetri. È tutto ciò che il Controller sa del mouse. Il metodo `passo`, chiamato dal timer, è il ciclo del gioco dell'immagine guida: fa avanzare il mondo e chiede alla View di ridisegnarsi. La parte che merita attenzione è il `dt`.

Il timer è stato creato con un periodo di 16 millisecondi, circa sessanta volte al secondo. Sarebbe comodo scrivere `mondo.stepNext(0.016)`, ed è l'errore del capitolo 1 in una forma più sottile. Il periodo del timer è una *richiesta*: il timer mette un evento in coda ogni 16 ms, ma l'evento viene eseguito quando il thread degli eventi è libero, e se il passo precedente, o un ridisegno, o un altro evento hanno preso 30 ms, il passo arriva dopo 30 ms. Su un computer carico i passi si diradano; con la finestra trascinata, su alcuni sistemi, si fermano per mezzo secondo. Se il mondo avanzasse di 16 ms a ogni passo, il suo tempo rallenterebbe quando il computer rallenta: le palline cadrebbero più piano sotto carico.



Il timer chiede 16 ms, il thread degli eventi ne concede quanti può: ogni passo misura il tempo passato davvero e lo dà al mondo.

La soluzione è *misurare*. `System.nanoTime()` restituisce un contatore di nanosecondi che cresce con il tempo reale, indipendente dall'orologio del sistema (che l'utente può cambiare) e con una precisione più che sufficiente. Il Controller ricorda il valore letto al passo precedente, e a ogni passo dà al mondo la differenza in secondi: 0,016 quando tutto va bene, 0,031 quando un ridisegno è stato lento, e il mondo avanza di quanto deve. Il mondo, a sua volta, non sa che esiste un timer: riceve un intervallo di tempo e lo applica.

Osservazione 2.1

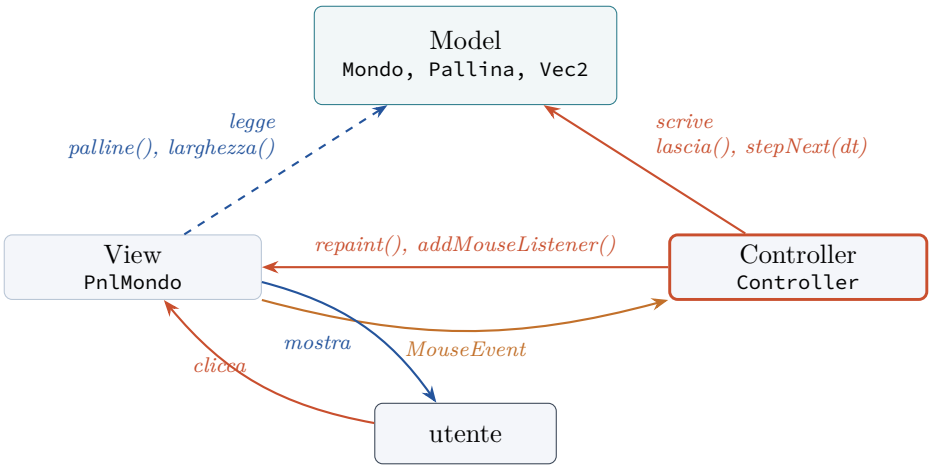
Il tempo del mondo e il tempo dello schermo sono due cose diverse, e il Controller è l'unico che le vede entrambe. Il tempo dello schermo è fatto di fotogrammi, tanti quanti il computer ne regge; il tempo del mondo è fatto di secondi, e deve essere lo stesso su ogni computer. Finché il passo è piccolo, dare al mondo il tempo misurato basta a tenerli separati. Quando il passo diventa grande (dopo una pausa, con la finestra trascinata, su un computer lentissimo) la fisica con un `dt` di mezzo secondo fa cose strane: una pallina attraversa il pavimento in un passo solo. È il problema del capitolo 4, e la soluzione, il passo fisso con l'accumulatore, sta sempre qui, nel Controller.

Attenzione 2.3: Il `dt` del primo passo

Se `ultimo` non viene inizializzato in `avvia`, al primo passo vale zero e `dt` è il valore intero di `nanoTime()` in secondi: un numero senza significato (il contatore parte da un'origine arbitraria, spesso l'avvio del sistema), che può valere ore o giorni. Il mondo avanza di giorni in un colpo e le palline finiscono a chilometri sotto il pavimento. Lo stesso succede se il timer viene fermato e fatto ripartire senza riallineare `ultimo`: il passo che segue la ripresa contiene tutta la pausa. La regola: **`ultimo` si legge nel momento in cui il timer parte**, e ogni volta che riparte.

2.4 Il triangolo: chi parla con chi

Le tre classi di Palline sono i tre ruoli di un'architettura che ha un nome dal 1979, quando fu descritta per Smalltalk: *Model-View-Controller*, MVC. Il nome è noto a tutti e il significato un po' meno, perché ogni libreria lo declina a modo suo. Qui lo definiamo per come lo usiamo, e la definizione è soprattutto un elenco di frecce permesse.



Le frecce permesse: il Controller scrive nel Model e comanda la View; la View legge il Model; il Model non conosce nessuno dei due.

Definizione 2.3: Model-View-Controller

Model-View-Controller è la divisione di un programma interattivo in tre ruoli. Il *Model* rappresenta il dominio e lo fa evolvere (definizione 2.1). La *View* legge il Model e lo mostra (definizione 2.2). Il *Controller* riceve l'input dell'utente dalla View, lo traduce in operazioni sul Model, fa scorrere il tempo e dice alla View quando aggiornarsi. Il Model non conosce né la View né il Controller; la View non modifica il Model; il Controller è l'unico che scrive nel Model. Un programma può avere più View sullo stesso Model, e più coppie Model-View-Controller per parti diverse.

In forma di tabella, per ciascun ruolo che cosa può fare e che cosa non deve fare:

Chi	**Che cosa fa**	**Che cosa non fa**
Model	tiene lo stato del dominio; lo fa evolvere (`stepNext`, `lascia`); dal capitolo 3, annuncia i cambiamenti	importare `java.awt` o `javax.swing`; conoscere View e Controller; disegnare; sapere che esiste un mouse
View	legge il Model e lo disegna; traduce fra metri e pixel; riceve l'input grezzo (i listener registrati dal Controller)	modificare il Model; tenere copie dei suoi dati; decidere che cosa *significa* un click; far passare il tempo
Controller	crea e collega Model e View; traduce l'input in operazioni sul Model; misura il tempo e chiama `stepNext`; chiede il ridisegno	disegnare; calcolare la fisica; tenere lo stato del dominio

Le regole hanno un perché ciascuna, e conviene dirlo, perché nei programmi veri le si infrange sempre con un buon motivo apparente. Il Model non conosce la View perché altrimenti non si può provare senza di lei, né averne due. La View non modifica il Model perché altrimenti due View litigano, e perché `paintComponent` viene chiamato quando Swing vuole. Il Controller è l'unico che scrive perché così ogni cambiamento del mondo ha un'origine sola, e quando qualcosa va storto c'è un posto solo in cui guardare. "Non tenere copie" vale per la View perché una copia va tenuta aggiornata, e prima o poi non lo è: la View legge il Model *nel momento in cui disegna*, e non può essere in ritardo.

2.4.1 Più View, e un Model in prestito

Che il Model non conosca la View ha una conseguenza pratica immediata: le View possono essere più d'una. Un secondo `PnlMondo` sullo stesso `Mondo`, in un'altra finestra, mostra le stesse palline da un'altra scala; una View testuale stampa a console la pallina più alta; la riga di stato è una View di un solo numero, quante palline ci sono. Nessuna delle tre sa delle altre, e il mondo non sa di nessuna. L'unico che le conosce tutte è il Controller, che a ogni passo chiede a ciascuna di aggiornarsi: funziona, e nel capitolo 3 vedremo come farlo meglio.

C'è anche una conseguenza sul tipo. La View usa del mondo solo tre metodi di lettura: `larghezza`, `altezza`, `palline`. Non le serve `lascia`, e non dovrebbe nemmeno poterlo chiamare. Quando il Model cresce, conviene dichiarare un'interfaccia con i soli metodi di lettura e darla alla View al posto della classe intera: il compilatore impedisce alla View di scrivere, e il Controller può passare alla View un mondo diverso (un mondo registrato, un mondo di prova) purché esponga quei tre metodi. Per Palline, oggi, sarebbe una cerimonia; è il genere di cosa che si aggiunge quando serve, sapendo che la struttura lo permette.

2.4.2 MVC dentro Swing

Lo schema non è un'invenzione nostra applicata a Swing dall'esterno: Swing stesso è costruito così, in piccolo, dentro ogni componente. Uno `JSlider` non tiene il proprio valore: lo tiene un `BoundedRangeModel`, e lo slider lo legge quando si disegna; cambiando il Model lo slider si aggiorna, e due slider sullo stesso Model si muovono insieme. Chi ha letto *Interfacce grafiche con Swing* ha già visto il Model di una tendina e di una lista. La tabella riassume le coppie principali.

Componente	Model	Che cosa tiene
`JSlider`, `JProgressBar`	`BoundedRangeModel`	valore, minimo, massimo
`JComboBox`, `JList`	`ComboBoxModel`, `ListModel`	le voci, la selezione
`JTable`	`TableModel`	righe, colonne, celle
`JTextField`, `JTextArea`	`Document`	il testo
`JButton`, `JCheckBox`	`ButtonModel`	premuto, selezionato, abilitato

In Swing il componente è insieme View e Controller (riceve il mouse e aggiorna il proprio Model), e il Model avvisa il componente con un evento quando cambia. È un MVC in cui la View ascolta il Model: lo vedremo nel capitolo 3, e vedremo che per il mondo di un gioco la variante "il Controller chiede il ridisegno a ogni passo" e la variante "la View ascolta il Model" servono entrambe, in momenti diversi.

2.5 Il premio: un mondo senza finestra

Il primo test della separazione, dicevamo nel capitolo 1, è che il mondo giri senza una finestra. Facciamolo. Un `main` crea il mondo, lascia una pallina a cinque metri d'altezza e fa passare tre secondi a passi di un centesimo, stampando ogni mezzo secondo:

```java
public class SenzaFinestra {

    public static void main(String[] args) {
        Mondo mondo = new Mondo(8, 6);
        mondo.lascia(new Vec2(4, 5), 0.25);
        Pallina p = mondo.palline().get(0);
        for (int passo = 1; passo <= 300; passo++) {
            mondo.stepNext(0.01);
            if (passo % 50 == 0)
                System.out.printf("t = %.1f s   y = %5.2f m   vy = %6.2f m/s%n",
                        passo / 100.0, p.posizione().y(), p.velocita().y());
        }
    }
}
```

```
t = 0,5 s   y =  3,75 m   vy =  -4,90 m/s
t = 1,0 s   y =  0,40 m   vy =   7,49 m/s
t = 1,5 s   y =  2,90 m   vy =   2,59 m/s
t = 2,0 s   y =  2,94 m   vy =  -2,32 m/s
t = 2,5 s   y =  0,53 m   vy =  -7,22 m/s
t = 3,0 s   y =  1,99 m   vy =   1,58 m/s
```

Nessuna finestra, nessun timer, nessun thread degli eventi: il mondo esiste e si evolve da solo, e i numeri sono quelli della fisica. A mezzo secondo la pallina è scesa di $gt^2/2 = 1{,}23$ metri, a 3,77; a un secondo ha già toccato terra (il centro arriva a 0,25 metri dopo 0,98 secondi) ed è in risalita con la velocità ridotta del 20%; a due secondi è vicina al vertice del primo rimbalzo. Questo `main` è già una seconda View, testuale, del mondo, e ha un vantaggio sulla finestra: si può leggere con calma.

Il passo in più è trasformare la lettura in una verifica automatica. Con JUnit 5, che ogni ambiente di sviluppo include, un test è un metodo che costruisce il mondo, lo fa evolvere e controlla il risultato con un'asserzione:

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;
import org.junit.jupiter.api.Test;

class MondoTest {

    @Test
    void unaPallinaLasciataCadeComeDiceLaFisica() {
        Mondo mondo = new Mondo(8, 6);
        mondo.lascia(new Vec2(4, 5.5), 0.25);
        for (int i = 0; i < 1000; i++)          // un secondo a passi di 1 ms
            mondo.stepNext(0.001);
        double y = mondo.palline().get(0).posizione().y();
        assertEquals(5.5 - 9.81 / 2, y, 0.01);   // y0 - g t^2 / 2
    }

    @Test
    void nessunaPallinaFinisceSottoIlPavimento() {
        Mondo mondo = new Mondo(8, 6);
        for (int i = 0; i < 20; i++)
            mondo.lascia(new Vec2(0.4 * i, 1 + 0.2 * i), 0.1 + 0.01 * i);
        for (int i = 0; i < 5000; i++)
            mondo.stepNext(0.001);
        for (Pallina p : mondo.palline())
            assertTrue(p.posizione().y() >= p.raggio() - 1e-9,
                    "pallina sotto il pavimento: y = " + p.posizione().y());
    }
}
```

Il primo test lascia una pallina da 5,5 metri e dopo un secondo (mille passi da un millesimo) si aspetta di trovarla a $5{,}5 - g/2 = 0{,}595$ metri, con una tolleranza di un centimetro: il passo piccolo serve a restare vicini alla formula esatta, e quanto piccolo debba essere lo dirà il capitolo 4. Il secondo lascia venti palline e dopo cinque secondi controlla che nessuna sia sotto il pavimento. Si eseguono in un decimo di secondo, senza aprire nulla, e si rieseguono ogni volta che la fisica cambia: quando nel capitolo 4 riscriveremo il rimbalzo, questi due test diranno subito se abbiamo rotto qualcosa.

Osservazione 2.2

I test sono il motivo più concreto per separare il Model, e il criterio più onesto per sapere se lo si è fatto. Un Model che per essere provato ha bisogno di una finestra, di un click simulato e di un occhio che guardi lo schermo non è un Model: è un pannello con un altro nome. Nel seguito ogni versione di Palline avrà i suoi test sul mondo, e nessun test sulla View.

2.6 L'esempio guida: Palline, prima versione

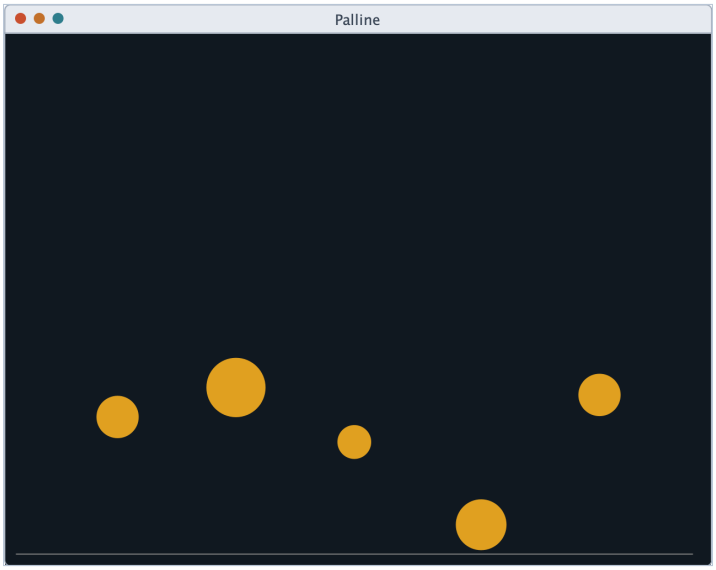
Le cinque classi viste (`Vec2`, `Pallina`, `Mondo`, `PnlMondo`, `Controller`) si montano in un `main` che è anche il punto in cui si vede l'architettura intera, in dieci righe:

```
public class PallineApp {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            Mondo mondo = new Mondo(8, 6);               // metri
            PnlMondo view = new PnlMondo(mondo);
            Controller controller = new Controller(mondo, view);

            JFrame frame = new JFrame("Palline");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.add(view);
            frame.setSize(640, 480);
            frame.setVisible(true);
            controller.avvia();
        });
    }
}
```

Prima il mondo, poi la View che lo guarda, poi il Controller che li collega; la finestra per ultima, e il timer parte quando la finestra è visibile. Il mondo è largo otto metri e alto sei: la finestra può avere qualunque dimensione, e il mondo ci starà dentro intero, con il pavimento in basso.



Palline, prima versione: cinque palline lasciate in punti diversi, dopo poco più di un secondo.

Lanciata, Palline fa poco: si clicca, una pallina cade, rimbalza, e resta a tremolare impercettibilmente sul pavimento. Ma fa quel poco bene. Cade con la stessa gravità su ogni computer, perché il tempo è misurato. Continua a cadere con la finestra coperta, perché il tempo passa nel Controller e non nel disegno. Si può provare senza finestra. E per aggiungere ciò che manca c'è un posto per ogni cosa: i rimbalzi fatti bene stanno nel mondo, i colori nella View, i tasti nel Controller.

Ricetta: dividere un programma interattivo in tre parti

1. **Il dominio prima di tutto:** elencare che cosa il programma *è* (le palline, la scatola, la gravità) e che cosa gli succede (una pallina lasciata, il tempo che passa), e scriverlo come classi e record senza nessun `import` grafico, con le unità di misura del dominio.
2. **Un metodo per far passare il tempo:** `stepNext(dt)` nel Model, con `dt` in secondi; nessun riferimento a fotogrammi o a timer.
3. **Una View che legge e basta:** un pannello che riceve il Model nel costruttore, lo traduce in pixel e lo disegna in `paintComponent`; espone la traduzione inversa (`screenToWorld`) per chi riceve il mouse.
4. **Un Controller che collega:** registra i listener sulla View, traduce l'input in operazioni del Model, possiede il timer, misura il tempo con `nanoTime` e a ogni passo chiama `stepNext` e `repaint`.
5. **Un test prima della finestra:** un `main` o un test JUnit che fa evolvere il Model e controlla i numeri. Se non si riesce a scriverlo, il Model non è ancora separato.

Esercizi

Esercizio 2.1: Metri e pixel (⋆)

Un `Mondo` di 8 per 6 metri è mostrato da un `PnlMondo` di 640 per 480 pixel (`BORDO` = 10). Quanti pixel vale un metro? Dove finisce sullo schermo il centro di una pallina in $(4, 5)$? E se la finestra viene allargata a 1000 per 480, quanti pixel di sfondo restano a destra del mondo?

Esercizio 2.2: Frecce permesse (⋆)

Per ciascuna chiamata dire se rispetta le regole della tabella del paragrafo 2.4, e se no perché: (a) `PnlMondo` chiama `mondo.lascia(...)` in `mousePressed`; (b) `Controller` chiama `view.repaint()`; (c) `Mondo` chiama `view.repaint()` alla fine di `stepNext`; (d) `PnlMondo` chiama `mondo.palline()` in `paintComponent`; (e) `Mondo` dichiara un campo `java.awt.Color colore` in `Pallina`; (f) `Controller` chiama `mondo.stepNext(dt)`.

Esercizio 2.3: Il passo mancante (⋆)

Nel `Controller` si toglie la riga `ultimo = System.nanoTime()` da `avvia`. Che cosa vale `dt` al primo passo, all'incirca, e dove finiscono le palline? Che cosa succede invece se `ultimo` è inizializzato correttamente ma il timer viene fermato per dieci secondi e poi fatto ripartire con `timer.start()` senza passare da `avvia`?

Esercizio 2.4: Due View (⋆⋆)

Si creano due `PnlMondo` sullo stesso `Mondo`, in due finestre, e un solo `Controller` costruito con la prima View. Che cosa mostra la seconda finestra nel tempo? Quando si aggiorna? Dire in una riga come correggere il Controller, e perché il Model non ha bisogno di cambiare.

Esercizio 2.5: Un risparmio apparente (⋆⋆)

Per "risparmiare una classe", un collega sposta la misura del tempo e la chiamata a `mondo.stepNext(dt)` dentro `paintComponent` di `PnlMondo`, e lascia al timer il solo `repaint()`. Il tempo è misurato con `nanoTime`, quindi le palline cadono alla velocità giusta. Elencare almeno tre cose che comunque si rompono.

I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.

Esercizio 2.6: Un contatore in tre parti (⋆)

Un Model `Contatore` con `valore()`, `incrementa()`, `decrementa()` e `azzera()`, limitato fra 0 e 100. Due View sullo stesso Model: un'etichetta grande con il numero e una `JProgressBar` che lo mostra come percentuale. Un Controller con tre pulsanti che chiamano il Model e poi chiedono alle due View di aggiornarsi. Il Model non deve importare niente di Swing, e deve avere un test (o un `main`) che lo prova senza finestra.

Soluzione: progetto `sol-dxtr-02-c02`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 2.4: Il valore nella View

L'errore tipico è tenere il numero in un campo dell'etichetta o del Controller e "aggiornare il Model" da lì: alla prima incoerenza (il pulsante che incrementa oltre 100) le due View mostrano numeri diversi. Il numero sta nel Model e in nessun altro posto; le View lo *leggono* quando si aggiornano.

Esercizio 2.7: Il semaforo pedonale (⋆⋆)

Un incrocio con un semaforo per le auto (verde, giallo, rosso) e uno per i pedoni (rosso, verde). Il Model `Semaforo` ha un metodo `stepNext(dt)` che fa scorrere il tempo e cambia le luci secondo un ciclo fisso (per esempio 20 s di verde auto, 3 di giallo, 12 di rosso auto e verde pedoni), e un metodo `chiamata()` che, se le auto hanno il verde da almeno 5 secondi, accorcia il verde a 2 secondi. Una View che disegna i due semafori con le luci accese e spente; un Controller con il timer, il tempo misurato, e un pulsante "Chiamata". Il Model si prova facendo passare 60 secondi a passi di un decimo e controllando la sequenza delle luci.

Soluzione: progetto `sol-dxtr-02-c02`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 2.5: Il tempo nel timer

Se il semaforo cambia luce "ogni n scatti del timer", il ciclo dura di più sui computer lenti e la prova senza finestra diventa impossibile. Il tempo entra nel Model solo attraverso `stepNext(dt)`, in secondi, e il Model conta i secondi.

Esercizio 2.8: Il gioco della vita (⋆⋆)

Il Model `Griglia` tiene una griglia di celle vive o morte su un toro (il bordo destro confina con il sinistro, quello in alto con quello in basso), con `inverti(riga, colonna)` e `generazione()`, che applica le regole di Conway: una cella viva con due o tre vicine vive sopravvive, una morta con esattamente tre vicine nasce, le altre muoiono o restano morte. La View disegna la griglia adattandola al pannello; il Controller inverte la cella sotto il click, e con il timer fa scattare una generazione ogni 0,2 secondi *di tempo misurato*, accumulando i `dt` dei passi. Il Model si prova con l'aliante, che dopo quattro generazioni è uguale a sé stesso spostato di una cella in diagonale.

Soluzione: progetto `sol-dxtr-02-c02`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 2.6: La griglia aggiornata sul posto

Applicare le regole cella per cella sulla stessa griglia che si sta leggendo è l'errore classico: le celle già aggiornate influenzano quelle dopo, e l'aliante si disfa. La generazione nuova si calcola in una griglia nuova, e si scambia alla fine.

Esercizio 2.9: Una seconda View, a parole (⋆⋆)

Aggiungere a Palline una seconda View, `ViewTestuale`, che non è un componente Swing: ha un metodo `aggiorna(Mondo)` che stampa a console una riga con il numero di palline, la quota della più alta e la velocità della più veloce. Il Controller la chiama una volta al secondo di tempo misurato, accumulando i `dt` come nel problema precedente, mentre continua a ridisegnare il pannello a ogni passo. Nessuna modifica a `Mondo` e a `PnlMondo`. *Suggerimento: la View testuale e il pannello sono due oggetti con due ritmi diversi, e il Controller è l'unico che li conosce entrambi.*

Soluzione: progetto `sol-dxtr-02-c02`, package `it.unibs.pajc.redolfi.e09`.

Risposte

Risposta 2.1. La scala è il minimo fra $(640-20)/8 = 77{,}5$ e $(480-20)/6 \approx 76{,}67$ pixel per metro: vince l'altezza, 76,67. Il centro di $(4, 5)$ finisce in $x = 10 + 4 \cdot 76{,}67 \approx 317$ e $y = 480 - 10 - 5 \cdot 76{,}67 \approx 87$ pixel dall'alto. Allargando a 1000 pixel la scala resta decisa dall'altezza (76,67, contro $980/8 = 122{,}5$): il mondo occupa $8 \cdot 76{,}67 \approx 613$ pixel e a destra restano $1000 - 10 - 613 = 377$ pixel di sfondo. Il mondo non si deforma: si allarga la cornice.

Risposta 2.2. (a) No: la View scrive nel Model e decide che cosa significa un click; è compito del Controller. (b) Sì: il Controller comanda la View. (c) No, due volte: il Model non conosce la View, e importerebbe Swing. (d) Sì: la View legge il Model mentre disegna. (e) No: il Model importa AWT; il colore è una scelta della View, la pallina ha al più un materiale. (f) Sì: è il Controller che fa scorrere il tempo.

Risposta 2.3. Con `ultimo` a zero, `dt` è `nanoTime()` diviso un miliardo: il contatore parte da un'origine arbitraria (spesso l'avvio della macchina), quindi `dt` vale tipicamente da minuti a giorni. Il primo `stepNext` applica la gravità per tutto quel tempo: le velocità diventano enormi, le posizioni finiscono a chilometri sotto il pavimento, il rimbalzo le riporta a terra con una velocità verticale gigantesca e le palline schizzano via dallo schermo. Nel secondo caso il passo che segue la ripresa ha `dt` di dieci secondi: le palline ferme ricevono dieci secondi di gravità in un colpo (velocità di 98 m/s verso il basso), attraversano il pavimento in un passo e rimbalzano verso l'alto come proiettili. In entrambi i casi la cura è la stessa: `ultimo` si riallinea ogni volta che il timer parte.

Risposta 2.4. La seconda finestra mostra il mondo com'era l'ultima volta che Swing ha deciso di ridisegnarla: all'apertura, quando viene coperta e scoperta, quando la si ridimensiona. Nel frattempo resta ferma, con le palline a mezz'aria, perché nessuno le chiede `repaint()`. Si corregge dando al Controller la lista delle View (o un secondo parametro) e chiamando `repaint` su ciascuna in `passo`. Il Model non cambia perché non ha mai saputo quante View lo guardano: è esattamente il premio del fatto che non le conosce. Nel capitolo 3 le View si registreranno da sole, e il Controller non dovrà nemmeno conoscerle.

Risposta 2.5. Almeno queste. Il mondo si ferma quando la finestra è coperta o ridotta a icona, perché `paintComponent` non viene chiamato (e alla riapertura il `dt` accumulato è enorme, con gli effetti della risposta precedente). Con due View il tempo passa due volte per fotogramma, e con nessuna View non passa affatto. Il test senza finestra diventa impossibile, perché `stepNext` viene chiamato solo da Swing. Il passo dipende da quando Swing decide di disegnare: due `repaint` ravvicinati vengono fusi in uno, e un ridisegno parziale (una zona scoperta) fa un passo di fisica intero. E la View scrive nel Model, cosa che la tabella vieta proprio perché produce tutto questo.

Capitolo 3

Il Model che avvisa: programmazione a eventi

La Palline del capitolo 2 ha un mondo separato dalla View, e un Controller che li collega. Manca ancora la terza delle tre domande del capitolo 1: come si parlano le parti. La riga di stato vuole sapere quante palline sono ferme, il suono vuole sapere quando c'è un urto e quanto è forte, lo slider della gravità vuole restare allineato con il mondo anche quando la gravità cambia per altre vie. Questo capitolo mostra come il Model *annuncia* ciò che gli succede a chi si è messo in ascolto, senza sapere chi sia: è il pattern Observer, che in Java si chiama programmazione a eventi. Chi ha letto *Interfacce grafiche con Swing* l'ha già incontrato dal lato di chi gli eventi li *riceve*, e ha scritto un componente che li emette; qui lo si vede dal lato di un Model che non è un componente, con gli strumenti della libreria e con le regole che evitano gli errori più comuni. Alla fine Palline ha la riga di stato, il *toc* a ogni rimbalzo, lo slider della gravità e il pulsante di pausa, e il mondo continua a non importare niente di Swing.

3.1 Chiedere o farsi avvisare

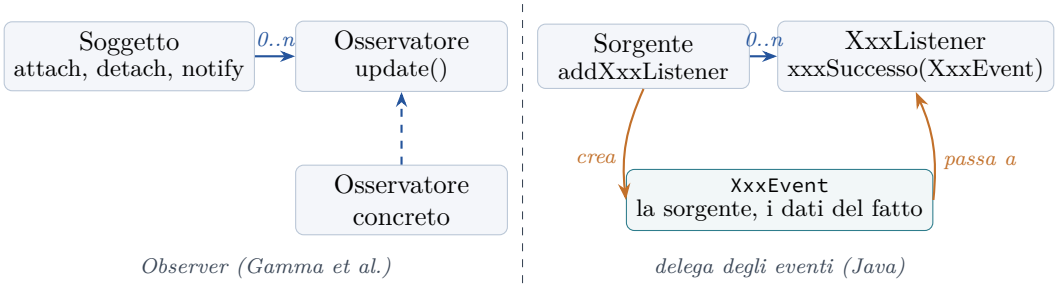
Partiamo dalla riga di stato: un'etichetta in fondo alla finestra che dice "5 palline, 2 ferme". Con gli strumenti del capitolo 2 c'è un modo ovvio di farla: il Controller, a ogni passo, dopo `stepNext` e prima di `repaint`, conta le palline e quelle ferme e scrive il testo. Funziona, e ha tre difetti. Il Controller deve conoscere l'etichetta, e ogni altra View che si aggiunge: diventa l'elenco di tutto ciò che va aggiornato. Il conteggio si rifà sessanta volte al secondo anche quando non cambia niente, e con mille palline sono sessantamila confronti al secondo per un testo che cambia una volta ogni tanto. E soprattutto non si estende al suono: un *toc* deve partire *quando* una pallina rimbalza, e a ogni passo il Controller non può sapere se nel passo appena fatto c'è stato un rimbalzo, a meno di non interrogare ogni pallina su che cosa le è successo, cioè di spostare nel Controller una contabilità che appartiene al mondo.

Questo modo si chiama *polling*, interrogazione: chi vuole sapere chiede, in continuazione. L'alternativa è che chi *sa* avvisi, quando c'è qualcosa da dire. Il mondo è l'unico che sa quando una pallina viene lasciata, rimbalza o si ferma, perché è lui a farlo succedere; se nel momento in cui lo fa succedere lo dice a chi si è dichiarato interessato, la riga di stato aggiorna il conteggio solo quando cambia, e il suono parte solo quando c'è un urto. Né la riga di stato né il suono hanno bisogno del Controller, e il Controller non ha bisogno di conoscerli.

Definizione 3.1: Observer

Il pattern *Observer* (osservatore) collega un oggetto che cambia, il *soggetto*, a un numero qualsiasi di oggetti che vogliono saperlo, gli *osservatori*, senza che il soggetto li conosca per tipo: il soggetto offre un metodo per registrarsi e uno per cancellarsi, tiene l'elenco degli osservatori registrati e, quando cambia, chiama su ciascuno un metodo convenuto. Gli osservatori dipendono dal soggetto; il soggetto dipende solo dall'interfaccia degli osservatori.

Java ha dato a questo pattern una forma precisa, che tutta la libreria usa: il *modello di delega degli eventi*. Il soggetto si chiama *sorgente*; ciò che è successo è descritto da un oggetto *evento*, che porta i dati del fatto; gli osservatori sono *listener* che implementano un'interfaccia con un metodo per ogni tipo di fatto; i metodi per registrarsi si chiamano `addXxxListener` e `removeXxxListener`. Un `JButton` è una sorgente, `ActionEvent` è l'evento, `ActionListener` l'interfaccia: lo schema che si usa ogni volta che si scrive `addActionListener`. La figura mette a confronto il pattern e la sua forma in Java.



Lo stesso pattern in due forme: a sinistra com'è descritto nel libro dei pattern, a destra come lo scrive Java, con l'evento che porta i dati.

La differenza che conta è l'oggetto evento. Nell'Observer classico l'osservatore riceve solo "sono cambiato" e deve tornare dal soggetto a chiedere che cosa; in Java l'evento porta con sé la sorgente e i dati del fatto (quale pulsante, quale tasto, quale pallina e a che velocità), e il listener ha tutto ciò che gli serve senza tornare indietro. È anche il motivo per cui un'interfaccia di ascolto ha spesso più metodi, uno per tipo di fatto, invece di un solo `update`.

3.2 Scrivere una sorgente di eventi

Il mondo ha tre cose da annunciare: una pallina è stata lasciata, una pallina ha rimbalzato, una pallina si è fermata. Il primo passo è decidere che cosa porta ciascun annuncio. Per la pallina lasciata e per quella ferma basta la pallina; per il rimbalzo serve anche la velocità con cui ha toccato terra, perché il suono la vuole, e serve nel momento dell'urto, perché un istante dopo la velocità è già quella di risalita. L'evento è quindi un record:

```
public record Rimbalzo(Pallina pallina, double velocita) {
}
```

Un record è la forma naturale di un evento: immutabile, con i dati del fatto e niente altro. Chi lo riceve può tenerlo, metterlo in una lista, stamparlo, e nessuno può cambiarlo dopo che è partito. Non estende `java.util.EventObject`, come fanno gli eventi di Swing: potrebbe, e in una libreria pubblica forse dovrebbe, ma qui il mondo è nostro e un record è più leggibile.

Il secondo passo è l'interfaccia di chi ascolta, con un metodo per fatto:

```
public interface MondoListener {

    default void pallinaLasciata(Pallina p) { }

    default void rimbalzo(Rimbalzo r) { }

    default void pallinaFerma(Pallina p) { }
}
```

I tre metodi hanno un corpo vuoto, con `default`: chi implementa l'interfaccia ridefinisce solo quelli che gli interessano. Il suono ridefinisce `rimbalzo`, la riga di stato `pallinaLasciata` e `pallinaFerma`, e nessuno è costretto a scrivere metodi vuoti. Swing, nato prima dei metodi `default`, risolve lo stesso problema con le classi *adapter* (`MouseAdapter`, `WindowAdapter`); con Java moderno l'interfaccia basta.

Il terzo passo è nel mondo: l'elenco dei listener, i due metodi per entrare e uscire, e la notifica nel punto in cui il fatto succede. La pallina lasciata è il caso più semplice:

```
private final List<MondoListener> listeners = new ArrayList<>();

public void addMondoListener(MondoListener l)    { listeners.add(l); }
public void removeMondoListener(MondoListener l) { listeners.remove(l); }

public void lascia(Vec2 punto, double raggio) {
    Pallina p = new Pallina(punto, raggio);
    palline.add(p);
    for (MondoListener l : List.copyOf(listeners))
        l.pallinaLasciata(p);
}
```

La pallina viene creata, *aggiunta alla lista*, e solo dopo annunciata: un listener che nel suo `pallinaLasciata` chiedesse `mondo.palline()` la troverebbe. È la prima regola, e la vedremo meglio fra poco. La notifica scorre una *copia* dell'elenco, `List.copyOf(listeners)`: se un listener, mentre viene avvisato, decide di cancellarsi ("avvisami la prima volta e basta"), modifica `listeners` mentre lo stiamo scorrendo, e senza la copia il `for` lancerebbe una `ConcurrentModificationException`. È lo stesso motivo per cui esiste `EventListenerList`, la classe che i componenti Swing usano per i loro listener e che *Interfacce grafiche con Swing* mostra all'opera: tiene listener di più tipi in un oggetto solo e restituisce una copia al momento della notifica. Per un Model con un solo tipo di listener una lista e una copia fanno lo stesso servizio con meno cerimonia; per un componente con `ActionListener`, `ChangeListener` e altri, `EventListenerList` è più comoda.

Osservazione 3.1

Il mondo è diventato una *stazione radio*: trasmette "pallina lasciata" a chiunque sia sintonizzato, e non sa se ad ascoltare ci sono un'etichetta, un suono, un registratore su file o nessuno. Il codice di `lascia` è lo stesso in tutti i casi. Chi scrive un listener nuovo non tocca il mondo; chi cambia il mondo non deve cercare tutti i posti che lo guardano. È la proprietà che nel capitolo 2 avevamo ottenuto per le View (il mondo non le conosce) estesa a tutto ciò che reagisce a un fatto.

3.3 Le proprietà: `PropertyChangeSupport`

Rimbalzi e palline ferme sono *fatti*: succedono in un istante e poi sono passati. La gravità e la pausa sono un'altra cosa: sono *proprietà* del mondo, valori che in ogni momento ci sono e che ogni tanto cambiano. Per le proprietà Java ha uno strumento pronto da trent'anni, nato con i JavaBeans e usato da Swing per tutte le proprietà dei componenti: `PropertyChangeSupport`, nel package `java.beans`, che non dipende da Swing e si può usare in un Model puro. Si tiene un'istanza nel mondo, si delegano a lei la registrazione e la cancellazione dei listener, e in ogni *setter* si annuncia il cambiamento con il nome della proprietà, il valore vecchio e quello nuovo:

```
private double gravita = 9.81;                          // m/s^2, in giu
private boolean inPausa;
private final PropertyChangeSupport proprieta = new PropertyChangeSupport(this);

public void addPropertyChangeListener(PropertyChangeListener l) {
    proprieta.addPropertyChangeListener(l);
}

public void removePropertyChangeListener(PropertyChangeListener l) {
    proprieta.removePropertyChangeListener(l);
}

public double gravita() { return gravita; }

public void setGravita(double g) {
    double vecchia = gravita;
    gravita = g;
    proprieta.firePropertyChange("gravita", vecchia, g);
}

public boolean inPausa() { return inPausa; }

public void setInPausa(boolean pausa) {
    boolean vecchia = inPausa;
    inPausa = pausa;
    proprieta.firePropertyChange("inPausa", vecchia, pausa);
}
```

Chi ascolta implementa `PropertyChangeListener`, che ha un metodo solo, `propertyChange`, ed è quindi una lambda; il metodo riceve un `PropertyChangeEvent`, che porta il nome della proprietà (`getPropertyName`), il valore vecchio e quello nuovo, come `Object`. Un listener che si registra riceve i cambiamenti di *tutte* le proprietà e distingue con il nome; chi vuole una sola proprietà può usare la variante di `addPropertyChangeListener` con il nome della proprietà come primo argomento.

C'è un dettaglio che `PropertyChangeSupport` fa da solo e che vale oro: **se il valore vecchio e quello nuovo sono uguali, non notifica nessuno**. È ciò che rende possibile legare una proprietà a un componente *nei due versi* senza cadere in un ciclo. Vediamolo con lo slider della gravità, nel Controller:

```
sldGravita.setValue((int) Math.round(mondo.gravita() * 100));
sldGravita.addChangeListener(
        e -> mondo.setGravita(sldGravita.getValue() / 100.0));
mondo.addPropertyChangeListener(e -> {
    if (e.getPropertyName().equals("gravita"))
        sldGravita.setValue((int) Math.round((Double) e.getNewValue() * 100));
});
```

Lo slider va da 0 a 2500 centesimi di metro al secondo quadrato. Quando l'utente lo muove, il suo `ChangeListener` scrive la gravità nel mondo; il mondo annuncia il cambiamento; il nostro `PropertyChangeListener` riporta il valore sullo slider, che però è già quello (`setValue` con il valore corrente non fa scattare il `ChangeListener`, come spiega *Interfacce grafiche con Swing*), e il giro si chiude. Quando invece la gravità cambia da un'altra parte (uno scenario che la imposta a quella lunare, un comando della tastiera), il mondo annuncia, lo slider si sposta, il suo `ChangeListener` chiama `setGravita` con lo stesso valore, e `PropertyChangeSupport`, trovando vecchio e nuovo uguali, non annuncia niente: il giro si chiude anche da questo lato. Il pulsante di pausa, un `JToggleButton`, si lega allo stesso modo, con `setSelected` da una parte e `setInPausa` dall'altra.

Attenzione 3.1: Il giro che non si chiude

Il meccanismo regge finché la conversione fra il componente e la proprietà è *esatta* nei due versi. Se lo slider fosse in metri interi e la gravità 9,81, lo slider riceverebbe 10, scriverebbe 10,0 nel mondo, il mondo annuncerebbe 9,81 → 10,0, lo slider riceverebbe di nuovo 10: si ferma, ma la gravità è cambiata senza che nessuno l'abbia chiesto. Con due conversioni che si arrotondano in modo diverso (uno `(int)` che tronca da una parte e un `Math.round` dall'altra) il valore può oscillare fra due arrotondamenti a ogni giro, e il giro non si chiude mai. La regola: **la proprietà sta nel Model nell'unità del Model, e il componente la rappresenta con una conversione invertibile**, come i centesimi dello slider.

Quando si usa l'uno e quando l'altro strumento, in una riga: `PropertyChangeSupport` per i *valori* che cambiano (una proprietà con nome, un vecchio e un nuovo), un'interfaccia di ascolto propria per i *fatti* che succedono (un rimbalzo, una mossa, una partita finita), con un record per i dati del fatto. Si possono mescolare nello stesso Model, come fa il mondo.

3.4 Le regole

Scrivere una sorgente di eventi è facile; scriverla in modo che non sorprenda chi ascolta richiede quattro regole, ciascuna con un errore che la motiva.

Si annuncia a stato coerente. Un listener che riceve un evento, nel suo metodo, legge il Model: conta le palline, ne guarda la posizione. Deve trovarlo in uno stato in cui tutto ciò che l'evento dice è già vero. Per questo `lascia` aggiunge la pallina alla lista *prima* di annunciarla, e per questo `stepNext` non annuncia i rimbalzi nel punto in cui li scopre, in mezzo al ciclo sulle palline, quando metà delle palline sono già avanzate e metà no, ma li raccoglie e li annuncia a passo concluso:

```
public void stepNext(double dt) {
    if (inPausa)
        return;
    Vec2 g = new Vec2(0, -gravita);
    List<Rimbalzo> rimbalzi = new ArrayList<>();
    List<Pallina> fermate = new ArrayList<>();
    for (Pallina p : palline) {
        if (p.ferma())
            continue;
        p.stepNext(dt, g);
        if (p.posizione().y() < p.raggio()) {
            double impatto = -p.velocita().y();
            p.rimbalzaSulPavimento(0.8);
            if (p.velocita().y() < SOGLIA_QUIETE) {
                p.fermati();
                fermate.add(p);
            } else {
                rimbalzi.add(new Rimbalzo(p, impatto));
            }
        }
    }
    for (MondoListener l : List.copyOf(listeners)) {    // a passo concluso
        for (Rimbalzo r : rimbalzi)
            l.rimbalzo(r);
        for (Pallina p : fermate)
            l.pallinaFerma(p);
    }
}
```

Il metodo è cresciuto rispetto al capitolo 2 anche per un altro motivo: una pallina che dopo il rimbalzo risale a meno di mezzo metro al secondo viene *fermata* (`fermati` la appoggia al pavimento e azzera la velocità) e da lì in poi `stepNext` la salta. È una soglia di quiete provvisoria, messa qui perché senza di essa le palline tremolanti del capitolo precedente produrrebbero centinaia di eventi di rimbalzo al secondo, e il *toc* diventerebbe il crepitio del capitolo 1; il capitolo 4 la sostituirà con la versione fatta bene. Si noti infine il `return` in testa quando il mondo è in pausa: in pausa il tempo non passa, e non parte nessun evento.

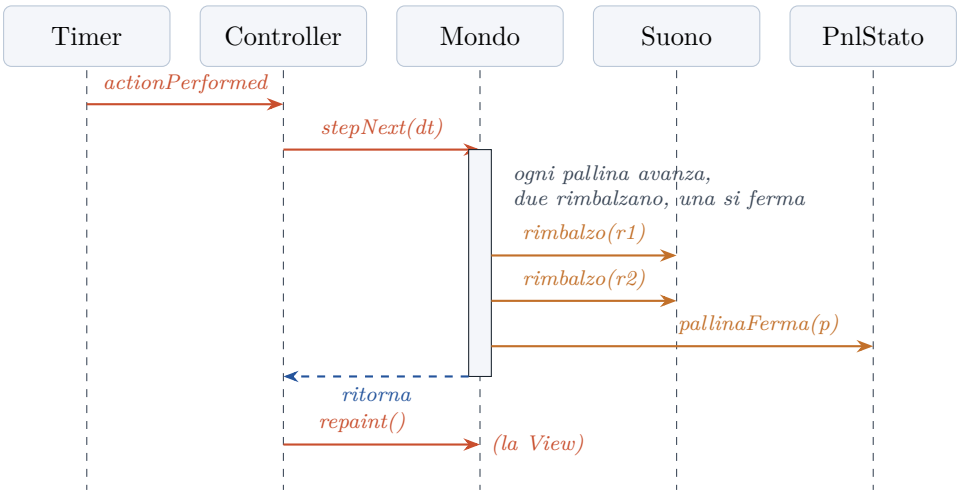
L'evento è immutabile e porta i dati, non il Model. `Rimbalzo` è un record con la pallina e la velocità d'impatto. Non porta il mondo intero (chi lo riceve ha già il mondo, se gli serve) e non è un oggetto che qualcuno potrebbe modificare dopo averlo ricevuto. Un listener che mette gli eventi in una lista per riprodurli più tardi, come il registratore del problema 3.9, può farlo solo perché sono immutabili.

Chi si cancella durante la notifica non rompe niente. È la copia della lista vista sopra. Un listener "una tantum" che nel suo metodo chiama `removeMondoListener(this)` è legittimo e frequente; senza la copia lancia un'eccezione dentro il Model, nel mezzo di `stepNext`, e il timer di Swing smette di chiamare il passo.

Chi non si cancella resta per sempre. Il rovescio: un listener registrato e dimenticato non viene mai liberato, perché il Model lo tiene nella sua lista, e continua a ricevere eventi. Una seconda finestra con la sua riga di stato, chiusa dall'utente, continuerebbe a contare

palline nel vuoto, e a tenere viva in memoria la finestra intera, con tutti i suoi componenti. La regola è che **chi si registra ha il dovere di cancellarsi** quando smette di esistere: una View in una finestra lo fa nel `windowClosed` della finestra, un oggetto con un ciclo di vita proprio in un suo metodo `chiudi`.

Un solo thread. Tutto ciò che abbiamo scritto gira sul thread degli eventi di Swing: il timer, il mouse, lo slider, `stepNext` e quindi tutte le notifiche. Per questo il mondo può usare un `ArrayList` senza sincronizzazione e la riga di stato può chiamare `setText` dentro `pallinaFerma`. Se un giorno il mondo avanzasse su un thread suo (una simulazione pesante, un server), le notifiche arriverebbero da quel thread, e i listener che toccano componenti Swing dovrebbero rientrare con `invokeLater`: è la regola di *Thread e concorrenza*, e qui basta sapere che vale anche per gli eventi del Model.



Un passo del ciclo: il mondo avanza, annuncia a passo concluso (i listener leggono uno stato coerente), poi il Controller chiede il ridisegno. Tutto sullo stesso thread.

3.5 La View che ascolta, la View che annuncia

Con gli eventi, la riga di stato diventa una View che si aggiorna da sola. È un'etichetta che si registra sul mondo nel costruttore e conta ciò che il mondo le dice:

```
public class PnlStato extends JLabel implements MondoListener {

    private int lasciate, ferme;

    public PnlStato(Mondo mondo) {
        super("nessuna pallina");
        mondo.addMondoListener(this);
    }

    @Override
    public void pallinaLasciata(Pallina p) { lasciate++; aggiorna(); }

    @Override
    public void pallinaFerma(Pallina p)    { ferme++;    aggiorna(); }

    private void aggiorna() {
        setText(lasciate + " palline, " + ferme + " ferme");
    }
}
```

Il Controller non la conosce, il mondo non la conosce; il `main` la crea e la mette in fondo alla finestra, e basta. Il conteggio cambia solo quando cambia qualcosa: un `setText` ogni pallina lasciata o fermata, non sessanta al secondo.

A questo punto una domanda si pone da sola: perché il pannello `PnlMondo` non fa lo stesso? Potrebbe registrarsi sul mondo e chiamare `repaint()` a ogni evento, liberando il Controller anche di questo. La risposta è che per il pannello non servirebbe: in un'animazione le palline si muovono a *ogni* passo, e il pannello va ridisegnato a ogni passo comunque; un evento "il mondo è avanzato" sessanta volte al secondo sarebbe solo un giro più lungo per arrivare allo stesso `repaint`. Il ridisegno su evento conviene quando il Model cambia *di rado* e *a scatti*: una scacchiera che cambia a ogni mossa, una lista che cambia a ogni inserimento, la riga di stato. Il ridisegno a ogni passo del ciclo conviene quando il Model cambia in continuazione. In un gioco tipicamente coesistono: il pannello di gioco a ogni passo, il punteggio e la riga di stato su evento.

C'è un'ultima freccia della figura del capitolo 2 da sistemare. Il Controller riceveva il mouse registrando un `MouseListener` sulla View e chiamando `view.screenToWorld` per avere il punto in metri: funziona, ma il Controller sa che la View ha pixel, e sa che il gesto è un `mousePressed`. La View può fare di meglio, diventando a sua volta una sorgente di eventi *suoi*, nel linguaggio del mondo:

```java
/** Chi vuole sapere dove l'utente ha cliccato, in coordinate del mondo. */
public interface PuntoListener {
    void puntoScelto(Vec2 punto);
}

private final List<PuntoListener> listeners = new ArrayList<>();

public PnlMondo(Mondo mondo) {
    this.mondo = mondo;
    setBackground(new Color(0x101820));
    addMouseListener(new MouseAdapter() {
        @Override
        public void mousePressed(MouseEvent e) {
            Vec2 p = screenToWorld(e.getPoint());
            for (PuntoListener l : List.copyOf(listeners))
                l.puntoScelto(p);
        }
    });
}

public void addPuntoListener(PuntoListener l)    { listeners.add(l); }
public void removePuntoListener(PuntoListener l) { listeners.remove(l); }
```

Il pannello traduce il gesto grezzo (un pixel, un pulsante del mouse) in un fatto nel linguaggio del dominio, "è stato scelto questo punto del mondo", e lo annuncia. Il Controller si riduce a una riga, `view.addPuntoListener(p -> mondo.lascia(p, 0.25))`: non sa più niente di pixel né di mouse, e decide solo che cosa quel punto *significa*. Se domani il punto arrivasse da un tocco, da una tastiera o da una rete, il Controller non cambierebbe. `PuntoListener` ha un solo metodo, quindi il listener è una lambda; `screenToWorld` è tornato privato, perché nessuno fuori dalla View ha più bisogno di pixel.

Osservazione 3.2

Le frecce del triangolo sono ora tutte eventi o letture. La View legge il Model e annuncia i gesti dell'utente; il Controller ascolta la View e scrive nel Model; il Model annuncia i fatti e le proprietà a chi vuole. Nessuna delle tre parti chiama un metodo di una classe concreta che non sia il Model: la View conosce un'interfaccia di ascolto, il Controller conosce il Model e le interfacce di ascolto della View. È l'MVC come lo intendono le descrizioni classiche, in cui "Model e View non parlano" significa esattamente questo: si parlano solo attraverso eventi.

3.6 L'esempio guida: Palline, seconda versione

Manca il suono. È un listener come la riga di stato, che ridefinisce solo `rimbalzo`: tiene un `Clip` di `javax.sound.sampled`, caricato una volta nel costruttore, e a ogni rimbalzo lo riavvolge e lo fa partire con un volume proporzionale alla velocità d'impatto. Per non dipendere da un file audio, il *toc* è sintetizzato: ottanta millisecondi di sinusoide a 440 Hz che si spegne.

```
public class Suono implements MondoListener {

    private final Clip clip;

    public Suono() throws Exception {
        clip = AudioSystem.getClip();
        clip.open(toc());                                  // caricato una volta
    }

    @Override
    public void rimbalzo(Rimbalzo r) {
        double volume = Math.min(1, r.velocita() / 10);   // da 10 m/s: pieno
        FloatControl gain =
                (FloatControl) clip.getControl(FloatControl.Type.MASTER_GAIN);
        double dB = 20 * Math.log10(Math.max(volume, 1e-3));
        gain.setValue((float) Math.max(gain.getMinimum(), dB));
        clip.setFramePosition(0);
        clip.start();
    }

    /** 80 ms di sinusoide a 440 Hz che si spegne: un "toc" senza file. */
    private static AudioInputStream toc() {
        float freq = 44100;
        int n = (int) (freq * 0.08);
        byte[] dati = new byte[2 * n];
        for (int i = 0; i < n; i++) {
            double t = i / freq;
            double onda = Math.sin(2 * Math.PI * 440 * t) * Math.exp(-t * 40);
            short v = (short) (Short.MAX_VALUE * onda);
            dati[2 * i] = (byte) v;
            dati[2 * i + 1] = (byte) (v >> 8);
        }
        AudioFormat formato = new AudioFormat(freq, 16, 1, true, false);
        return new AudioInputStream(new ByteArrayInputStream(dati), formato, n);
    }
}
```

Il volume è in decibel, e la conversione da una frazione fra 0 e 1 è $20 \log_{10}$: metà velocità sono circa -6 dB. Il punto che conta per l'architettura è un altro: il suono è caricato *una volta*, nel costruttore, e non a ogni urto come nella Palline del capitolo 1; e il mondo non sa che esiste. Nel capitolo 6 vedremo dove mettere i suoni quando sono più d'uno.

Il `main` monta tutto. Rispetto al capitolo 2 ci sono la barra con lo slider e il pulsante, la riga di stato in fondo, e il suono registrato sul mondo prima di costruire la finestra; il Controller riceve anche lo slider e il pulsante, che lega alle proprietà come visto nella sezione 3.3.

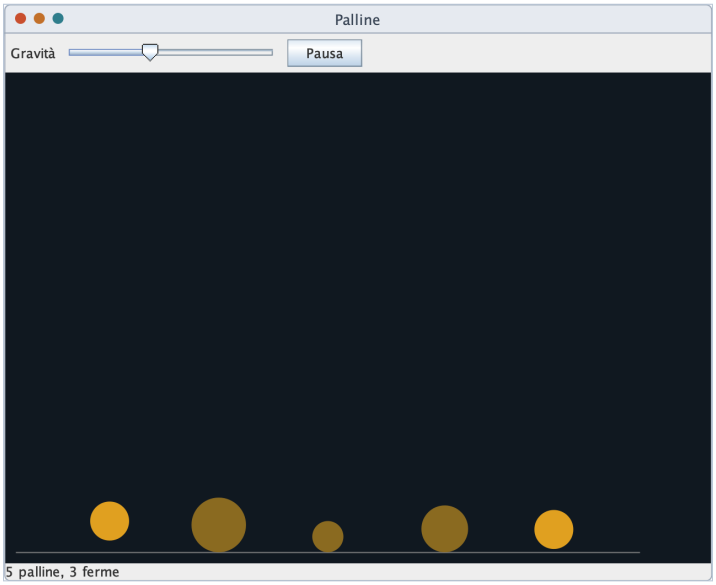
```
public static JFrame finestra(Mondo mondo) {
    PnlMondo view = new PnlMondo(mondo);
    PnlStato stato = new PnlStato(mondo);
    JSlider sldGravita = new JSlider(0, 2500);
    JToggleButton btnPausa = new JToggleButton("Pausa");
    var controller = new Controller(mondo, view, sldGravita, btnPausa);

    JPanel barra = new JPanel(new FlowLayout(FlowLayout.LEFT));
    barra.add(new JLabel("Gravità"));
    barra.add(sldGravita);
    barra.add(btnPausa);

    JFrame frame = new JFrame("Palline");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.add(barra, BorderLayout.NORTH);
    frame.add(view, BorderLayout.CENTER);
    frame.add(stato, BorderLayout.SOUTH);
    frame.setSize(640, 520);
    frame.addWindowListener(new WindowAdapter() {
        @Override
        public void windowOpened(WindowEvent e) { controller.avvia(); }
    });
    return frame;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        Mondo mondo = new Mondo(8, 6);
        try {
            mondo.addMondoListener(new Suono());
        } catch (Exception e) {
            System.err.println("niente audio: " + e.getMessage());
        }
        finestra(mondo).setVisible(true);
    });
}
```

Il Controller parte nel `windowOpened` della finestra, cioè quando la finestra è davvero sullo schermo, invece che subito dopo `setVisible`: è un dettaglio, ma evita che il primo passo misuri anche il tempo in cui il sistema apre la finestra. Se l'audio non è disponibile (un computer senza scheda audio, un server) il programma lo dice e va avanti senza suono: il mondo non se ne accorge, perché un listener in meno non cambia niente.



Palline, seconda versione: la barra con la gravità e la pausa, la riga di stato che conta, le palline ferme più scure.

Palline ora si ascolta e si comanda. Il pannello disegna le palline ferme con un colore più scuro, leggendo `ferma()` dal Model, e un velo con la scritta "pausa" quando il mondo è in pausa, leggendo `inPausa()`: due decisioni della View su dati del Model, e nessun dato della View. Lo slider e il pulsante restano allineati con il mondo qualunque cosa succeda, e il *toc* arriva al momento giusto, con il volume giusto, senza leggere il disco. Quello che non va ancora bene è la fisica: palline che si fermano con una soglia arbitraria, nessun urto fra loro, nessuna parete. È il capitolo 4.

Ricetta: aggiungere eventi a un Model

1. **Fatti o proprietà?** Per ciò che *succede* (un rimbalzo, una mossa) un'interfaccia `XxxListener` con un metodo `default` per fatto e un record per i dati; per ciò che *vale* e cambia (la gravità, la pausa) un `PropertyChangeSupport` e un annuncio in ogni setter.
2. **`add`, `remove`, elenco:** una lista nel Model, i due metodi con i nomi della convenzione; la notifica scorre una copia.
3. **Si annuncia a stato coerente:** dopo aver aggiornato le strutture, mai da un costruttore, a passo concluso se il passo tocca molti oggetti.
4. **Chi ascolta si registra da solo** (nel costruttore della View, nel `main` per il suono) e **si cancella quando muore**.
5. **La View annuncia i gesti** nel linguaggio del dominio (`puntoScelto(Vec2)`), e il Controller decide che cosa significano.

Esercizi

Esercizio 3.1: Chi viene avvisato (⋆)

Sul mondo sono registrati, nell'ordine, i listener A, B e C. Nel suo `pallinaLasciata`, B chiama `mondo.removeMondoListener(B)` e poi `mondo.addMondoListener(D)`. Si chiama `mondo.lascia(...)` due volte. Chi riceve il primo evento, e in che ordine? Chi riceve il secondo? Che cosa cambierebbe se la notifica scorresse `listeners` invece di `List.copyOf(listeners)`?

Esercizio 3.2: Contare le chiamate (⋆)

Il ciclo gira a 60 passi al secondo. In dieci secondi l'utente lascia 3 palline, che rimbalzano in tutto 20 volte e si fermano tutte e tre. Quante volte viene chiamato `setText` della riga di stato se il Controller la aggiorna a ogni passo? E se la riga di stato ascolta il mondo come `PnlStato`? Quante volte parte il *toc*?

Esercizio 3.3: Il giro dello slider (⋆)

Lo slider della gravità è in centesimi; un comando imposta la gravità lunare con `mondo.setGravita(1.62)`. Elencare, nell'ordine, le chiamate che seguono (listener del mondo, `setValue`, `ChangeListener`, `setGravita`, annuncio) e dire dove e perché il giro si ferma. Poi ripetere con uno slider in *decimi* e `(int)` al posto di `Math.round`: quanto vale la gravità alla fine?

Esercizio 3.4: Troppo presto (⋆⋆)

Un collega sposta la notifica `pallinaLasciata` dentro il costruttore di `Pallina`, passando il mondo come parametro, "così ogni pallina si annuncia da sola". La riga di stato continua a funzionare; un nuovo listener che in `pallinaLasciata` chiama `mondo.palline().size()` per mostrare il totale sbaglia sempre di uno. Spiegare perché, e dire quale regola della sezione 3.4 è stata infranta.

Esercizio 3.5: La finestra fantasma (⋆⋆)

Un menu "Nuova finestra" apre un secondo `JFrame` con un altro `PnlMondo` e un altro `PnlStato` sullo stesso mondo, con `DISPOSE_ON_CLOSE`. L'utente apre e chiude dieci finestre. Che cosa resta in memoria, e perché il collettore non lo libera? Quanti `setText` vengono eseguiti alla prossima pallina lasciata? Dove va messa la cancellazione, e di che cosa?

I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.

Esercizio 3.6: Il termostato (⋆)

Un Model `Termostato` con due proprietà legate: la temperatura *misurata*, che cambia col tempo (`stepNext(dt)`: sale di un grado al secondo se il riscaldamento è acceso, scende di mezzo grado al secondo altrimenti) e la temperatura *desiderata*, impostata dall'utente; una terza proprietà derivata, `riscaldamentoAcceso`, che si spegne quando la misurata raggiunge la desiderata e si riaccende quando scende mezzo grado sotto (un'*isteresi*, altrimenti si accenderebbe e spegnerebbe a ogni passo). Tutte e tre annunciate con `PropertyChangeSupport`. Due View: uno `JSpinner` legato nei due versi alla desiderata e un pannello che disegna un termometro con la misurata e una tacca sulla desiderata; un terzo listener che stampa a console ogni accensione e spegnimento con l'istante. Il Model si prova senza finestra facendo passare un minuto.

Soluzione: progetto `sol-dxtr-02-c03`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 3.2: La proprietà derivata che non annuncia

`riscaldamentoAcceso` non ha un setter: cambia quando cambiano le altre due. Se non la si annuncia (confrontando il valore prima e dopo ogni cambiamento), la console non sa mai quando il riscaldamento parte. Una proprietà derivata si annuncia in ogni punto in cui può cambiare, e `firePropertyChange` tace da solo quando non è cambiata.

Esercizio 3.7: Il cronometro (⋆⋆)

Un Model `Cronometro` con `avvia`, `ferma`, `giro` e `azzera`, che riceve il tempo con `stepNext(dt)` e annuncia due fatti a un `CronometroListener`: `tick(double tempo)` ogni decimo di secondo di tempo misurato, e `giro(int numero, double tempoGiro, double tempoTotale)` a ogni giro. Scriverlo prima con una lista di listener a mano, poi con `EventListenerList`, e dire in un commento che cosa cambia. View: il tempo grande in alto e la lista dei giri sotto (una `JList` con un `DefaultListModel` aggiornato dalil listener); Controller con i quattro pulsanti e un `javax.swing.Timer`.

Soluzione: progetto `sol-dxtr-02-c03`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 3.3: Il tick a ogni passo

Annunciare `tick` a ogni passo del timer (60 volte al secondo) invece che ogni decimo di secondo misurato riempie l'interfaccia di aggiornamenti inutili e lega il ritmo degli eventi a quello dello schermo. Il Model accumula i `dt` e annuncia quando il decimo è passato, anche più volte in un passo se il passo è stato lungo.

Esercizio 3.8: La lista della spesa (⋆⋆)

Un Model `Spesa` con una lista di voci (un record `Voce` con nome, prezzo e il booleano `presa`) e le operazioni `aggiungi`, `rimuovi`, `segna`; annuncia a uno `SpesaListener` i fatti `aggiunta(int indice, Voce v)`, `rimossa(int indice, Voce v)` e `cambiata(int indice, Voce v)`. Due View: una `JList` il cui `ListModel` è una classe nostra che implementa `AbstractListModel` ascoltando `Spesa` (e traducendo i tre fatti in `fireIntervalAdded`, `fireIntervalRemoved`, `fireContentsChanged`), e un'etichetta con il totale delle voci non ancora prese. Un campo di testo e tre pulsanti come Controller.

Soluzione: progetto `sol-dxtr-02-c03`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 3.4: Due liste, una verità

Se il `ListModel` tiene una *copia* delle voci, prima o poi copia e Model divergono (una voce segnata nel Model e non nella copia). Il `ListModel` nostro non tiene niente: `getElementAt` legge direttamente da `Spesa`, e gli eventi servono solo a dire alla `JList` *che cosa* ridisegnare.

Esercizio 3.9: Il registratore (⋆⋆⋆)

Un listener `Registratore` che si registra sul mondo di Palline e sul pannello (`PuntoListener`) e scrive su un file di testo, una riga per evento, l'istante (il tempo del mondo, che il registratore accumula dai `dt`: aggiungere a `Mondo` un metodo `tempo()` che lo restituisce) e il fatto: `LASCIA x y r`, `RIMBALZO x v`, `FERMA x`. Un secondo programma legge il file e *rifaà* la scena: crea un mondo nuovo, lo fa avanzare con lo stesso passo fisso di un centesimo e, quando il tempo del mondo raggiunge l'istante di una riga `LASCIA`, lascia la pallina; i rimbalzi e le fermate del file si confrontano con quelli che il mondo nuovo annuncia, e il programma stampa quanti coincidono entro un millesimo di secondo.

Suggerimento: la riproduzione coincide solo se i passi sono gli stessi. Registrare con il `dt` misurato e riprodurre a passo fisso dà differenze: è l'argomento con cui si apre il capitolo 4.

Soluzione: progetto `sol-dxtr-02-c03`, package `it.unibs.pajc.redolfi.e09`.

Risposte

Risposta 3.1. Primo evento: la copia contiene A, B, C; ricevono A, B, C in quest'ordine. D è stato aggiunto durante la notifica, ma la copia era già fatta: non riceve il primo evento. Secondo evento: la lista è ora A, C, D; ricevono A, C, D. Senza la copia, il `remove` di B dentro il `for` farebbe lanciare una `ConcurrentModificationException` alla prossima iterazione, dentro `lascia`, cioè dentro il mondo: C non riceverebbe niente e l'eccezione risalirebbe fino al listener del mouse.

Risposta 3.2. A ogni passo: 600 chiamate a `setText` in dieci secondi, di cui 594 scrivono il testo che c'era già. Ascoltando il mondo: una per pallina lasciata e una per pallina ferma, 6 in tutto. Il *toc* parte 20 volte, una per rimbalzo, e nessuna per le tre fermate, che sono un evento diverso.

Risposta 3.3. `setGravita(1.62)`: la gravità passa da 9,81 a 1,62 e `PropertyChangeSupport` annuncia (valori diversi). Il listener del Controller chiama `sldGravita.setValue(162)`; lo slider cambia valore e fa scattare il suo `ChangeListener`, che chiama `setGravita(1.62)`; vecchio e nuovo sono uguali e `PropertyChangeSupport` non annuncia: il giro si ferma qui, con la gravità a 1,62 e lo slider a 162. Con lo slider in decimi e `(int)`: `setValue((int)(1.62*10))` = 16; il `ChangeListener` chiama `setGravita(1.6)`; 1,62 e 1,6 sono diversi, quindi si annuncia; il listener chiama `setValue((int)(1.6*10))`, cioè 16 (o 15, se $1{,}6 \cdot 10$ viene 15,999… in virgola mobile); se 16, il valore non cambia e il giro si ferma con la gravità a 1,6 invece di 1,62; se 15, lo slider cambia, `setGravita(1.5)`, un altro annuncio, `setValue(15)`: si ferma a 1,5. In ogni caso la gravità è stata cambiata da una conversione, non dall'utente.

Risposta 3.4. Il costruttore di `Pallina` viene eseguito dentro `new Pallina(...)`, cioè *prima* di `palline.add(p)`: quando il listener riceve l'annuncio la pallina esiste ma non è ancora nella lista, e `size()` è ancora il totale precedente. La riga di stato funziona per caso, perché conta per conto suo e non legge la lista. La regola infranta è la prima: si annuncia a stato coerente, dopo aver aggiornato le strutture, e mai da un costruttore (che per di più lascia uscire `this` prima che l'oggetto sia finito di costruire).

Risposta 3.5. Ogni `PnlStato` si è registrato sul mondo nel suo costruttore e nessuno lo ha cancellato: il mondo tiene dieci riferimenti a dieci etichette, ciascuna delle quali tiene (attraverso la catena dei contenitori) la propria finestra con tutti i componenti. Il collettore libera solo ciò che non è più raggiungibile, e tutto questo è raggiungibile dal mondo. Alla prossima pallina lasciata vengono eseguiti undici `setText`, dieci dei quali su etichette che nessuno vede. La cancellazione va nel `windowClosed` della seconda finestra (un `WindowAdapter`), e deve togliere *sia* il `PnlStato` dal mondo (`removeMondoListener`) *sia* l'eventuale listener delle proprietà; il `PnlMondo` non si è registrato su niente e non ha bisogno di nulla. Un modo pulito è dare a `PnlStato` un metodo `scollega(Mondo)` che la finestra chiama alla chiusura.

Capitolo 4

Un mondo che rimbalza bene: il modello fisico

La Palline del capitolo 3 ha l'architettura giusta e la fisica sbagliata: le palline si fermano con una soglia messa lì per far tacere il suono, non si urtano fra loro, non ci sono pareti, e con un passo di tempo lungo (dopo una pausa, con la finestra trascinata) attraversano il pavimento. Questo capitolo completa la risposta alla seconda domanda del capitolo 1, come scorre il tempo, e riscrive il mondo perché rimbalzi bene: il tempo a passi fissi, il modo giusto di integrare il moto, il rimbalzo che riporta la pallina fuori dalla superficie e la ferma quando deve, gli urti fra palline con le masse, gli ostacoli, le palline veloci che attraversano le cose sottili. È il capitolo più "fisico" della dispensa, ma il suo punto è architetturale: tutto ciò che segue sta nel Model, e alla fine la View e il Controller non saranno cambiati di una riga. È il premio della separazione fatta nei capitoli 2 e 3, e il capitolo lo incassa.

4.1 Il tempo a passi fissi

Il Controller misura il tempo passato fra un giro e l'altro e lo dà al mondo: `stepNext(0.016)` quando tutto va bene, `stepNext(0.031)` quando un fotogramma è lento. Il mondo applica quel `dt` alle sue formule. Sembra il massimo della fedeltà, e ha due difetti che si vedono solo guardando bene.

Il primo: **la fisica dipende dal passo**. Le formule che useremo (la velocità che cresce di $g\,dt$, la posizione di $v\,dt$) sono approssimazioni, tanto migliori quanto più `dt` è piccolo, e due passi da 16 ms non danno lo stesso risultato di uno da 32. Con il `dt` misurato il risultato di una partita dipende da quanti fotogrammi il computer ha fatto: una pallina lanciata nello stesso modo arriva in un posto leggermente diverso su due macchine, e la stessa partita non si può riprodurre (il registratore del problema 3.9 ne faceva le spese). Il secondo: **il passo grande rompe tutto**. Dopo una pausa del sistema di mezzo secondo il Controller chiama `stepNext(0.5)`: in un passo la gravità aggiunge 5 m/s, la posizione salta di metri, una pallina che era a mezzo metro dal pavimento si ritrova sotto, e un'altra attraversa una parete senza toccarla.

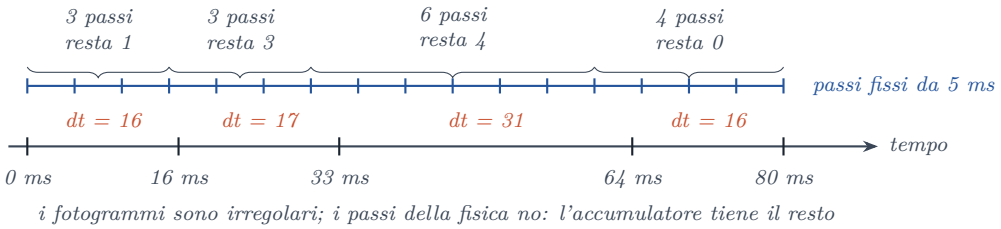
La soluzione, nota in ogni motore di gioco, è separare i due tempi fino in fondo: **la fisica avanza sempre con lo stesso passo**, piccolo e fisso, e il tempo misurato serve solo a decidere *quanti* passi fare. Il mondo tiene un *accumulatore* del tempo ricevuto e non ancora simulato:

```
public static final double PASSO = 1.0 / 200;        // s: il passo fisso
private static final double DT_MASSIMO = 0.1;        // s: oltre si butta

private double accumulatore;                         // s non ancora simulati

public void stepNext(double dt) {
    if (inPausa)
        return;
    accumulatore += Math.min(dt, DT_MASSIMO);
    while (accumulatore >= PASSO - 1e-12) {
        passoFisso();
        accumulatore -= PASSO;
    }
    // ... le notifiche raccolte durante i passi, come nel capitolo precedente
}
```

Con un `dt` di 16 ms e un passo di 5 ms il mondo fa tre passi e tiene da parte un millisecondo; al giro dopo ne riceve altri 16, ne ha 17, fa tre passi e ne tiene due; poi 18, quattro passi. In media fa duecento passi al secondo, qualunque sia il ritmo dei fotogrammi, e ogni passo è identico agli altri. La figura mostra l'accumulatore al lavoro su una sequenza di fotogrammi irregolari.



Il tempo misurato decide quanti passi fare; ogni passo è sempre di 5 ms. Il resto non simulato passa al giro successivo.

Due dettagli del codice. `Math.min(dt, DT_MASSIMO)` taglia i passi enormi: se il sistema è stato fermo due secondi, il mondo ne simula al massimo un decimo e butta il resto. Senza questo taglio, dopo una pausa lunga il mondo dovrebbe fare centinaia di passi in un colpo, il giro del timer durerebbe più di 16 ms, il `dt` successivo sarebbe ancora più grande, e così via: è la *spirale della morte*, un programma che non riesce mai a mettersi in pari. Il `- 1e-12` nel confronto evita che cinque `dt` da 0,001 sommati diano 0,004999… invece di 0,005 e il passo slitti al giro dopo; con i `double` succede.

Osservazione 4.1

Il premio del passo fisso è il *determinismo*: lo stesso scenario, con gli stessi comandi agli stessi istanti di tempo del mondo, produce esattamente lo stesso risultato, su qualunque computer e con qualunque ritmo di fotogrammi. Sembra un lusso, ed è il fondamento di tre cose: la riproduzione di una partita registrando solo i comandi (capitolo 5), il gioco in rete in cui due computer simulano lo stesso mondo scambiandosi i comandi, e i test automatici, che devono dare sempre lo stesso risultato. Il prezzo è che la View può disegnare un mondo in ritardo di qualche millisecondo (il resto nell'accumulatore); i motori che vogliono il massimo della fluidità interpolano fra l'ultimo stato e quello precedente, e il capitolo 8 dice dove leggerne.

4.2 Integrare il moto

Dentro il passo fisso c'è il moto di ogni pallina. Dalla fisica sappiamo che la velocità cambia secondo l'accelerazione e la posizione secondo la velocità; in un passo di durata Δt l'approssimazione più semplice è il *metodo di Eulero*: $\vec{v} \leftarrow \vec{v} + \vec{g}\,\Delta t$ e $\vec{x} \leftarrow \vec{x} + \vec{v}\,\Delta t$. Due righe, che però si possono scrivere in due ordini, e i due ordini non sono equivalenti.

Nell'ordine *esplicito* la posizione si aggiorna con la velocità vecchia, poi si aggiorna la velocità. Nell'ordine *semi-implicito* prima si aggiorna la velocità, poi la posizione con la velocità *nuova*. Il programma seguente fa rimbalzare una pallina senza perdite per dieci secondi nei due modi, con un passo di un sessantesimo, e stampa l'energia (cinetica più potenziale, per unità di massa), che in un mondo senza attriti dovrebbe restare quella iniziale, $g \cdot 5 = 49{,}05$:

```
double g = 9.81, dt = 1.0 / 60;
double yE = 5, vE = 0, yS = 5, vS = 0;          // esplicito, semi-implicito
for (int passo = 1; passo <= 600; passo++) {
    yE += vE * dt;                               // esplicito: la v vecchia
    vE -= g * dt;
    if (yE < 0) { yE = -yE; vE = -vE; }
    vS -= g * dt;                                // semi-implicito: prima v...
    yS += vS * dt;                               // ...poi y con la v nuova
    if (yS < 0) { yS = -yS; vS = -vS; }
    if (passo % 120 == 0)
        System.out.printf("t = %2d s   esplicito: E = %6.2f"
                + "   semi-implicito: E = %6.2f%n",
                passo / 60, 0.5 * vE * vE + g * yE, 0.5 * vS * vS + g * yS);
}
```

```
t =  2 s   esplicito: E =  53,66   semi-implicito: E =  50,45
t =  4 s   esplicito: E =  55,63   semi-implicito: E =  49,10
t =  6 s   esplicito: E =  60,34   semi-implicito: E =  50,50
t =  8 s   esplicito: E =  62,42   semi-implicito: E =  49,16
t = 10 s   esplicito: E =  67,24   semi-implicito: E =  50,55
```

Con l'ordine esplicito l'energia *cresce da sola*: in dieci secondi è aumentata del 37%, e la pallina rimbalza ogni volta più in alto di quando è partita, senza che nessuno la spinga. Con l'ordine semi-implicito oscilla fra 49 e 50,5 e non va da nessuna parte: non è esatta, ma è *limitata*, e lo resta per sempre. Il motivo sta nel fatto che il metodo semi-implicito conserva una quantità vicina all'energia vera (è un integratore *simplettico*, e chi vuole il perché lo trova nel capitolo 8); quello che conta qui è la regola, che già il capitolo 2 aveva adottato senza dirlo: **prima la velocità, poi la posizione con la velocità nuova**.

Attenzione 4.1: Le due righe invertite

L'ordine esplicito è quello che viene naturale ("dove sono, poi come cambia la velocità") e il risultato sbagliato non si vede subito: nei primi secondi le due versioni sono indistinguibili, e il guadagno di energia si nota solo quando le palline cominciano a rimbalzare più in alto di dove erano state lasciate, o quando un pendolo accelera da solo. Nel mondo di Palline, con la restituzione a 0,8, la perdita a ogni rimbalzo maschererebbe il guadagno per un po', e poi palline mai del tutto ferme. Un test sull'energia, come quello della sezione 4.6, lo scopre in un decimo di secondo.

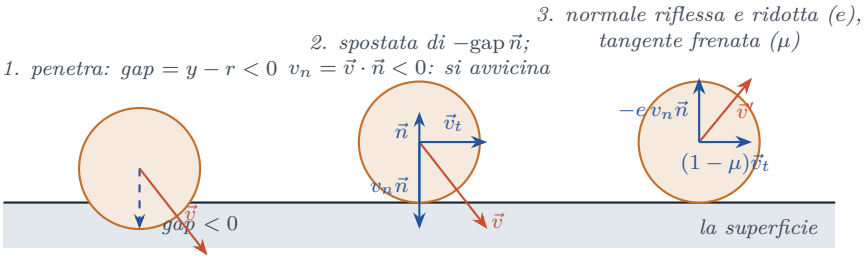
Nel mondo di Palline le due righe stanno nel cuore del passo fisso, applicate a ogni pallina che non dorme:

```
Vec2 g = new Vec2(0, -gravita);
for (Pallina p : palline) {
    if (p.ferma())
        continue;
    p.setVelocita(p.velocita().piu(g.per(dt)));                // prima v...
    p.setPosizione(p.posizione().piu(p.velocita().per(dt)));   // ...poi x
}
```

La pallina ha guadagnato due *setter* visibili solo nel package, perché il moto e gli urti ora li calcola il mondo, che vede tutte le palline insieme: la singola pallina non può sapere se sta per urtarne un'altra. Ha anche una massa, proporzionale all'area (πr^2, con densità unitaria: contano solo i rapporti) e un coefficiente di restituzione suo, che nel capitolo 5 diventerà il materiale.

4.3 Il rimbalzo, fatto bene

Il rimbalzo del capitolo 2 riportava la pallina a terra e invertiva la componente verticale della velocità. Funzionava sul pavimento e solo lì: una parete verticale, una mensola, un piano inclinato vogliono la stessa cosa in un'altra direzione. La versione generale ragiona in termini di *superficie* e *normale*: una superficie fissa, piana, e il versore $\vec{n}$ perpendicolare a essa, che punta dalla superficie verso la pallina. Per il pavimento $\vec{n} = (0, 1)$; per la parete sinistra $(1, 0)$; per quella destra $(-1, 0)$. Tutto il rimbalzo si scrive in funzione di $\vec{n}$, in tre mosse.



Il rimbalzo in tre mosse: fuori dalla superficie, scomposizione della velocità lungo la normale e la tangente, riflessione della sola componente normale.

Prima mossa: fuori dalla superficie. Alla fine di un passo la pallina può essere *dentro* la superficie di un po', perché il passo è discreto: la distanza fra il bordo della pallina e la superficie, il *gap*, è negativa. Si sposta la pallina lungo la normale di quanto basta, $-\text{gap}\,\vec{n}$, e torna appoggiata. La Palline del capitolo 1 non lo faceva, e le palline affondavano.

Seconda mossa: solo se si avvicina. Si scompone la velocità in una componente *normale*, $v_n = \vec{v} \cdot \vec{n}$ (il prodotto scalare: quanto della velocità va nella direzione di $\vec{n}$), e una *tangente*, $\vec{v}_t = \vec{v} - v_n\vec{n}$. Se $v_n \geq 0$ la pallina si sta già allontanando dalla superficie, e non c'è nessun rimbalzo da fare: è il caso di una pallina appena rimbalzata che nel passo successivo risulta ancora a contatto. Senza questo controllo la si rifletterebbe di nuovo verso il basso, e poi di nuovo verso l'alto, a ogni passo: è il *tremolio* del capitolo 2.

Terza mossa: riflettere la normale, frenare la tangente. La componente normale si inverte e si riduce del coefficiente di *restituzione* e (0,8 per la gomma: dopo il rimbalzo

risale all'80% della velocità con cui è arrivata, cioè al 64% dell'altezza, perché l'altezza va col quadrato); la componente tangente, quella che scorre lungo la superficie, perde una frazione μ per attrito. La nuova velocità è $(1-\mu)\,\vec{v}_t - e\,v_n\,\vec{n}$.

C'è una quarta mossa, che risponde alla domanda lasciata aperta nel capitolo 3: quando una pallina è *ferma*? Quando, dopo il rimbalzo su una superficie d'appoggio (una su cui può stare: il pavimento, la faccia superiore di una mensola), la velocità che le resta è sotto una soglia piccola. Allora si ferma davvero: velocità zero, e il mondo la salta nei passi successivi finché qualcosa non la sveglia. In codice, il contatto con una superficie fissa è un metodo solo:

```
private static final double SOGLIA_QUIETE = 0.15;    // m/s
private static final double ATTRITO = 0.3;           // tangente persa per urto

/** Contatto con una superficie fissa: n la normale, gap la distanza. */
private void contatto(Pallina p, Vec2 n, double gap, boolean appoggio) {
    if (gap >= 0)
        return;
    p.setPosizione(p.posizione().meno(n.per(gap)));            // 1. fuori
    double vn = p.velocita().scalare(n);
    if (vn >= 0)                                        // si allontana gia
        return;
    Vec2 tangente = p.velocita().meno(n.per(vn));              // 2. scomposta
    Vec2 nuova = tangente.per(1 - ATTRITO).meno(n.per(vn * p.restituzione()));
    if (appoggio && nuova.modulo() < SOGLIA_QUIETE) {          // 3. quiete
        p.fermati();
        fermate.add(p);
    } else {
        p.setVelocita(nuova);
        if (-vn > SOGLIA_QUIETE)
            rimbalzi.add(new Rimbalzo(p, -vn));
    }
}
```

Il pavimento e le due pareti sono tre chiamate, con la normale e il gap di ciascuna:

```
Vec2 x = p.posizione();
contatto(p, new Vec2(0, 1), x.y() - p.raggio(), true);        // pavimento
contatto(p, new Vec2(1, 0), x.x() - p.raggio(), false);       // sinistra
contatto(p, new Vec2(-1, 0), larghezza - x.x() - p.raggio(), false);
```

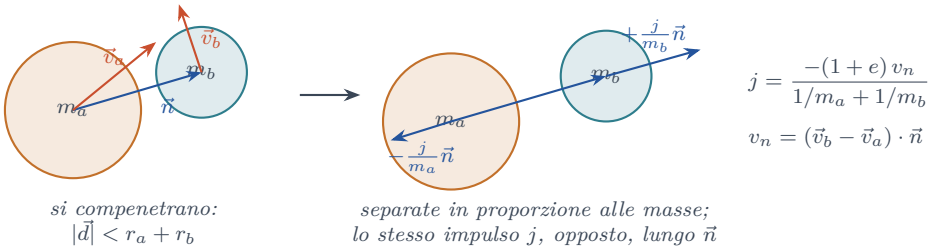
Il `Vec2` ha guadagnato `scalare` (il prodotto scalare) e `versore` (lo stesso verso con lunghezza uno), le due operazioni che servono per ragionare con le normali. L'evento `Rimbalzo` parte solo per gli urti sopra la soglia: i piccoli assestamenti di una pallina che si sta fermando non fanno *toc*, e la soglia arbitraria del capitolo precedente non serve più, perché la quiete è decisa dalla fisica.

Attenzione 4.2: Rimbalzare sempre

Senza il controllo `vn >= 0` una pallina appoggiata riceve la gravità, scende di un soffio sotto il pavimento, viene riportata su e riflessa: ha di nuovo una velocità verso l'alto, piccola; al passo dopo è scesa di nuovo, viene riflessa di nuovo. Non si ferma mai, trema, e se la restituzione è alta la velocità residua può anche crescere, perché a ogni passo si aggiunge $g\,\Delta t$ e si riflette tutto. È un errore silenzioso: nessuna eccezione, solo palline che vibrano sul pavimento e un *toc* continuo. Il controllo costa un confronto e vale per ogni superficie, anche quelle fra due palline.

4.4 Le palline fra loro

Due palline si urtano quando la distanza fra i centri è minore della somma dei raggi. Trovarle è facile: si guardano tutte le coppie. Risolverle richiede lo stesso schema del contatto con una superficie, con due differenze: la superficie è l'altra pallina, che si muove e ha una massa; e la normale è la congiungente dei centri. La figura mostra la geometria.



L'urto fra due palline: la normale è la congiungente dei centri; si separano e ricevono lo stesso impulso con segno opposto, diviso per le rispettive masse.

Le tre mosse diventano queste. Prima si *separano*: la compenetrazione $r_a + r_b - |\vec{d}|$ si divide fra le due in proporzione inversa alle masse (la più leggera si sposta di più). Poi si guarda la velocità *relativa* lungo la normale, $v_n = (\vec{v}_b - \vec{v}_a)\cdot\vec{n}$: se è positiva le palline si stanno già allontanando e non c'è niente da fare (lo stesso controllo del paragrafo precedente, altrimenti due palline appoggiate si respingerebbero per sempre). Infine si applica un *impulso*: la quantità di moto j che passa dall'una all'altra lungo la normale, calcolata perché la velocità relativa dopo l'urto sia $-e$ volte quella prima. La formula è quella della figura, e viene dalla conservazione della quantità di moto più la definizione di restituzione; chi vuole la derivazione la trova nei riferimenti del capitolo 8. In codice:

```
/** Urto fra due palline: separate, poi l'impulso lungo la normale. */
private void urto(Pallina a, Pallina b) {
    Vec2 d = b.posizione().meno(a.posizione());
    double dist = d.modulo(), somma = a.raggio() + b.raggio();
    if (dist >= somma)
        return;
    Vec2 n = dist == 0 ? new Vec2(1, 0) : d.per(1 / dist);
    double penetrazione = somma - dist, mTot = a.massa() + b.massa();
    a.setPosizione(a.posizione().meno(n.per(penetrazione * b.massa() / mTot)));
    b.setPosizione(b.posizione().piu(n.per(penetrazione * a.massa() / mTot)));
    double vn = b.velocita().meno(a.velocita()).scalare(n);   // relativa su n
    if (vn >= 0)                                       // si separano gia
        return;
    double e = Math.min(a.restituzione(), b.restituzione());
    double j = -(1 + e) * vn / (1 / a.massa() + 1 / b.massa());
    a.setVelocita(a.velocita().meno(n.per(j / a.massa())));
    b.setVelocita(b.velocita().piu(n.per(j / b.massa())));
    rimbalzi.add(new Rimbalzo(a, -vn));
}
```

Il caso `dist == 0`, due centri coincidenti, non succede in natura ma succede nei programmi (due palline lasciate nello stesso punto): la normale non è definita e se ne sceglie una qualunque, altrimenti si divide per zero e le due palline finiscono a coordinate `NaN`, da cui non tornano più. Il `setVelocita` della pallina ha un effetto in più che ora serve: una pallina che dormiva e riceve una spinta si sveglia (`ferma` torna falso), e il mondo riprende a muoverla.

Osservazione 4.2

La verifica a coppie costa $n(n-1)/2$ confronti per sottopasso: con cento palline sono cinquemila, con mille mezzo milione, duecento volte al secondo. Fino a qualche centinaio di palline un computer qualunque non se ne accorge; oltre, si divide il mondo in una griglia di celle grandi come la pallina più grande e si confrontano solo le palline della stessa cella e delle otto vicine. È l'ottimizzazione più classica dei motori fisici, e il capitolo 8 ne dà un assaggio; qui non serve, e aggiungerla prima che serva sarebbe complessità senza motivo.

4.5 Ostacoli e tunneling

Un ostacolo rettangolare fisso (una mensola, un gradino, i mattoni di un gioco) è una superficie come il pavimento, con una differenza: la normale dipende da dove la pallina lo tocca. Il modo più pulito di trovarla è il *punto più vicino*: si proietta il centro della pallina sul rettangolo, limitando ciascuna coordinata all'intervallo del rettangolo (`Math.clamp`, che Java ha dal 21), e il punto che ne esce è quello del rettangolo più vicino al centro. Se la distanza fra il centro e quel punto è minore del raggio c'è contatto, e la normale è il versore che va dal punto al centro: verso l'alto se la pallina è sopra, di lato se è di fianco, obliqua sugli spigoli, dove il rimbalzo esce in diagonale come ci si aspetta.

```
public record Ostacolo(double x, double y, double larghezza, double altezza) {

    /** Il punto del rettangolo piu vicino a p (p stesso se e' dentro). */
    public Vec2 puntoPiuVicino(Vec2 p) {
        return new Vec2(Math.clamp(p.x(), x, x + larghezza),
                        Math.clamp(p.y(), y, y + altezza));
    }
}
```

Nel passo fisso, dopo pavimento e pareti, per ogni ostacolo:

```
for (Ostacolo o : ostacoli) {
    Vec2 q = o.puntoPiuVicino(p.posizione());
    Vec2 d = p.posizione().meno(q);
    double dist = d.modulo();
    if (dist < p.raggio()) {
        Vec2 n = dist == 0 ? new Vec2(0, 1) : d.versore();
        contatto(p, n, dist - p.raggio(), n.y() > 0.7);
    }
}
```

L'ultimo argomento dice se la superficie è d'appoggio: lo è quando la normale punta abbastanza verso l'alto (la sua componente y è almeno il 70% della lunghezza, cioè un'inclinazione sotto i 45 gradi), così una pallina si può fermare sopra una mensola ma non appesa sotto. Il caso `dist == 0` è il centro della pallina *dentro* il rettangolo: può succedere solo se la pallina lo ha attraversato in un passo, ed è il problema successivo.

Il tunneling. Una pallina si muove a scatti, di $v\,\Delta t$ per passo. Se lo scatto è più lungo dello spessore di un ostacolo, la pallina può trovarsi prima dell'ostacolo in un passo e dopo nel passo successivo, senza mai risultare a contatto: lo attraversa come se non ci fosse. Con un passo di 5 ms e una mensola di 20 cm succede sopra i 40 m/s, che non è una velocità da gioco; ma con i mattoni sottili di un Breakout, le pareti di un labirinto o un proiettile, succede subito. La figura mostra il fenomeno e il rimedio più semplice.



Tunneling: una pallina veloce salta un ostacolo sottile. Con i sottopassi ogni spostamento è piccolo rispetto alla pallina e il contatto non sfugge.

Il rimedio semplice sono i *sottopassi*: se nel passo qualche pallina si sposterebbe di più di mezzo raggio, il passo si divide in tanti sottopassi quanti servono perché nessuno scatto superi quella misura. Il passo fisso resta fisso, visto da fuori; dentro è un ciclo:

```
private void passoFisso() {
    tempo += PASSO;
    passi++;
    double vMax = 0, rMin = Double.MAX_VALUE;
    for (Pallina p : palline) {
        vMax = Math.max(vMax, p.velocita().modulo());
        rMin = Math.min(rMin, p.raggio());
    }
    int sottopassi = Math.max(1, (int) Math.ceil(vMax * PASSO / (rMin / 2)));
    double dt = PASSO / sottopassi;
    for (int i = 0; i < sottopassi; i++) {
        // ... il moto, gli urti fra palline, pavimento, pareti e ostacoli
    }
}
```

Con palline di 10 cm e velocità di qualche metro al secondo il sottopasso è uno; una pallina lanciata a 40 m/s ne chiede quattro, e li chiede solo finché corre. Un ostacolo di spessore minore di mezzo raggio può ancora sfuggire; il rimedio completo, che controlla il *segmento* percorso dalla pallina invece dei suoi estremi, è nel capitolo 8 come lettura.

4.6 Misurare

Un motore fisico si può solo *guardare*? No, e il capitolo 2 aveva promesso che ogni versione del mondo avrebbe avuto i suoi test. Qui i test sono più interessanti, perché la fisica ha delle leggi, e le leggi si possono controllare. Il mondo espone l'energia meccanica totale e il numero di passi fatti:

```
public double energia() {
    double e = 0;
    for (Pallina p : palline)
        e += p.energia(gravita);     // 0.5 m v^2 + m g y, in Pallina
    return e;
}

public long passi() { return passi; }
```

e quattro test dicono se il motore si comporta come deve:

```
@Test
void lEnergiaRestaLimitataEAllaFineECalata() {
    Mondo mondo = conVentiPalline();        // venti palline, una mensola
    double iniziale = mondo.energia();
    for (int i = 0; i < 2000; i++) {                // dieci secondi
        mondo.stepNext(Mondo.PASSO);
        assertTrue(mondo.energia() <= iniziale * 1.02);
    }
    assertTrue(mondo.energia() < iniziale * 0.5);   // dissipata dai rimbalzi
}

@Test
void ilRisultatoNonDipendeDaiFotogrammi() {
    Mondo regolare = conVentiPalline(), irregolare = conVentiPalline();
    for (int i = 0; i < 300; i++)
        regolare.stepNext(0.02);                      // 6 s a passi regolari
    double[] dts = { 0.016, 0.031, 0.009, 0.024 };  // 0,08 s: 75 giri = 6 s
    for (int i = 0; i < 75; i++)
        for (double dt : dts)
            irregolare.stepNext(dt);
    assertEquals(regolare.passi(), irregolare.passi());
    for (int i = 0; i < 20; i++)
        assertEquals(regolare.palline().get(i).posizione(),
                     irregolare.palline().get(i).posizione());
}
```

Il primo test dice due cose: l'energia non supera mai quella iniziale più un 2% (l'oscillazione dell'integratore e le piccole spinte della correzione di penetrazione, che alza una pallina di qualche millimetro), e dopo dieci secondi di rimbalzi è meno della metà, perché la restituzione e l'attrito la dissipano. Con l'ordine esplicito di Eulero il primo `assertTrue` fallisce in pochi secondi di mondo. Il secondo test è il determinismo: lo stesso scenario fatto avanzare con fotogrammi regolari e con fotogrammi irregolari che sommano allo stesso tempo dà lo stesso numero di passi e posizioni *identiche*, non approssimativamente uguali ma uguali bit per bit, perché il mondo ha fatto esattamente gli stessi calcoli nello stesso ordine. Gli altri due test (nessuna pallina sotto il pavimento o oltre le pareti dopo dieci secondi; in un urto frontale senza gravità la quantità di moto totale prima e dopo è la stessa a meno di 10^{-9}) stanno nel codice del capitolo con un quinto, una pallina a 40 m/s che non attraversa una lastra di 5 cm.

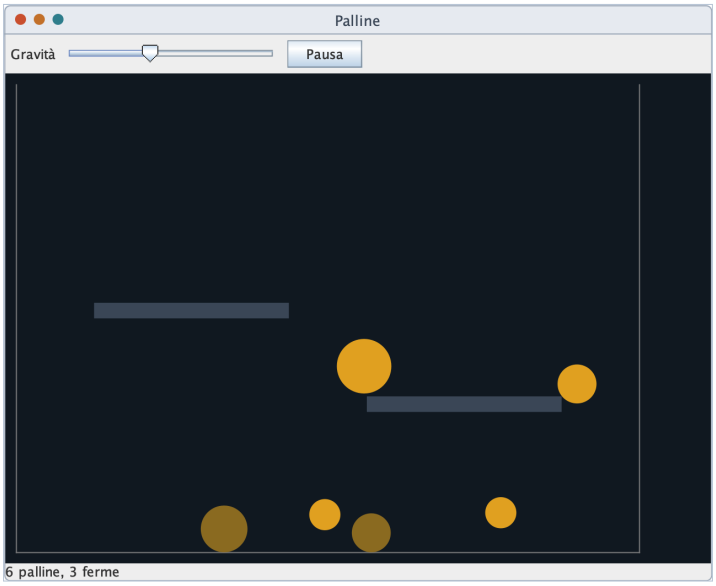
Osservazione 4.3

Il test della quantità di moto ha trovato un errore mentre questo capitolo veniva scritto: le palline che dormono erano escluse dal contatto con le superfici, e un urto poteva spostarne una dentro il pavimento senza che nulla la riportasse fuori. È il genere di errore che a occhio non si vede (una pallina ferma un millimetro sotto il pavimento) e che un test trova in un decimo di secondo. I test del Model non sono una formalità: sono il modo in cui si sa che il motore funziona.

4.7 L'esempio guida: Palline, terza versione

Il mondo ha guadagnato le pareti, gli ostacoli (con `aggiungi(Ostacolo)` e la lista `ostacoli()` in lettura), un metodo `lancia` che lascia una pallina con una velocità iniziale, e il motore descritto in queste pagine. La View ha guadagnato quattro righe, per disegnare le pareti e gli ostacoli leggendo la lista; il Controller e la riga di stato sono *identici* a quelli del capitolo 3, copiati senza una modifica. Il `main` aggiunge due mensole:

```
public static Mondo scenario() {
    Mondo mondo = new Mondo(8, 6);
    mondo.aggiungi(new Ostacolo(1.0, 3.0, 2.5, 0.2));          // due mensole
    mondo.aggiungi(new Ostacolo(4.5, 1.8, 2.5, 0.2));
    return mondo;
}
```



Palline, terza versione: pareti, due mensole, palline che si urtano, palline ferme sul pavimento e sulle mensole.

Palline ora si comporta come ci si aspetta da delle palline. Cadono con la stessa gravità su ogni computer e, in più, con lo stesso risultato; rimbalzano e si fermano quando l'energia è finita, senza tremare; si urtano fra loro, le piccole rimbalzano via dalle grandi, le grandi spostano appena le piccole; si fermano sulle mensole e scivolano giù dagli spigoli; una pallina veloce non attraversa più niente. E il *toc* arriva solo per gli urti veri. Tutto questo è successo dentro il Model: chiunque avesse scritto una View per la Palline del capitolo precedente la può usare oggi senza accorgersi del cambiamento, se non per il fatto che le palline si comportano meglio. È ciò che si intende quando si dice che un'architettura "paga": non in astratto, ma nel giorno in cui si riscrive una parte e le altre restano al loro posto.

Ricetta: un motore di palline in un passo fisso

1. **Il tempo:** un passo fisso piccolo (5 ms), un accumulatore del tempo ricevuto, un tetto al `dt` (0,1 s); `stepNext(dt)` fa tanti passi quanti ce ne stanno.
2. **Il moto:** prima la velocità, poi la posizione con la velocità nuova; se qualcuno corre, sottopassi finché nessuno scatto supera mezzo raggio.
3. **Gli urti fra palline:** per ogni coppia compenetrata, separare in proporzione alle masse e, solo se si avvicinano, l'impulso lungo la normale.
4. **Le superfici fisse:** per ogni pallina, pavimento, pareti e ostacoli con la stessa funzione: fuori dalla superficie, e solo se si avvicina, normale riflessa con la restituzione e tangente frenata; quiete sotto la soglia su una superficie d'appoggio.
5. **Gli annunci alla fine** del passo, a stato coerente; **i test** su energia, pareti, determinismo e quantità di moto prima di dire che funziona.

Esercizi

Esercizio 4.1: Quanti passi (⋆)

Il passo fisso è di 5 ms e `DT_MASSIMO` di 0,1 s. Il Controller chiama `stepNext` con questa sequenza di `dt`: 16, 17, 16, 250, 16 ms. Quanti passi fa il mondo a ogni chiamata, e quanto resta nell'accumulatore dopo ciascuna? Quanto tempo di mondo è stato simulato in tutto, e quanto è andato perso?

Esercizio 4.2: Tre rimbalzi (⋆)

Una pallina è lasciata ferma da 2 metri (misurati al centro, raggio trascurabile) con restituzione 0,5 e senza attrito. A che altezza risale dopo il primo, il secondo e il terzo rimbalzo? Dopo quanti rimbalzi la velocità di risalita scende sotto la soglia di quiete di 0,15 m/s? Con la restituzione a 0,8?

Esercizio 4.3: La normale giusta (⋆)

Un ostacolo occupa il rettangolo da $(2, 1)$ a $(4, 2)$. Per ciascuna posizione del centro di una pallina di raggio 0,3, dire qual è il punto più vicino, se c'è contatto e quale normale esce: (a) $(3, 2{,}2)$; (b) $(4{,}2, 2{,}2)$; (c) $(1{,}8, 1{,}5)$; (d) $(3, 0{,}9)$; (e) $(3, 1{,}5)$. In quale caso la superficie conta come appoggio?

Esercizio 4.4: Chi si muove di più (⋆⋆)

Due palline con raggi 0,1 e 0,3 (masse in rapporto 1 a 9) si compenetrano di 2 cm lungo la congiungente. Di quanto si sposta ciascuna nella separazione? Poi, senza gravità, la piccola arriva a 3 m/s contro la grande ferma, urto frontale, restituzione 1: quali sono le velocità dopo l'urto? (Usare la formula dell'impulso.) E se la grande fosse infinitamente pesante?

Esercizio 4.5: Sottopassi (⋆⋆)

Nel mondo ci sono palline di raggio minimo 0,1 e la più veloce va a 12 m/s. Quanti sottopassi fa il passo fisso di 5 ms? Un mattone è spesso 4 cm: la pallina può attraversarlo? Se sì, qual è il modo più semplice di impedirlo con gli strumenti del capitolo, e che cosa costa?

I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.

Esercizio 4.6: La legge del rimbalzo (⋆)

Un programma senza finestra che lascia cadere una pallina da varie altezze, con varie restituzioni, nel `Mondo` di questo capitolo, registra l'altezza massima raggiunta dopo ogni rimbalzo ascoltando gli eventi e campionando la quota, e la confronta con la legge $h_n = h_0\, e^{2n}$: stampa una tabella con l'altezza misurata, quella prevista e l'errore percentuale per i primi cinque rimbalzi, e spiega in un commento da dove viene l'errore residuo (il passo, la correzione di penetrazione, la quiete).

Soluzione: progetto `sol-dxtr-02-c04`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 4.3: Misurare al momento sbagliato

L'altezza massima non è quella al momento dell'evento `Rimbalzo` (che arriva a terra), né quella a un istante fisso dopo: va cercata campionando la quota a ogni passo e tenendo il massimo fra un rimbalzo e il successivo. Chi legge la posizione "un po' dopo" il rimbalzo misura un'altezza qualsiasi.

Esercizio 4.7: Il piano inclinato (⋆⋆)

Aggiungere al mondo un ostacolo nuovo, `Rampa`: un segmento da (x_1, y_1) a (x_2, y_2) con spessore trascurabile, su cui le palline rotolano e rimbalzano. Serve il punto più vicino di un segmento (proiezione del centro sul segmento, limitata agli estremi) e la normale che ne esce; la rampa è superficie d'appoggio se la normale punta abbastanza in alto. Mostrare con una View le palline che scendono lungo la rampa, si fermano in fondo e si accumulano. Un test senza finestra verifica che una pallina lasciata sopra la rampa arriva in fondo e si ferma sul pavimento, mai dentro la rampa.
Suggerimento: il punto più vicino di un segmento AB a P è $A + t\,(B - A)$ con $t = \mathrm{clamp}\big(\frac{(P-A)\cdot(B-A)}{|B-A|^2}, 0, 1\big)$.

Soluzione: progetto `sol-dxtr-02-c04`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 4.4: La rampa che non frena

Su una rampa la componente tangente è quasi tutta la velocità: con il solo attrito d'urto la pallina frena solo quando tocca, e tocca a ogni passo. Il risultato è un rallentamento che dipende dal passo. Se la pallina deve rotolare in modo credibile, l'attrito lungo la rampa va applicato per unità di tempo (una frazione per secondo di contatto), non per urto.

Esercizio 4.8: Il biliardo (⋆⋆)

Un tavolo senza buche visto dall'alto: niente gravità (`setGravita(0)`), quattro sponde con restituzione 0,9, sette palline ferme a triangolo e una bianca. Il rotolamento sul panno frena ogni pallina di una frazione della velocità al secondo (un attrito viscoso: $\vec{v} \leftarrow \vec{v}\,(1 - k\,\Delta t)$ in ogni passo, con $k \approx 0{,}6$), e sotto la soglia di quiete la pallina si ferma anche senza una superficie d'appoggio. Il Controller tira con il mouse: si trascina dalla bianca e la velocità è proporzionale al segmento tracciato, mostrato come stecca. Il mondo deve aspettare che tutte le palline siano ferme prima di accettare un altro colpo.

Soluzione: progetto `sol-dxtr-02-c04`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 4.5: L'attrito nel posto sbagliato

L'attrito del panno agisce *sempre*, non solo negli urti: se lo si mette in `contatto` le palline scorrono per sempre. Va nel moto, a ogni sottopasso, e la quiete va decisa lì, confrontando la velocità con la soglia; la quiete delle superfici d'appoggio del capitolo non serve, perché qui le palline non cadono.

Esercizio 4.9: Il flipper minimo (⋆⋆⋆)

Un piano inclinato visto dall'alto: la gravità è verso il basso dello schermo ma ridotta ($3\ \mathrm{m/s^2}$), due pareti laterali e una in alto, due *respingenti* circolari fissi con restituzione 1,3 (restituiscono più energia di quanta ricevono: una superficie attiva), due *alette* in basso, segmenti che ruotano attorno a un perno di 40 gradi quando si tengono premuti i tasti Z e M, e il foro in fondo da cui la pallina esce. Un punteggio per ogni respingente colpito, annunciato come evento.
Suggerimento: un'aletta che ruota ha una velocità propria nel punto di contatto ($\omega \times r$): per il rimbalzo si usa la velocità relativa fra la pallina e quel punto, come nell'urto fra due palline, ma con la massa dell'aletta infinita.

Soluzione: progetto `sol-dxtr-02-c04`, package `it.unibs.pajc.redolfi.e09`.

Attenzione 4.6: La restituzione sopra uno

Un respingente con $e > 1$ aggiunge energia a ogni urto, e il test dell'energia del capitolo *deve* fallire su questo mondo: non è un errore del test, è il gioco che vuole così. Resta però un limite da imporre: una pallina che rimbalza fra due respingenti vicini guadagna energia a ogni colpo e in pochi secondi va a velocità assurde. Un tetto alla velocità dopo l'impulso è la cosa più semplice.

Risposte

Risposta 4.1. 16 ms: 3 passi (15 ms), resta 1. 17: accumulatore 18, 3 passi, resta 3. 16: 19, 3 passi, resta 4. 250: tagliato a 100, accumulatore 104, 20 passi, resta 4. 16: 20, 4 passi, resta 0. In tutto $3+3+3+20+4=33$ passi, 165 ms di mondo; il tempo ricevuto era 315 ms, 150 dei quali buttati dal taglio (il quarto fotogramma ne portava 250 e ne è passato un decimo). Il mondo è rimasto indietro di 150 ms rispetto all'orologio, una volta, e poi ha ripreso il passo: meglio di venti passi arretrati che un giro del timer lunghissimo.

Risposta 4.2. L'altezza va con il quadrato della velocità e la velocità si riduce di e a ogni rimbalzo: $h_n = h_0\, e^{2n}$. Con $e = 0{,}5$: 0,5 m, 0,125 m, 0,031 m. La velocità di arrivo è $\sqrt{2gh_0} = 6{,}26$ m/s e quella di risalita dopo n rimbalzi $6{,}26 \cdot 0{,}5^n$: scende sotto 0,15 m/s per $n = 6$ (0,098 m/s; al quinto è ancora 0,196). Con $e = 0{,}8$ servono $n = 17$ rimbalzi ($6{,}26 \cdot 0{,}8^{17} = 0{,}14$). Nel mondo vero l'attrito e la correzione di penetrazione anticipano un po' la quiete.

Risposta 4.3. (a) Punto più vicino $(3, 2)$, distanza $0{,}2 < 0{,}3$: contatto, normale $(0, 1)$, appoggio. (b) $(4, 2)$, distanza $\sqrt{0{,}04 + 0{,}04} = 0{,}283 < 0{,}3$: contatto sullo spigolo, normale $(0{,}71, 0{,}71)$, componente y pari a 0,71 volte la lunghezza: appoggio, al limite. (c) $(2, 1{,}5)$, distanza 0,2: contatto laterale, normale $(-1, 0)$, non appoggio. (d) $(3, 1)$, distanza 0,1: contatto da sotto, normale $(0, -1)$, non appoggio (la pallina non può stare appesa). (e) Il centro è dentro: punto più vicino è il centro stesso, distanza 0, il codice usa la normale $(0, 1)$ e la spinge fuori verso l'alto di un raggio intero; è il caso del tunneling, da evitare con i sottopassi.

Risposta 4.4. Le masse sono $\pi\,0{,}01$ e $\pi\,0{,}09$: la separazione di 2 cm si divide in proporzione inversa, 1,8 cm alla piccola e 0,2 cm alla grande. Nell'urto, con $m_a = 1$, $m_b = 9$ (in unità di $\pi\,0{,}01$), $v_n = 0 - 3 = -3$, $e = 1$: $j = 2 \cdot 3/(1 + 1/9) = 5{,}4$; la piccola passa a $3 - 5{,}4 = -2{,}4$ m/s (torna indietro), la grande a $0 + 5{,}4/9 = 0{,}6$ m/s. Quantità di moto: $3 = -2{,}4 + 9 \cdot 0{,}6$. Con la grande infinitamente pesante $1/m_b = 0$, $j = 6$: la piccola riparte a -3 m/s e la grande non si muove, come contro una parete.

Risposta 4.5. Lo scatto per passo è $12 \cdot 0{,}005 = 6$ cm, il limite è mezzo raggio, 5 cm: $\lceil 6/5 \rceil = 2$ sottopassi, scatti di 3 cm. Un mattone di 4 cm non viene attraversato con scatti di 3 cm (un centro che era sopra di almeno un raggio arriva al più a 3 cm più in basso, e il contatto scatta appena il bordo entra). Con scatti maggiori dello spessore più il raggio sì. Il rimedio più semplice è abbassare il limite dello scatto (un quarto di raggio, o il minimo fra mezzo raggio e lo spessore minimo degli ostacoli): costa più sottopassi, cioè più tempo di calcolo, ma solo quando qualcuno corre.

Capitolo 5

Comportamenti, stati e comandi

Il mondo del capitolo 4 rimbalza bene, ma è un mondo di palline tutte uguali, e la Palline promessa nel capitolo 1 aveva tre materiali, un menu, la pausa, *Ctrl+Z* e *Riproduci*. Ognuna di queste cose, scritta nel modo che viene naturale, produce uno dei difetti visti all'inizio: la catena di `if` sul nome del materiale, i tre booleani che si contraddicono, il codice del tasto che muove direttamente il mondo e che nessuno può annullare. Questo capitolo introduce i quattro pattern che risolvono questi problemi nei giochi, e non solo: *Strategy* per i comportamenti intercambiabili, *Template Method* per uno scheletro comune con i dettagli lasciati alle sottoclassi, *State* per gli stati del gioco, *Command* per l'input che diventa un oggetto. Sono i quattro pattern più usati in assoluto, e Palline li usa tutti e quattro nello stesso momento; alla fine del capitolo avrà i materiali, gli stati, l'annullamento e la riproduzione, e il mondo del capitolo 4 sarà cambiato solo per far posto ai materiali.

5.1 Comportamenti intercambiabili: Strategy

Tre materiali: la gomma rimbalza bene, il piombo quasi per niente ed è pesante, la bolla di sapone rimbalza quasi tutto, pesa pochissimo e *scoppia* se l'urto è forte. Il primo tentativo è quello della Palline del capitolo 1: una stringa, e una catena di `if` che per ogni nome restituisce il coefficiente giusto. Funziona per la restituzione; per la densità serve una seconda catena identica; per lo scoppio della bolla una terza, con dentro del codice e non un numero; e ogni materiale nuovo vuol dire trovare tutte le catene e aggiungere un ramo a ciascuna. Il compilatore non aiuta: un nome scritto male finisce nel ramo di default e nessuno se ne accorge.

Il secondo tentativo è un record `Materiale` con tre campi, il nome, la restituzione e la densità. Molto meglio: i dati di un materiale stanno insieme, e la pallina porta il suo materiale invece di un nome da interpretare. Ma lo scoppio della bolla non è un numero, è un *comportamento*: "dopo un urto a più di tanto sparisco". Quando ciò che distingue le varianti è codice e non dati, il pattern è *Strategy*: un'interfaccia che dichiara il comportamento, una classe per variante, e l'oggetto che ha bisogno del comportamento ne tiene una e la chiama senza sapere quale sia.

```
public sealed interface Materiale permits Gomma, Piombo, Bolla {

    double restituzione();

    double densita();

    /** Che cosa succede dopo un urto a quella velocita: di solito niente. */
    default boolean sopravvive(double impatto) {
        return true;
    }
}

public record Gomma() implements Materiale {
    public double restituzione() { return 0.8; }
    public double densita()      { return 1.0; }
}

public record Bolla() implements Materiale {
    public double restituzione() { return 0.95; }
    public double densita()      { return 0.05; }

    @Override
    public boolean sopravvive(double impatto) {
        return impatto < 2.5;                          // m/s
    }
}
```

Il piombo è come la gomma con altri numeri (0,15 e 11). La pallina tiene un `Materiale` invece del coefficiente di restituzione del capitolo precedente, e ne ricava massa e restituzione:

```
public class Pallina extends Entita {

    private final double raggio;
    private final Materiale materiale;

    public double massa() {
        return materiale.densita() * Math.PI * raggio * raggio;
    }
    public double restituzione() { return materiale.restituzione(); }
    public Materiale materiale() { return materiale; }
```

E nel contatto con una superficie, dove prima c'era solo il rimbalzo, il mondo chiede al materiale se la pallina sopravvive:

```
if (!p.materiale().sopravvive(-vn)) {          // la Strategy decide
    p.scoppia();
    scoppiate.add(p);
    schizzo(p.posizione(), n, -vn, 16);
    return;
}
```

Il mondo non sa che esistono le bolle. Sa che ogni pallina ha un materiale e che il materiale

sa dire se si sopravvive a un urto; la gomma e il piombo rispondono sempre sì con il metodo `default`, la bolla risponde a modo suo. Un quarto materiale (il vetro, che scoppia solo sopra i 6 m/s e rimbalza poco) è una classe nuova e nessuna riga del mondo.

Definizione 5.1: Strategy

Il pattern *Strategy* rende intercambiabile un comportamento: un'interfaccia dichiara l'operazione, ogni variante è una classe che la implementa, e l'oggetto che ne ha bisogno (il *contesto*: qui la pallina e il mondo) ne tiene un'istanza e la chiama attraverso l'interfaccia. Si aggiunge una variante scrivendo una classe, non toccando il contesto; si cambia variante a tempo di esecuzione sostituendo l'oggetto. Le interfacce funzionali e le lambda sono Strategy in piccolo: un `Comparator` è la strategia di ordinamento di una lista.

5.1.1 Strategy o `switch`?

C'è un posto in cui il materiale decide qualcosa e la Strategy non va bene: l'aspetto. La gomma è arancione, il piombo grigio con un riflesso, la bolla un cerchio azzurro trasparente. Il modo "a Strategy" sarebbe un metodo `disegna(Graphics2D g)` nel materiale, e il materiale finirebbe per importare AWT, violando la regola del capitolo 2: l'aspetto è una decisione della View, non del Model. La View, allora, deve distinguere i materiali dall'esterno. Qui torna utile che l'interfaccia sia `sealed`: l'elenco delle implementazioni è chiuso, e uno `switch` con i pattern di tipo JAVA 21 le può enumerare, con il compilatore che garantisce che non ne manchi una.

```
/** L'aspetto dipende dal materiale: lo decide la View, non il materiale. */
private void disegna(Graphics2D g2, Pallina p) {
    double s = scala(), r = p.raggio() * s;
    double x = worldToScreenX(p.posizione().x()), y =
    worldToScreenY(p.posizione().y());
    Ellipse2D forma = new Ellipse2D.Double(x - r, y - r, 2 * r, 2 * r);
    switch (p.materiale()) {
        case Gomma gomma -> {
            g2.setColor(p.ferma() ? new Color(0x8A6A20) : new Color(0xE0A020));
            g2.fill(forma);
        }
        case Piombo piombo -> {
            g2.setColor(new Color(0x6A7080));
            g2.fill(forma);
            g2.setColor(new Color(0xA0A8B8));
            g2.fill(new Ellipse2D.Double(x - r / 2, y - r / 2, r / 2, r / 3));
        }
        case Bolla bolla -> {
            g2.setColor(new Color(0x60C0FF));
            g2.draw(forma);
        }
    }
}
```

Lo `switch` non ha `default`: se domani si aggiunge il vetro ai `permits`, questo metodo smette di compilare finché qualcuno non decide che aspetto ha il vetro. È esattamente il

comportamento che si vuole dalla View, e che la catena di `if` sul nome non dava. Le due tecniche si dividono il lavoro così.

	Strategy (metodo nell'interfaccia)	**`switch` su un `sealed`**
Chi conosce le varianti	nessuno: il contesto vede solo l'interfaccia	chi fa lo `switch`, tutte
Aggiungere una variante	una classe nuova, nessun'altra modifica	una classe nuova e un ramo in ogni `switch` (il compilatore li elenca)
Aggiungere un'operazione	un metodo in più in ogni variante	uno `switch` nuovo, dove serve
Quando conviene	l'operazione *appartiene* alla variante (la fisica del materiale) e l'insieme può crescere, anche da fuori	l'operazione appartiene a chi guarda (la View, un file, una statistica) e l'insieme è chiuso

Nei programmi veri le due cose convivono, come in Palline: il materiale decide la fisica (Strategy), la View decide i colori (`switch`). Una terza via, che il codice storico di molti giochi usa, è una *mappa* da classe a disegnatore, `Map<Class<?>, Renderer>`, riempita all'avvio: è una Strategy scelta dal tipo, adatta quando le varianti arrivano da fuori (un *plugin*) e lo `switch` non le può conoscere; paga con il fatto che una sottoclasse di `Gomma` non trova il suo disegnatore, perché la mappa cerca la classe esatta.

Attenzione 5.1: La Strategy che sa troppo

Il segno di una Strategy sbagliata è un parametro che le dice chi la chiama, o un `instanceof` dentro di lei per sapere su che cosa sta lavorando. Un `Materiale` con un metodo `disegna(Graphics2D g, boolean ferma, boolean selezionata)` è un pezzo di View finito nel Model; una `sopravvive` che controlla `if (mondo instanceof MondoDiProva)` è un Model che conosce i suoi test. Se la variante ha bisogno di sapere chi la usa, l'operazione sta dalla parte sbagliata: va in uno `switch` di chi la usa.

5.2 Uno scheletro da completare: Template Method

I rimbalzi ora fanno uno *schizzo*: una manciata di schegge che partono dal punto dell'urto, volano per mezzo secondo frenate dall'aria, e svaniscono. Le schegge si muovono come le palline (hanno una posizione, una velocità, sentono la gravità) ma non come le palline: sentono la gravità meno, non rimbalzano su niente, muoiono dopo mezzo secondo. Scrivere una seconda classe con lo stesso `stepNext` copiato e tre righe diverse è la cosa più naturale e la più fragile: quando il capitolo 4 ha cambiato l'ordine delle due righe di Eulero l'avrebbe dovuto cambiare in due posti.

Il pattern per questa situazione è il *Template Method*: una classe base scrive lo *scheletro* dell'operazione una volta per tutte, fissandone la struttura, e lascia alle sottoclassi dei punti precisi in cui dire la loro, i *metodi gancio* (*hook*), che hanno un comportamento di default e si ridefiniscono solo quando serve.

```
public abstract class Entita {

    protected Vec2 posizione;
    protected Vec2 velocita = Vec2.ZERO;
    private double eta;                          // secondi dalla nascita

    /** Un passo di dt secondi con la gravita g: schema fisso, dettagli no. */
    public final void stepNext(double dt, Vec2 g) {
        velocita = velocita.piu(g.per(dt * pesoDellaGravita()));
        posizione = posizione.piu(velocita.per(dt));
        eta += dt;
        dopoIlMoto(dt);
    }

    /** Quanto conta la gravita: 1 di norma, meno per cio' che galleggia. */
    protected double pesoDellaGravita() {
        return 1;
    }

    /** Che cos'altro succede a ogni passo: niente, di norma. */
    protected void dopoIlMoto(double dt) {
    }

    /** Se e' ancora nel mondo: chi dice no viene tolto alla fine del passo. */
    public boolean viva() {
        return true;
    }
}
```

`stepNext` è `final`: nessuna sottoclasse può riscriverlo, e l'ordine delle due righe di Eulero è deciso una volta sola. I tre ganci sono `protected`, hanno un default, e una sottoclasse ne ridefinisce quanti vuole. `Pallina` ridefinisce `viva` (una pallina scoppiata non c'è più) e basta; la scheggia tutti e tre:

```
public class Particella extends Entita {

    private static final double VITA = 0.5;

    public Particella(Vec2 posizione, Vec2 velocita) {
        super(posizione);
        this.velocita = velocita;
    }

    @Override
    protected double pesoDellaGravita() { return 0.4; }

    @Override
    protected void dopoIlMoto(double dt) {
        velocita = velocita.per(1 - 2 * dt);            // l'aria la frena
    }

    @Override
    public boolean viva() { return eta() < VITA; }
}
```

Nel passo fisso del mondo palline e schegge avanzano con la stessa chiamata, `stepNext(dt, g)`, e alla fine del passo chi non è più vivo viene tolto con `removeIf(e -> !e.viva())`: le bolle scoppiate e le schegge invecchiate escono dal mondo con la stessa riga, senza che il mondo sappia perché.

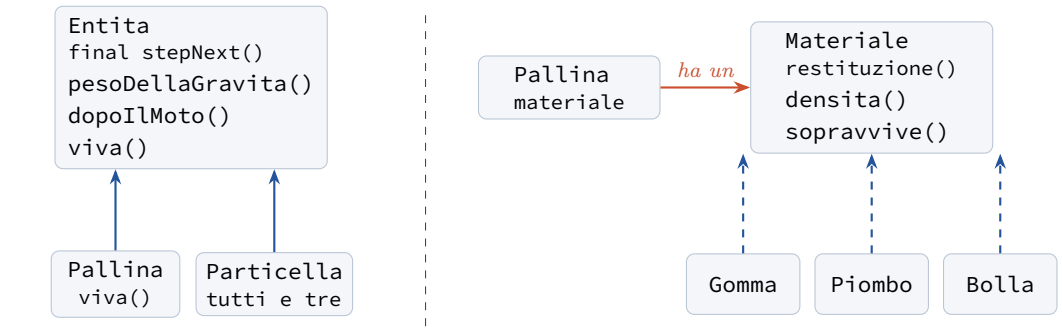
Definizione 5.2: Template Method

Il pattern *Template Method* definisce in una classe base lo scheletro di un'operazione, come sequenza fissa di passi, e delega alcuni passi a metodi che le sottoclassi possono ridefinire (i *ganci*), con un default nella base. La struttura è decisa una volta, nella base, e protetta (il metodo scheletro è `final`); le varianti cambiano solo i passi che devono. È il pattern dell'ereditarietà "fatta bene": la sottoclasse non riscrive, completa.

Attenzione 5.2: Il gancio chiamato dal costruttore

Un costruttore di `Entita` che chiamasse `pesoDellaGravita()` per inizializzare un campo ("così lo calcolo una volta sola") eseguirebbe il metodo della sottoclasse *prima* che il costruttore della sottoclasse sia partito, con i campi della sottoclasse ancora a zero o `null`: la `Particella` risponderebbe con il valore sbagliato, o lancerebbe una `NullPointerException` su un campo proprio. Java lo permette e non avvisa. La regola: **un costruttore non chiama metodi ridefinibili**; i ganci si chiamano dai metodi, quando l'oggetto è completo. Il gancio vuoto dimenticato è l'altra trappola: una classe che doveva ridefinire `viva` e non lo fa produce entità immortali, e nessun errore. Dove un gancio *deve* essere ridefinito, lo si dichiara `abstract` e il compilatore lo pretende.

Template Method e Strategy risolvono lo stesso problema, variare una parte di un comportamento, con i due strumenti opposti di Java: l'ereditarietà il primo, la composizione la seconda. La figura li mette a confronto.



Template Method: la variante estende *e ridefinisce i ganci*

Strategy: la variante è un oggetto che il contesto ha

Lo stesso bisogno, due strumenti: a sinistra la variante è una sottoclasse, a destra un oggetto sostituibile. Linea piena: eredita; tratteggiata: implementa; rossa: tiene un riferimento.

Quando l'uno e quando l'altro? La Strategy si può cambiare a tempo di esecuzione (una pallina potrebbe cambiare materiale; una `Particella` non può diventare una `Pallina`) e si può condividere (un solo oggetto `Gomma` per tutte le palline di gomma); il Template Method tiene lo stato della variante nello stesso oggetto e accede ai campi della base. La regola di Bloch, "preferire la composizione all'ereditarietà", dice che in caso di dubbio si parte dalla Strategy; il Template Method resta la scelta giusta quando le varianti sono davvero *specie* di una stessa cosa, con uno scheletro comune corposo, come le entità di un mondo.

5.3 Gli stati del gioco: State

Palline ha un menu iniziale (il mondo fermo e una scritta), il gioco in corso, la pausa e la riproduzione. I tre booleani del capitolo 1 erano il primo modo di scriverlo, e il peggiore: otto combinazioni, quattro insensate, e ogni metodo che deve ricordarsi di controllare i booleani giusti. Il secondo modo è un `enum` con quattro valori e uno `switch` in ogni punto in cui lo stato conta: nel passo del timer (il tempo passa solo in gioco e in riproduzione), nel click (lascia una pallina solo in gioco), nei tasti (spazio avvia dal menu, mette in pausa dal gioco, riprende dalla pausa). È un passo avanti enorme: uno stato solo alla volta, per costruzione. Ma gli `switch` sono tre e crescono con gli stati: aggiungere "partita finita" vuol dire trovarli tutti.

Il terzo modo rovescia la tabella: invece di tre `switch` con quattro rami, quattro *oggetti* con tre metodi. Ogni stato è una classe che dice che cosa fanno il tempo, il click e i tasti *finché si è in quello stato*, e il Controller tiene l'oggetto dello stato corrente e gli gira tutto:

```
public interface Stato {

    default void entra(Controller c) { }

    default void stepNext(Controller c, double dt) { }

    default void punto(Controller c, Vec2 p) { }

    default void tasto(Controller c, int codice) { }

    /** Il messaggio che la View mostra in questo stato (null: nessuno). */
    String messaggio();
}
```

I default dicono "non succede niente", che è la risposta giusta nella maggior parte delle caselle della tabella: in pausa il tempo non passa e i click non fanno niente, e `Pausa` non scrive nemmeno quei metodi. Scrive solo che cosa fa lo spazio:

```
public class Pausa implements Stato {

    @Override
    public void tasto(Controller c, int codice) {
        if (codice == KeyEvent.VK_SPACE)
            c.cambia(new InCorso());
    }

    @Override
    public String messaggio() { return "pausa"; }
}
```

`InCorso` è lo stato pieno: il tempo fa avanzare il mondo, il click lascia una pallina del materiale scelto, i tasti sono comandi e cambi di stato:

```
public class InCorso implements Stato {

    @Override
    public void stepNext(Controller c, double dt) {
        c.mondo().stepNext(dt);
    }

    @Override
    public void punto(Controller c, Vec2 p) {
        c.esegui(new Lascia(p, 0.25, c.materiale()));
    }

    @Override
    public void tasto(Controller c, int codice) {
        switch (codice) {
            case KeyEvent.VK_SPACE -> c.cambia(new Pausa());
            case KeyEvent.VK_R -> c.cambia(new Riproduzione());
            case KeyEvent.VK_Z -> c.annulla();
            case KeyEvent.VK_C -> c.esegui(new Svuota());
            case KeyEvent.VK_1 -> c.setMateriale(new Gomma());
            case KeyEvent.VK_2 -> c.setMateriale(new Piombo());
            case KeyEvent.VK_3 -> c.setMateriale(new Bolla());
            default -> { }
        }
    }

    @Override
    public String messaggio() { return null; }
}
```

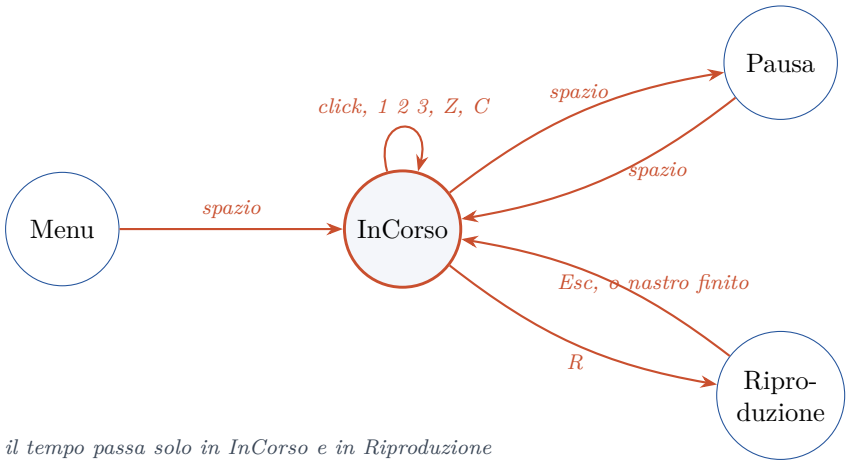
Il Controller non ha più nessun `if` sullo stato. Tiene lo stato corrente, gli passa il tempo del timer, i punti scelti sulla View e i tasti, e offre un unico metodo per cambiarlo:

```
private Stato stato;

/** L'unico posto in cui si passa da uno stato all'altro. */
public void cambia(Stato nuovo) {
    stato = nuovo;
    view.setMessaggio(nuovo.messaggio());
    nuovo.entra(this);
    view.repaint();
}

private void stepNext() {
    long adesso = System.nanoTime();
    double dt = (adesso - ultimo) / 1e9;
    ultimo = adesso;
    stato.stepNext(this, dt);
    view.repaint();
}
```

Si noti che la pausa non è più una proprietà del mondo, come nel capitolo 3: in pausa il Controller semplicemente non chiama `stepNext`. La pausa è uno stato del *gioco*, non del mondo fisico, e il mondo del capitolo 4 non ne sa nulla. È una correzione, ed è il tipo di correzione che i pattern aiutano a vedere: quando una cosa trova il suo posto, sparisce dagli altri.



Gli stati di Palline e le transizioni: ogni freccia è una chiamata a `cambia`, dentro il metodo `tasto` o `stepNext` dello stato di partenza.

Definizione 5.3: State

Il pattern *State* rappresenta ogni stato di un oggetto con una classe che implementa la stessa interfaccia: il comportamento che dipende dallo stato sta nelle classi stato, e l'oggetto (il *contesto*, qui il Controller) delega loro ogni richiesta e sostituisce l'oggetto stato quando avviene una transizione. Si usa quando uno stesso oggetto risponde in modi diversi alle stesse richieste secondo la fase in cui si trova, e le fasi sono più di due o tre. Un `enum` con uno `switch` basta quando gli stati sono pochi e il comportamento che cambia è in un posto solo.

Attenzione 5.3: La transizione a metà

Uno stato che chiama `c.cambia(...)` e poi continua a lavorare come se fosse ancora lo stato corrente è l'errore tipico: `Riproduzione.stepNext` che dopo aver chiamato `cambia(new InCorso())` continuasse a eseguire comandi dal nastro, o `InCorso.tasto` che dopo `cambia(new Pausa())` facesse ancora avanzare il mondo. La regola: **dopo `cambia` si esce**, con un `return`, o si mette `cambia` per ultimo. E un'altra: `cambia` è *l'unico* posto in cui si assegna lo stato, così tutto ciò che va fatto a ogni transizione (il messaggio della View, `entra`, un suono) sta lì e non si dimentica.

5.4 L'input come oggetto: Command

Nel click di `InCorso` non c'è una chiamata a `mondo.lascia`: c'è un oggetto `Lascia` creato e passato a `c.esegui`. Il gesto dell'utente non tocca il mondo, *crea un oggetto* che descrive ciò che va fatto, e lo dà al Controller perché lo esegua. Sembra un giro inutile, e serve a tre cose che altrimenti non si possono fare: *annullare*, *riprodurre*, e dare lo stesso comando a tasti, pulsanti, menu e rete senza scriverlo quattro volte. È il pattern *Command*.

```
public interface Comando {

    void esegui(Mondo mondo);

    /** Disfa cio' che esegui ha fatto. Di norma non si puo: non fa niente. */
    default void annulla(Mondo mondo) {
    }

    default boolean annullabile() {
        return false;
    }
}

/** Lascia una pallina. Annullare = togliere l'ultima lasciata. */
public record Lascia(Vec2 punto, double raggio, Materiale materiale)
        implements Comando {

    @Override
    public void esegui(Mondo mondo) {
        mondo.lascia(punto, raggio, materiale);
    }

    @Override
    public void annulla(Mondo mondo) {
        mondo.rimuoviUltima();
    }

    @Override
    public boolean annullabile() { return true; }
}
```

`Lascia` è un record: porta i dati del comando ed è immutabile, come un evento. Annullare una pallina lasciata è facile perché esiste l'operazione inversa, togliere l'ultima. Svuotare il

mondo no: una volta svuotato, le palline non ci sono più, e per annullare bisogna averle *salvate prima*. È il pattern *Memento* in due righe: il comando si tiene un'istantanea dello stato che sta per distruggere.

```
/** Svuota il mondo. Per annullare serve un'istantanea di prima: un Memento. */
public class Svuota implements Comando {

    private List<Pallina> prima;

    @Override
    public void esegui(Mondo mondo) {
        prima = List.copyOf(mondo.palline());
        mondo.svuota();
    }

    @Override
    public void annulla(Mondo mondo) {
        mondo.ripristina(prima);
    }

    @Override
    public boolean annullabile() { return true; }
}
```

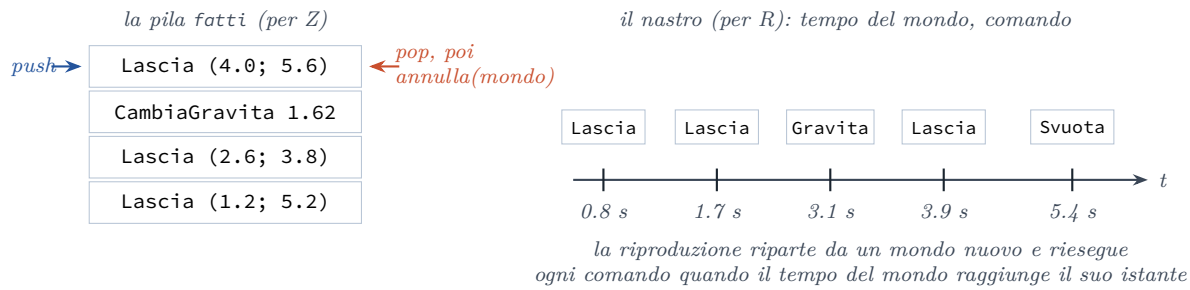
Il Controller esegue i comandi e li ricorda in due strutture: una *pila* di quelli annullabili, da cui *Z* toglie l'ultimo e lo annulla, e un *nastro* con tutti i comandi e l'istante di tempo del mondo in cui sono stati eseguiti:

```
/** Un comando con l'istante (tempo del mondo) in cui e' stato eseguito. */
public record Registrato(double tempo, Comando comando) {
}

private final Deque<Comando> fatti = new ArrayDeque<>();
private final List<Registrato> nastro = new ArrayList<>();

/** Esegue un comando; se annullabile lo impila; in ogni caso lo registra. */
public void esegui(Comando c) {
    c.esegui(mondo);
    if (c.annullabile())
        fatti.push(c);
    nastro.add(new Registrato(mondo.tempo(), c));
}

public void annulla() {
    if (!fatti.isEmpty())
        fatti.pop().annulla(mondo);
}
```



Due usi degli stessi oggetti comando: la pila per annullare (LIFO), il nastro con gli istanti per riprodurre.

La riproduzione è uno stato, e il più interessante dei quattro. Entrando, chiede al Controller un mondo *nuovo*, identico a quello iniziale, e prende il nastro; a ogni passo fa avanzare il mondo e, quando il tempo del mondo raggiunge l'istante del prossimo comando registrato, lo esegue:

```
public class Riproduzione implements Stato {

    private List<Controller.Registrato> nastro;
    private int prossimo;

    @Override
    public void entra(Controller c) {
        nastro = c.nastro();
        prossimo = 0;
        c.ricomincia();                              // un mondo nuovo
    }

    @Override
    public void stepNext(Controller c, double dt) {
        Mondo mondo = c.mondo();
        mondo.stepNext(dt);
        while (prossimo < nastro.size()
                && nastro.get(prossimo).tempo() <= mondo.tempo()) {
            nastro.get(prossimo).comando().esegui(mondo);
            prossimo++;
        }
        boolean finito = prossimo >= nastro.size()
                && mondo.palline().stream().allMatch(Pallina::ferma);
        if (finito)
            c.cambia(new InCorso());                     // si torna a giocare
    }

    @Override
    public String messaggio() { return "riproduzione  -  Esc interrompe"; }
}
```

E la scena si rifà *identica*: stesse palline, stessi rimbalzi, stesse bolle che scoppiano nello stesso istante. Non perché il nastro contenga le posizioni (contiene solo sette comandi) ma perché la fisica del capitolo 4 è a passo fisso: lo stesso mondo con gli stessi comandi agli stessi istanti di tempo del mondo fa esattamente gli stessi calcoli. Con il `dt` misurato del capitolo 2 la riproduzione sarebbe "simile". È il premio del determinismo promesso là, e il motivo per cui

i giochi registrano le partite come nastri di comandi: una partita di un'ora è qualche migliaio di comandi, non un'ora di posizioni.

Definizione 5.4: Command

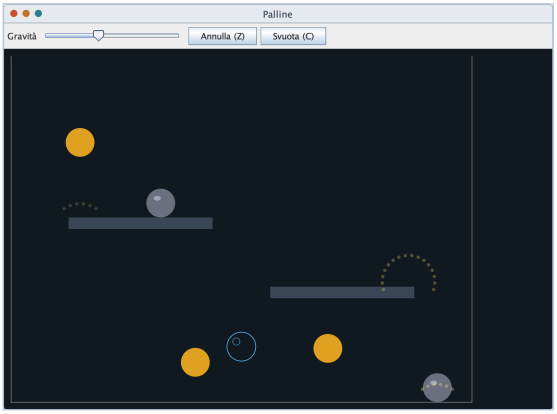
Il pattern *Command* rappresenta una richiesta come un oggetto, con un metodo che la esegue e, se serve, uno che la annulla. Chi la crea (il gesto dell'utente, un tasto, un messaggio di rete) non sa come verrà eseguita; chi la esegue (il Controller) non sa da dove viene. Gli oggetti comando si possono mettere in coda, in una pila per annullare, in un nastro per riprodurre, inviare in rete. Quando un comando distrugge informazione, per annullarlo si salva prima un'istantanea (*Memento*).

Attenzione 5.4: Il nastro con il tempo sbagliato

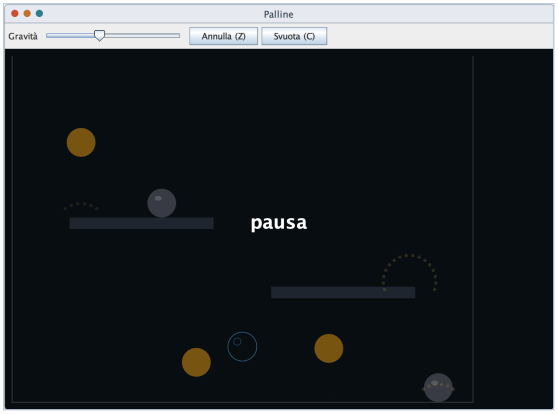
Il nastro registra il *tempo del mondo*, non quello dell'orologio. Se registrasse `System.nanoTime()`, la riproduzione dipenderebbe da quanto tempo reale è passato fra due comandi, cioè dai fotogrammi e dalle pause, e lo stesso nastro darebbe scene diverse. Il tempo del mondo avanza solo con i passi fissi: è l'unico tempo in cui "3,1 secondi" vuol dire sempre lo stesso numero di passi. Anche la casualità va trattata così: un mondo che usasse un `Random` dovrebbe ricrearlo con lo stesso seme in `ricomincia`, altrimenti i rimbalzi casuali della riproduzione sarebbero diversi dall'originale.

5.5 L'esempio guida: Palline, quarta versione

Il Controller è cresciuto per far posto allo stato corrente, al materiale scelto, alla pila e al nastro. Il resto della finestra è una barra con lo slider della gravità (che ora genera un comando `CambiaGravita`, annullabile come gli altri), i pulsanti "Annulla" e "Svuota" e una riga con i tasti. Il pannello mostra il messaggio dello stato sopra il mondo, come un velo, e disegna le schegge sfumate secondo la vita che resta loro.



In gioco: gomma, piombo, bolle e lo schizzo di un rimbalzo.



In pausa: il velo con il messaggio.

Palline ha ora tutto ciò che il capitolo 1 aveva promesso tranne i suoni veri e gli scenari: tre materiali con tre fisiche e tre aspetti, un menu, la pausa, l'annullamento di qualunque azione e la riproduzione fedele di tutta la scena. E ogni cosa sta in un posto solo: un materiale nuovo è una classe, uno stato nuovo è una classe, un comando nuovo è una classe, e il mondo

del capitolo 4 è cambiato in due punti (il materiale al posto del coefficiente, le schegge). I tre booleani non esistono più, e con loro le cinque combinazioni senza senso.

Ricetta: quattro pattern per un gioco

1. **Un comportamento che varia per tipo** (il materiale, l'arma, il nemico): un'interfaccia e una classe per variante (Strategy); se l'operazione appartiene a chi guarda e non alla variante, un'interfaccia `sealed` e uno `switch`.
2. **Oggetti della stessa specie con uno scheletro comune** (le entità): una classe base con il metodo scheletro `final` e i ganci `protected` con default (Template Method); mai ganci dal costruttore.
3. **Fasi del gioco** (menu, in corso, pausa, finito): un'interfaccia `Stato` con i default "non succede niente", una classe per fase, un solo metodo `cambia` nel contesto (State).
4. **Le azioni dell'utente**: oggetti `Comando` con `esegui` e `annulla`, una pila per annullare, un nastro con il tempo del mondo per riprodurre (Command e Memento).

Esercizi

Esercizio 5.1: Dove sta il vetro (⋆)

Si aggiunge il materiale `Vetro`: restituzione 0,3, densità 2,5, si rompe sopra i 6 m/s, e la View lo disegna bianco semitrasparente. Elencare *tutti* i file che cambiano, dire per ciascuno se la modifica è imposta dal compilatore o va ricordata, e che cosa succederebbe se ci si dimenticasse del pannello.

Esercizio 5.2: Chi chiama chi (⋆)

Nel Template Method di `Entita`, per una `Particella` che avanza di un passo, scrivere la sequenza esatta dei metodi chiamati e di quale classe è ciascuno. Che cosa succederebbe se `stepNext` non fosse `final` e `Particella` lo ridefinisse con le due righe di Eulero nell'ordine esplicito?

Esercizio 5.3: La tabella degli stati (⋆)

Scrivere la tabella stati per eventi (righe: Menu, InCorso, Pausa, Riproduzione; colonne: tempo, click, spazio, R, Esc, Z) con in ogni casella che cosa succede. Quante caselle sono "niente"? Quali dei metodi `default` di `Stato` servono a questo?

Esercizio 5.4: Annulla, annulla (⋆⋆)

L'utente lascia tre palline, svuota, lascia una pallina, poi preme Z tre volte. Che cosa c'è nel mondo dopo ciascuna pressione? E se dopo il primo Z lasciasse un'altra pallina e poi premesse Z due volte? Che cosa manca al Controller per offrire anche "ripeti" (*Ctrl+Y*), e che cosa deve succedere alla pila del ripeti quando si esegue un comando nuovo?

Esercizio 5.5: Un nastro infedele (⋆⋆)

Un collega registra nel nastro `System.nanoTime() / 1e9` invece di `mondo.tempo()`, e in `Riproduzione.stepNext` confronta con l'orologio. La riproduzione "sembra" funzionare. Descrivere due situazioni in cui la scena riprodotta è diversa dall'originale e dire perché; poi spiegare perché anche con il tempo del mondo la riproduzione sarebbe infedele se il mondo usasse il `dt` misurato del capitolo 2.

I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.

Esercizio 5.6: Il semaforo a stati (⋆)

Riscrivere il semaforo pedonale del capitolo 2 con il pattern State: una classe per fase (`VerdeAuto`, `Giallo`, `RossoAuto`), ognuna con la sua durata, il suo `stepNext` che decide quando passare alla successiva e il suo modo di rispondere alla chiamata pedonale (solo il verde la ascolta). Il Model tiene lo stato corrente e un unico metodo `cambia`. Confrontare in un commento con la versione a `enum` e dire quale si preferisce per tre fasi, e quale per dieci.

Soluzione: progetto `sol-dxtr-02-c05`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 5.5: Il tempo dentro lo stato

Lo stato che conta il proprio tempo in un campo e viene *riusato* (lo stesso oggetto `Giallo` per tutti i cicli) riparte dal tempo del ciclo precedente. O si crea uno stato nuovo a ogni transizione (`new Giallo()`), o `entra` azzera il contatore.

Esercizio 5.7: Il distributore automatico (⋆⋆)

Un distributore di bevande con gli stati `InAttesa`, `ConCredito`, `InErogazione` e `FuoriServizio`, e i comandi `Moneta(int centesimi)`, `Scelta(String prodotto)`, `Resto` e `Guasto`/`Riparato` come oggetti `Comando`. Ogni stato decide che cosa fa ciascun comando (una moneta in erogazione viene restituita, una scelta senza credito mostra il prezzo), e l'erogazione dura tre secondi di tempo misurato. Il Model annuncia gli eventi (credito cambiato, prodotto erogato, resto dato) e una View Swing con pulsanti li mostra; una prova senza finestra manda una sequenza di comandi e verifica gli stati e il credito.

Soluzione: progetto `sol-dxtr-02-c05`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 5.6: Lo stato che tocca la View

Gli stati e i comandi del distributore sono *Model*: non importano Swing e non scrivono nelle etichette. Comunicano con gli eventi del capitolo 3, e la View ascolta. Uno stato che chiama `lblCredito.setText` è il modo più rapido per rendere il Model non provabile.

Esercizio 5.8: Figure con annulla e ripeti (⋆⋆)

Un piccolo editor: un pannello su cui si disegnano rettangoli ed ellissi trascinando il mouse, si spostano trascinandoli, si cancellano con il tasto destro. Ogni azione è un `Comando` con `esegui` e `annulla` (`Aggiungi`, `Sposta` con la posizione di prima, `Cancella` con la figura tolta e il suo indice); due pile, una per annullare e una per ripetere, con *Ctrl+Z* e *Ctrl+Y*; un comando nuovo svuota la pila del ripeti. Un menu "Modifica" con le due voci abilitate solo quando c'è qualcosa da annullare o ripetere (una proprietà del Controller, ascoltata dal menu).

Soluzione: progetto `sol-dxtr-02-c05`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 5.7: Lo spostamento in cento comandi

Registrare un comando `Sposta` per ogni evento di trascinamento del mouse riempie la pila di cento spostamenti di un pixel, e *Ctrl+Z* torna indietro di un pixel alla volta. Il comando si crea una volta, al rilascio, con la posizione di partenza e quella di arrivo; durante il trascinamento si muove la figura direttamente, come anteprima.

Esercizio 5.9: Snake (⋆⋆⋆)

Il serpente su una griglia: avanza di una cella ogni T secondi di tempo misurato (T che diminuisce a ogni mela mangiata), la testa segue la direzione corrente, il corpo segue la testa; mangiare una mela lo allunga, urtare il bordo o sé stesso termina la partita. Gli stati: `Menu`, `InCorso`, `Pausa`, `Finita` (con il punteggio e un tasto per ricominciare). Le frecce sono comandi `Gira(Direzione)` che il Model applica *al prossimo passo* e non subito, e che ignora se la direzione è opposta a quella corrente. Il Model si prova senza finestra guidando il serpente verso una mela e verso il muro.
Suggerimento: due frecce premute fra un passo e l'altro devono valere entrambe, in ordine: una piccola coda di direzioni in attesa, non una sola variabile.

Soluzione: progetto `sol-dxtr-02-c05`, package `it.unibs.pajc.redolfi.e09`.

Attenzione 5.8: La direzione applicata subito

Se la freccia cambia la direzione nel momento in cui viene premuta, premere "su" e poi "sinistra" nello stesso intervallo fa girare il serpente di 180 gradi rispetto a dove andava, dentro il proprio corpo: partita finita senza colpa. La direzione è un comando in coda, consumato un passo alla volta.

Risposte

Risposta 5.1. Tre file. `Materiale`: il nome va aggiunto a `permits`, altrimenti `Vetro` non compila (imposto). `Vetro`: la classe nuova, con i due numeri e `sopravvive` ridefinito (è il lavoro). `PnlMondo`: lo `switch` nel metodo `disegna` non è più esaustivo e il compilatore si ferma finché non c'è il ramo `case Vetro` (imposto). Il mondo, il Controller, gli stati e i comandi non cambiano: il mondo chiede al materiale e il materiale risponde. Se lo `switch` avesse un `default`, il pannello compilerebbe e il vetro verrebbe disegnato come il default (o non disegnato): è il motivo per non mettere `default` in uno `switch` su un `sealed`.

Risposta 5.2. Il metodo `stepNext` di `Entita`, `final` e della base, che chiama `pesoDellaGravita` di `Particella` (ridefinito: 0,4), aggiorna velocità e posizione, incrementa l'età, e chiama il `dopoIlMoto` di `Particella` (ridefinito: frena). `viva` non viene chiamato da `stepNext` ma dal mondo, alla fine del passo, e sarà quello di `Particella`. Se `stepNext` non fosse `final` e la scheggia lo riscrivesse con l'ordine esplicito, le schegge guadagnerebbero energia come visto nel capitolo 4; vivono mezzo secondo e non si noterebbe, ma la struttura sarebbe rotta: due integratori diversi nello stesso mondo, e la prossima correzione in uno solo.

Risposta 5.3. Ventiquattro caselle. Menu: solo spazio (va in InCorso), cinque "niente". InCorso: tempo (`stepNext`), click (Lascia), spazio (Pausa), R (Riproduzione), Z (annulla), Esc niente: una sola. Pausa: solo spazio (InCorso), cinque "niente". Riproduzione: tempo (`stepNext` e riesegue), Esc (InCorso), gli altri quattro niente. In tutto quindici "niente" su ventiquattro, e sono esattamente i metodi `default` di `Stato` (`stepNext`, `punto`, `tasto`, `entra`) che le classi non ridefiniscono: senza i default, quindici metodi vuoti da scrivere a mano.

Risposta 5.4. La pila dopo le azioni è: L1, L2, L3, Svuota (con le tre palline nel memento), L4. Primo *Z*: annulla L4, resta un mondo vuoto. Secondo: annulla Svuota, tornano le tre palline. Terzo: annulla L3, restano due. Nel secondo scenario, dopo il primo *Z* si lascia L5 (pila: L1, L2, L3, Svuota, L5); due *Z* tolgono L5 e poi Svuota: tre palline. Per "ripeti" serve una seconda pila in cui `annulla` mette il comando appena annullato, e *Ctrl+Y* lo toglie e lo riesegue; eseguire un comando nuovo deve *svuotare* la pila del ripeti, perché il futuro che conteneva non è più coerente con il presente (ripetere L4 dopo aver lasciato L5 produrrebbe una storia che non è mai esistita).

Risposta 5.5. Prima situazione: durante la partita originale l'utente ha messo in pausa per dieci secondi fra due comandi; il nastro registra dieci secondi di orologio, la riproduzione li aspetta facendo avanzare il mondo, che nell'originale era fermo: le palline arrivano dieci secondi più avanti nella loro caduta. Seconda: un fotogramma lento (o il taglio di `DT_MASSIMO`) ha fatto sì che il mondo originale, in un secondo di orologio, simulasse meno di un secondo; la riproduzione, su una macchina più scarica, ne simula uno intero, e lo stesso comando cade su un mondo in uno stato diverso. Con il tempo del mondo nessuno dei due problemi esiste, perché "2,3 secondi di mondo" è sempre lo stesso numero di passi. Ma se il mondo usasse il `dt` misurato, due esecuzioni con fotogrammi diversi farebbero calcoli diversi anche a parità di tempo di mondo, e le posizioni differirebbero di poco a ogni passo e di molto dopo qualche rimbalzo: il nastro sarebbe fedele negli istanti dei comandi e infedele nel risultato.

Capitolo 6

Creare e condividere: fabbriche, risorse e singleton

I quattro pattern del capitolo 5 riguardano il *comportamento*: che cosa fanno gli oggetti e chi decide. Restano, dalla Palline del capitolo 1, tre difetti che riguardano invece la *creazione*: il suono caricato da disco a ogni urto, gli scenari copiati l'uno dall'altro, e, in mezzo, una domanda che ogni gioco si pone presto: chi crea gli oggetti, quanti ne servono, e dove stanno quelli che devono stare in un posto solo. Questo capitolo risponde con quattro pattern più piccoli dei precedenti e molto frequentati: *Singleton* (un oggetto solo, accessibile da ovunque, con i suoi costi, e l'alternativa che la dispensa preferisce), *Factory* (il `new` concentrato in un posto, e gli scenari letti da un file), *Flyweight* (un oggetto condiviso da mille) e *Object Pool* (oggetti riusati invece che ricreati). In chiusura, la tabella di tutti i pattern visti, e di quelli solo nominati. Alla fine Palline è completa come l'aveva promessa il capitolo 1; la palestra del capitolo 7 la rimonta per intero.

6.1 Le risorse, e dove stanno

Il `Suono` del capitolo 3 teneva il suo `Clip` in un campo e lo caricava nel costruttore: una volta sola, e questo bastava. Con due suoni (il *toc* del rimbalzo e il *pop* della bolla che scoppia), poi le immagini, i font, i livelli, la domanda diventa: *dove* stanno le risorse caricate, e chi le raggiunge? Devono stare in un posto solo (caricarle due volte è spreco), devono essere caricate una volta sola (all'avvio, non al primo urto), e devono essere raggiungibili da chi ne ha bisogno, che può essere lontano dal `main`: un listener del mondo, uno stato, la View.

La risposta classica, e la più usata nei giochi, è il *Singleton*: una classe di cui esiste *una sola* istanza, creata dalla classe stessa e accessibile da ovunque attraverso un metodo statico. In Java il modo giusto di scriverlo è un `enum`, e per le risorse l'enum ha una forma particolarmente elegante: una costante per risorsa, con la risorsa caricata nel costruttore della costante.

```
public enum SuoniGlobali {

    TOC(440, 0.08), POP(1200, 0.12);

    private final Clip clip;

    SuoniGlobali(double frequenza, double durata) {
        try {
            clip = AudioSystem.getClip();
            clip.open(sintetizza(frequenza, durata));
        } catch (Exception e) {
            throw new IllegalStateException("audio non disponibile", e);
        }
    }

    public void suona(double volume) {
        FloatControl gain =
                (FloatControl) clip.getControl(FloatControl.Type.MASTER_GAIN);
        double dB = 20 * Math.log10(Math.max(volume, 1e-3));
        gain.setValue((float) Math.max(gain.getMinimum(), dB));
        clip.setFramePosition(0);
        clip.start();
    }
}
```

Da qualunque punto del programma, `SuoniGlobali.TOC.suona(0.7)` suona il *toc*: nessun oggetto da passare, nessun costruttore da chiamare. La JVM garantisce che le costanti di un `enum` siano create una volta sola, nel momento in cui la classe viene inizializzata, in modo sicuro anche con più thread; non si può fare `new`, non si può clonare, e la serializzazione restituisce la stessa istanza. È l'*item 3* di *Effective Java*: "un `enum` con un solo elemento è il modo migliore di implementare un singleton".

Definizione 6.1: Singleton

Il pattern *Singleton* garantisce che una classe abbia una sola istanza e offre un punto di accesso globale a quell'istanza: il costruttore è privato, l'istanza è creata dalla classe e restituita da un metodo statico (o è la costante di un `enum`). Si usa per ciò che per sua natura è unico nel programma: la configurazione, un registro, una cache di risorse. Il suo costo è che chiunque lo usa dipende da una classe concreta e da uno stato globale.

Due forme più vecchie si incontrano in ogni codice Java esistente e vanno riconosciute. L'*holder idiom*, una classe annidata statica che tiene l'istanza (`private static class Holder { static final Suoni ISTANZA = new Suoni(); }`) e un `getInstance()` che la restituisce: la JVM crea l'istanza al primo accesso, in modo sicuro, senza sincronizzazione esplicita. E il *double-checked locking*, un `getInstance()` con un `if (istanza == null)` fuori e dentro un blocco `synchronized`: funziona solo se il campo è `volatile`, per anni è stato scritto male in metà dei programmi, e oggi è una curiosità storica. Se si deve scrivere un singleton a mano, l'enum; se lo si legge, si sappia che cosa si sta guardando.

6.1.1 Il costo del globale

`SuoniGlobali` funziona, ed è esattamente ciò che Palline faceva di sbagliato nel capitolo 1, solo fatto bene. Ma ha tre costi, che si vedono quando il programma cresce o quando lo si prova.

- **Chi lo usa non lo dichiara.** Un listener che chiama `SuoniGlobali.TOC.suona()` non ha nulla nel costruttore né nei parametri che dica "ho bisogno dell'audio": la dipendenza è nascosta nel corpo di un metodo. Chi legge la classe non la vede; chi la riusa in un altro programma scopre che si porta dietro l'audio.
- **Non si sostituisce.** Un test del mondo con i suoni registrati come listener carica la scheda audio, su un server senza audio lancia `IllegalStateException`, e in ogni caso *suona* durante il test. Non c'è modo di dire "per questo test usa un audio che non fa niente", perché il riferimento è una costante statica.
- **L'ordine di inizializzazione è implicito.** L'enum si inizializza al primo accesso, da qualunque thread lo faccia: se il primo accesso avviene dentro `paintComponent`, la scheda audio si apre sul thread degli eventi, nel mezzo di un ridisegno. Con più singleton che si usano a vicenda l'ordine diventa un rompicapo.

L'alternativa non è un pattern, è il contrario di un pattern: **si costruisce l'oggetto una volta, nel `main`, e lo si passa a chi lo usa**. Chi ne ha bisogno lo dichiara nel costruttore; chi lo crea decide quale versione dare. Si chiama *iniezione delle dipendenze* (*dependency injection*), e i framework che la automatizzano (Spring, Guice) non fanno altro che il `new` e il passaggio al posto nostro. In Palline: un'interfaccia che dice che cosa il gioco chiede all'audio, una classe che lo fa davvero, una che non fa niente, e il listener che riceve quella che gli viene data.

```java
/** Cio' che il gioco chiede all'audio. Chi lo fornisce e' un dettaglio. */
public interface Audio {

    void toc(double volume);

    void pop();

    /** L'audio che non suona: per i test, i server, il volume a zero. */
    Audio MUTO = new Audio() {
        public void toc(double volume) { }
        public void pop() { }
    };
}

/** Il listener che suona: riceve l'Audio da usare, non lo cerca da solo. */
public class Suoni implements MondoListener {

    private final Audio audio;

    public Suoni(Audio audio) {
        this.audio = audio;
    }

    @Override
    public void rimbalzo(Rimbalzo r) {
        audio.toc(Math.min(1, r.velocita() / 10));
    }

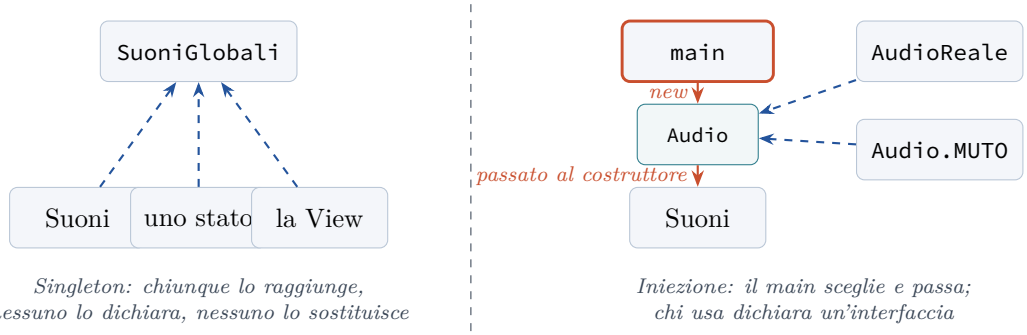
    @Override
    public void pallinaScoppiata(Pallina p) {
        audio.pop();
    }
}
```

`AudioReale` implementa `Audio` con i due `Clip` caricati nel costruttore, esattamente come il `Suono` del capitolo 3. Il `main` costruisce quello che può e lo passa:

```java
public static void main(String[] args) {
    Audio audio;
    try {
        audio = new AudioReale();                // costruito una volta, qui...
    } catch (Exception e) {
        System.err.println("niente audio: " + e.getMessage());
        audio = Audio.MUTO;
    }
    Audio scelto = audio;
    SwingUtilities.invokeLater(                  // ...e passato
            () -> finestra(scelto, null).setVisible(true));
}
```

L'audio è sempre uno, caricato una volta, come con il singleton. Ma `Suoni` dichiara nel costruttore di averne bisogno, un test gli dà `Audio.MUTO`, un server pure, e il giorno in cui

l'audio verrà da file invece che da una sinusoide cambierà `AudioReale` e nient'altro. Il prezzo è un parametro in più nei costruttori lungo la strada che va dal `main` a chi usa l'audio: la figura mostra le due strutture.



Lo stesso oggetto unico, raggiunto in due modi: attraverso una costante globale, o passato da chi lo ha creato a chi lo usa.

Osservazione 6.1

Il Singleton non è sbagliato: è *costoso*, e il costo si paga in un momento diverso da quello in cui lo si scrive. La regola pratica di questa dispensa: un oggetto unico si *costruisce* una volta nel `main` e si *passa*; se deve essere davvero raggiungibile da ovunque (la configurazione letta all'avvio, un registro dei log) si usa l'enum, dichiarando un'interfaccia che permetta di sostituirlo nei test. La cosa da non fare è la terza: un singleton perché "è comodo non passare il parametro". Quel parametro è la documentazione di chi dipende da che cosa.

6.2 Chi crea: Factory

Gli scenari della Palline del capitolo 1 erano sei metodi copiati, ciascuno una sequenza di `new` e di `add`, e il settimo non è mai stato scritto. Il problema non è il `new` in sé, è che i `new` di uno scenario sono *dati* (dove sta ogni pallina, di che materiale, dove sono le mensole) scritti come *codice*: cambiare la piramide vuol dire ricompilare, e aggiungere uno scenario vuol dire programmare. La soluzione è una classe che sa costruire un mondo da una *descrizione*, e la descrizione sta in un file di testo:

```
mondo 8 6
# una piramide di piombo con le bolle sopra
ostacolo 2.0 1.0 4.0 0.2
pallina 3.0 1.5 0.3 piombo
pallina 3.6 1.5 0.3 piombo
pallina 3.3 2.1 0.3 piombo
pallina 2.0 5.5 0.25 bolla
```

Una riga per elemento, con il tipo in testa e i numeri dopo; le righe vuote e i commenti si saltano. La *fabbrica* legge il testo riga per riga e fa i `new` che prima erano sparsi nei sei metodi:

```
public class FabbricaScenari {

    public static Mondo daTesto(String testo) {
        Mondo mondo = null;
        for (String riga : testo.split("\n")) {
            riga = riga.strip();
            if (riga.isEmpty() || riga.startsWith("#"))
                continue;
            String[] c = riga.split("\\s+");
            switch (c[0]) {
                case "mondo" -> mondo = new Mondo(d(c[1]), d(c[2]));
                case "gravita" -> richiesto(mondo).setGravita(d(c[1]));
                case "ostacolo" -> richiesto(mondo).aggiungi(
                        new Ostacolo(d(c[1]), d(c[2]), d(c[3]), d(c[4])));
                case "pallina" -> richiesto(mondo).lascia(
                        new Vec2(d(c[1]), d(c[2])), d(c[3]),
                        Materiale.daNome(c[4]));
                default -> throw new IllegalArgumentException(
                        "riga non capita: " + riga);
            }
        }
        return richiesto(mondo);
    }
```

`d` converte una stringa in `double` e `richiesto` controlla che la riga `mondo` sia venuta prima delle altre, con un messaggio chiaro se non è così: una fabbrica che legge dati scritti a mano deve dire *che cosa* non ha capito e *dove*, perché l'errore è nel file e non nel codice. Il materiale arriva per nome, e a tradurlo è una seconda fabbrica, più piccola, nel posto più naturale:

```
public sealed interface Materiale permits Gomma, Piombo, Bolla {

    Materiale GOMMA = new Gomma();
    Materiale PIOMBO = new Piombo();
    Materiale BOLLA = new Bolla();

    /** La fabbrica: dal nome scritto in un file al materiale condiviso. */
    static Materiale daNome(String nome) {
        return switch (nome.toLowerCase()) {
            case "gomma" -> GOMMA;
            case "piombo" -> PIOMBO;
            case "bolla" -> BOLLA;
            default -> throw new IllegalArgumentException(
                    "materiale sconosciuto: " + nome);
        };
    }
    // ...
}
```

Definizione 6.2: Factory

Una *fabbrica* (*Factory*) è un metodo o una classe il cui compito è creare oggetti, così che chi li usa non debba conoscere né la classe concreta né i dettagli della costruzione. Ne esistono tre forme: il *metodo statico di fabbrica* (`Materiale.daNome`, `List.of`, `Integer.valueOf`), che può restituire un'istanza condivisa o una sottoclasse; il *Factory Method* del libro dei pattern, un metodo ridefinibile che una sottoclasse usa per decidere che cosa creare (`crea()` in una classe base); e la *classe fabbrica*, che costruisce oggetti complessi da una descrizione (`FabbricaScenari`). Un `Supplier<T>` è una fabbrica passata come oggetto: il Controller del capitolo 5 ne riceve una per ricominciare da un mondo nuovo.

6.2.1 Le risorse del programma

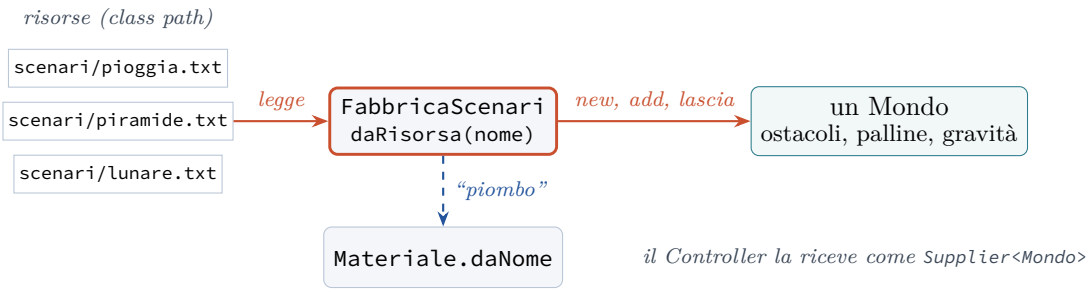
Dove sta il file dello scenario? Non in una cartella del disco scelta a mano: un programma distribuito come `.jar` non ha cartelle accanto, e un programma lanciato da Eclipse ha come *cartella di lavoro* quella del progetto, da riga di comando quella da cui lo si lancia, e un percorso relativo come `"scenari/piramide.txt"` funziona in un caso e non nell'altro. I file che fanno parte del programma (scenari, immagini, suoni, font) si mettono *nel class path*, accanto alle classi, e si leggono con `getResourceAsStream`:

```
/** Uno scenario fra le risorse del programma: /scenari/nome.txt. */
public static Mondo daRisorsa(String nome) {
    String percorso = "/scenari/" + nome + ".txt";
    try (InputStream in = FabbricaScenari.class.getResourceAsStream(percorso)) {
        if (in == null)
            throw new IllegalArgumentException("manca " + percorso);
        return daTesto(new String(in.readAllBytes(), StandardCharsets.UTF_8));
    } catch (IOException e) {
        throw new IllegalStateException(e);
    }
}
```

In Eclipse basta una cartella `res` (o `resources`) dichiarata come *source folder*: ciò che contiene finisce accanto alle classi compilate e dentro il `.jar`, e il percorso che inizia con `/` è lo stesso ovunque. Il `null` restituito quando la risorsa non c'è va controllato subito, con un messaggio che dica il percorso cercato: altrimenti l'errore si presenta come una `NullPointerException` tre chiamate più in là.

Attenzione 6.1: Il percorso che funziona solo in Eclipse

`new FileReader("res/scenari/piramide.txt")` funziona finché il programma viene lanciato dalla cartella del progetto, e smette di funzionare il giorno in cui lo si lancia da un'altra cartella, lo si impacchetta in un `.jar` o lo si consegna. È l'errore più frequente nei progetti d'esame, e il più imbarazzante perché si manifesta sul computer di chi corregge. Le risorse del programma si leggono con `getResource` e `getResourceAsStream`; i file dell'utente (un salvataggio, un'esportazione) con un percorso scelto dall'utente, attraverso un `JFileChooser`.



La fabbrica: dai dati di un file agli oggetti del mondo. Chi chiede uno scenario conosce solo un nome.

Nel Controller del capitolo 5 la fabbrica entra come un `Supplier<Mondo>`, e cambiare scenario è cambiare fabbrica: `cambiaScenario` riceve una lambda che chiama `daRisorsa` con il nome scelto. La riproduzione, che chiedeva al Controller un mondo nuovo per ricominciare, non cambia: ottiene il mondo dalla fabbrica corrente, e il nastro rieseguito su quel mondo dà la stessa scena.

6.3 Uno per tutti: Flyweight

Le tre costanti `GOMMA`, `PIOMBO` e `BOLLA` nell'interfaccia `Materiale` meritano un nome, perché risolvono un problema che in un gioco vero diventa grosso. Una pallina ha un materiale; mille palline di gomma, con `new Gomma()` per ciascuna, hanno mille oggetti `Gomma` identici. Per un record vuoto sono poche decine di kilobyte, nulla; ma se il materiale portasse con sé l'immagine da disegnare (uno *sprite* di qualche centinaio di kilobyte, come in ogni gioco con una grafica) sarebbero centinaia di megabyte per mille palline uguali. Il pattern *Flyweight* (peso piuma) separa lo stato *intrinseco*, uguale per tutte le palline di gomma (restituzione, densità, l'immagine), da quello *estrinseco*, proprio di ciascuna (posizione, velocità): il primo sta in un oggetto condiviso, il secondo nell'oggetto che lo usa.

Nel codice di Palline questo è già così, e il capitolo 5 lo aveva detto en passant: i materiali sono immutabili e senza stato proprio, quindi un solo oggetto per materiale basta a tutte le palline. Le costanti e `daNome` fanno sì che quell'oggetto sia uno: `Materiale.daNome("gomma")` restituisce sempre `GOMMA`, e `p.materiale() == Materiale.GOMMA` è un confronto lecito.

Definizione 6.3: Flyweight

Il pattern *Flyweight* condivide un oggetto fra molti utilizzatori separando lo stato intrinseco (uguale per tutti, immutabile, nell'oggetto condiviso) da quello estrinseco (diverso per ciascuno, tenuto fuori e passato ai metodi quando serve). Una fabbrica restituisce l'istanza condivisa invece di crearne una nuova. In Java i record immutabili sono flyweight naturali: `Integer.valueOf` condivide i piccoli interi, `String.intern` le stringhe uguali, e un `enum` è un flyweight per costruzione.

Attenzione 6.2: Il flyweight con un campo che cambia

Un `Materiale` con un campo `colore` modificabile ("così l'utente può cambiare il colore della gomma") cambia il colore di *tutte* le palline di gomma, presenti e future, e di quelle nel nastro della riproduzione. Il flyweight regge solo se è immutabile: ciò che può cambiare per una singola pallina è stato estrinseco, e sta nella pallina. Se il colore della gomma deve cambiare, si crea un materiale nuovo e lo si dà alle palline nuove.

6.4 Riusare invece di creare: Object Pool

Le schegge dello schizzo nascono a ogni rimbalzo, sei o sedici alla volta, e muoiono dopo mezzo secondo. Con la pioggia di palline sono centinaia di `new Particella` al secondo, e altrettanti oggetti da raccogliere. Il collettore di Java è molto bravo con gli oggetti piccoli e di vita breve, e in Palline questo costo non si misura; ma l'idea che risolve il caso in cui si misura (migliaia di proiettili al secondo, oggetti costosi da costruire, una piattaforma con poca memoria) merita di essere vista, perché è in ogni motore di gioco. È l'*Object Pool*: un numero fisso di oggetti creati all'avvio, e chi ne ha bisogno ne prende uno libero, lo azzera e lo usa; quando ha finito, l'oggetto torna disponibile.

```
public class PoolParticelle {

    private final List<Particella> tutte = new ArrayList<>();

    public PoolParticelle(int quante) {
        for (int i = 0; i < quante; i++)
            tutte.add(new Particella());
    }

    /** Una particella spenta da riaccendere, o null se sono tutte in volo. */
    public Particella prendi(Vec2 posizione, Vec2 velocita) {
        for (Particella p : tutte)
            if (!p.viva()) {
                p.accendi(posizione, velocita);
                return p;
            }
        return null;
    }

    /** Le particelle, vive e spente: la View disegna solo le vive. */
    public List<Particella> tutte() {
        return Collections.unmodifiableList(tutte);
    }
}
```

Il mondo crea un pool di quattrocento particelle nel costruttore e nello schizzo chiama `particelle.prendi(punto, v)` invece di `new`; se le quattrocento sono tutte in volo, lo schizzo perde una scheggia, e nessuno se ne accorge. La `Particella` ha guadagnato `accendi`, che la riporta nel punto dato con l'età a zero, e la View salta quelle non vive. Le entità morte non vengono più tolte dalla lista: restano nel pool, spente, finché qualcuno le riaccende.

Definizione 6.4: Object Pool

Un *Object Pool* tiene un insieme di oggetti già costruiti e li presta: chi ne ha bisogno ne chiede uno, lo usa e lo restituisce (o lo marca come libero), invece di crearne uno nuovo e lasciarlo al collettore. Conviene quando gli oggetti sono costosi da costruire (una connessione di rete, un thread, un buffer grande) o quando se ne creano e distruggono moltissimi al secondo; non conviene per oggetti piccoli e pochi, dove il collettore è più veloce e il codice più semplice. I pool di thread e di connessioni di *Thread e concorrenza* sono lo stesso pattern.

Attenzione 6.3: L'oggetto riusato con lo stato del giro prima

La trappola del pool è un oggetto riacceso *a metà*: la posizione nuova ma l'età del giro precedente, così che muore subito; o la velocità vecchia, così che parte nella direzione sbagliata; o un listener registrato nella vita precedente, che riceve eventi di un oggetto che credeva morto. `accendi` deve azzerare *tutto* lo stato, e il modo per non dimenticare un campo è avere un solo metodo che lo fa (`rinasci` in `Entita`) e nessun altro punto in cui lo stato si inizializza.

6.5 La tabella dei pattern

Undici pattern in cinque capitoli, più due nominati di sfuggita. La tabella li raccoglie con il problema che risolvono e il punto di Palline in cui compaiono: è la mappa con cui rileggere il codice finale.

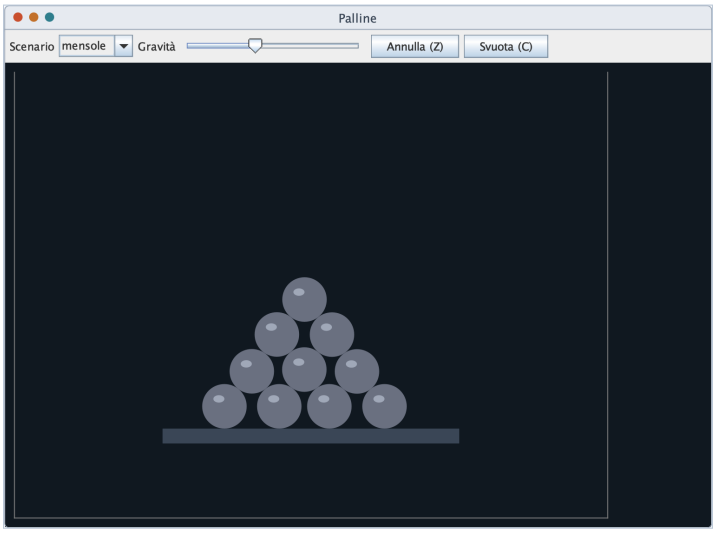
Pattern	Il problema che risolve	In Palline
MVC (cap. 2)	separare ciò che il programma è da come si mostra e come si comanda	`Mondo`, `PnlMondo`, `Controller`
Observer (cap. 3)	avvisare chi vuole sapere, senza conoscerlo	`MondoListener`, `PropertyChangeSupport`, `PuntoListener`
Strategy (cap. 5)	un comportamento intercambiabile per tipo	`Materiale` e `sopravvive`
Template Method	uno scheletro comune con i dettagli nelle sottoclassi	`Entita.stepNext` e i tre ganci
State	un oggetto che si comporta secondo la fase in cui è	`Menu`, `InCorso`, `Pausa`, `Riproduzione`
Command	un'azione come oggetto: da eseguire, annullare, registrare	`Lascia`, `Svuota`, `CambiaGravita`; pila e nastro
Memento	salvare uno stato per ripristinarlo	la lista salvata in `Svuota`
Singleton (cap. 6)	un'istanza sola, raggiungibile da ovunque	`SuoniGlobali` (e perché non lo si usa)
Factory	concentrare la creazione, costruire da una descrizione	`FabbricaScenari`, `Materiale.daNome`, il `Supplier<Mondo>`
Flyweight	un oggetto condiviso da molti	`Materiale.GOMMA` e le altre costanti
Object Pool	riusare oggetti invece di crearli	`PoolParticelle`
Composite	un albero di oggetti trattato come un oggetto solo	non usato: le scene dei giochi grandi
Decorator	aggiungere comportamento avvolgendo un oggetto	non usato: un `Audio` che registra ciò che suona
Visitor	un'operazione nuova su una gerarchia chiusa	non usato: oggi è uno `switch` su un `sealed`
Builder	costruire un oggetto con molti parametri a passi	non usato: `StringBuilder`, `HttpRequest.newBuilder`

I quattro in fondo sono nel libro dei pattern e in molte librerie, e non sono serviti a Palline: è un'informazione utile quanto le altre. Un pattern si usa quando c'è il problema che risolve, e

un programma che ne usa dieci dove ne bastano tre è più difficile da leggere di uno che non ne usa nessuno. Il criterio è sempre lo stesso: si parte dal codice più semplice, si riconosce il problema quando compare (la catena di `if` che cresce, i booleani che si contraddicono, il `new` copiato), e *allora* si mette il pattern che lo risolve, con il suo nome, così che chi legge dopo lo riconosca.

6.6 L'esempio guida: Palline, quinta versione

La barra ha una tendina con gli scenari (*mensole*, *pioggia*, *piramide*, *lunare*), ciascuno un file di testo fra le risorse; il Controller ha `cambiaScenario` e un metodo `ascoltaSempre` per i listener che devono seguire il mondo anche quando viene sostituito, come i suoni; il `main` costruisce l'audio e lo passa; il mondo prende le schegge dal pool.



Palline, quinta versione: lo scenario "piramide" letto da un file, con le bolle che cadono sulle palline di piombo.

Palline è completa. Ha tutto ciò che il capitolo 1 aveva descritto e nessuno dei difetti con cui si apriva: il mondo in metri e secondi, separato dalla finestra e provato dai test; gli eventi che fanno parlare le parti; una fisica a passo fisso che rimbalza bene; i materiali, gli stati, i comandi con l'annullamento e la riproduzione; i suoni caricati una volta e passati; gli scenari in un file che chiunque può scrivere. Il capitolo 7 chiede di rimontarla per intero, con due cose in più, e quello dopo dice dove andare quando Palline non basta più.

Ricetta: creare e condividere, in quattro regole

1. **Un oggetto unico si costruisce nel `main` e si passa** a chi lo dichiara nel costruttore, dietro un'interfaccia che permetta di sostituirlo (`Audio.MUTO` nei test). L'enum singleton solo per ciò che *deve* essere globale.
2. **I `new` di uno stesso tipo di oggetto stanno in un posto solo** (un metodo statico, una classe fabbrica), e se sono dati li si legge da una descrizione; le risorse del programma stanno nel class path e si aprono con `getResourceAsStream`.
3. **Ciò che è uguale per molti si condivide** in un oggetto immutabile, con una fabbrica che restituisce sempre lo stesso.
4. **Si riusa solo ciò che costa**: un pool per oggetti pesanti o numerosissimi, con un unico metodo che azzera tutto lo stato; per il resto, il collettore.

Esercizi

Esercizio 6.1: Quando si carica (⋆)

Con `SuoniGlobali`, in quale istante viene aperta la scheda audio se il primo uso è `SuoniGlobali.TOC.suona()` dentro `Suoni.rimbalzo`? Su quale thread? Che cosa succede se la scheda audio non c'è, e dove si manifesta l'errore? Rispondere alle stesse tre domande per `AudioReale` costruito nel `main`.

Esercizio 6.2: Il test che suona (⋆)

Un test JUnit del mondo registra `new Suoni(...)` come listener per verificare che un rimbalzo produca una chiamata a `toc` con il volume giusto. Scrivere, in poche righe, l'`Audio` da passare al test perché non suoni e permetta di contare le chiamate e leggere l'ultimo volume. Perché con `SuoniGlobali` questo test non si può scrivere?

Esercizio 6.3: La riga sbagliata (⋆)

Per ciascuna riga di scenario dire che cosa fa `FabbricaScenari.daTesto` e con quale messaggio, se fallisce: (a) `pallina 2 5 0.25 Gomma`; (b) `pallina 2 5 gomma`; (c) `ostacolo 1 3 2.5 0.2` come prima riga del file; (d) `mondo 8 6` due volte; (e) `pallina 2 5 0.25 vetro`. Quali di questi errori sarebbe meglio segnalare con il numero di riga, e come?

Esercizio 6.4: Intrinseco o estrinseco (⋆)

Per ciascun dato dire se è stato intrinseco (sta nel `Materiale` condiviso) o estrinseco (sta nella `Pallina`), e perché: la restituzione; la posizione; il colore con cui la gomma si disegna; il numero di rimbalzi fatti da una pallina; l'immagine della bolla; il fatto che una pallina è ferma; la soglia oltre cui la bolla scoppia.

Esercizio 6.5: Pool pieno (⋆⋆)

Il pool ha 400 particelle, ogni rimbalzo ne accende 6 e ognuna vive 0,5 secondi. Quanti rimbalzi al secondo saturano il pool? Che cosa succede oltre: errore, rallentamento, schegge mancanti? E se invece di saltare la scheggia `prendi` *spegnesse e riaccendesse la particella più vecchia*, che cosa cambierebbe nell'aspetto degli schizzi e nel costo?

I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.

Esercizio 6.6: La configurazione in due modi (⋆)

Una classe `Configurazione` con tre valori letti da un file di proprietà fra le risorse (`config.properties`: larghezza del mondo, gravità, volume), scritta in due versioni: un `enum` singleton con `Configurazione.ISTANZA.gravita()`, e una classe normale costruita nel `main` con `Configurazione.daRisorsa()` e passata a chi la usa. Un programma di prova senza finestra usa un `Mondo` con la gravità della configurazione in entrambi i modi, e un test che vuole provare il mondo con una gravità diversa mostra, in un commento, che cosa si può fare nella seconda versione e non nella prima.

Soluzione: progetto `sol-dxtr-02-c06`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 6.4: Il file di proprietà letto dalla cartella sbagliata

`new FileInputStream("config.properties")` legge dalla cartella di lavoro; il file va fra le risorse e si legge con `getResourceAsStream`, come gli scenari. Se le proprietà devono poter essere modificate dall'utente, la copia modificabile sta in una cartella dell'utente (`System.getProperty("user.home")`) e quella fra le risorse è il default.

Esercizio 6.7: La fabbrica di forme (⋆⋆)

Una `FabbricaForme` che legge da un file di testo un elenco di forme per la View (`cerchio x y r`, `rettangolo x y w h`, `poligono x1 y1 x2 y2 ...`) e restituisce una lista di `Shape` di `java.awt.geom` in coordinate del mondo; un pannello che le disegna con la trasformazione di `PnlMondo`. Le forme sono decorazioni di uno scenario (un albero, una casa) e il file dello scenario le richiama con una riga `sfondo nomefile`. Gestire gli errori del file con il numero di riga.

Soluzione: progetto `sol-dxtr-02-c06`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 6.5: La fabbrica che conosce AWT nel Model

Le forme di `java.awt.geom` sono oggetti della *View*. La fabbrica delle forme sta dalla parte della View e il mondo tiene solo il nome del file di sfondo: se `Mondo` importasse `java.awt.geom.Path2D` per tenere lo sfondo, la regola del capitolo 2 sarebbe infranta per una decorazione.

Esercizio 6.8: Il pool di proiettili (⋆⋆)

Una navicella in mezzo a un pannello che ruota con le frecce e spara con la barra spaziatrice: ogni proiettile è un'entità che vive due secondi, attraversa i bordi e riappare dal lato opposto (universo toroidale); i proiettili vengono da un pool di 64, e la navicella non spara se il pool è vuoto. Mostrare in un angolo quanti proiettili sono in volo e quanti liberi. Un test senza finestra spara 100 volte in un secondo e verifica che i proiettili vivi non superino mai 64 e che dopo tre secondi siano tutti liberi.

Soluzione: progetto `sol-dxtr-02-c06`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 6.6: Il proiettile riacceso con la vita vecchia

Un proiettile ripreso dal pool con l'età del giro precedente muore subito, e la navicella "spara a vuoto" in modo intermittente, difficile da riprodurre. L'unico metodo che azzera lo stato, chiamato dal pool, è la cura.

Esercizio 6.9: Un tema a caldo (⋆⋆⋆)

Tre *temi* grafici per Palline (classico, notte, carta) descritti da file di proprietà fra le risorse (colore di sfondo, colore per materiale, spessore dei bordi, se le schegge si disegnano); un `Tema` immutabile e condiviso (Flyweight) ottenuto da una fabbrica `Tema.daNome`; la View riceve il tema corrente attraverso una Strategy `Disegnatore` che sa disegnare una pallina e una scheggia con quel tema; un menu cambia il tema a caldo senza toccare il Model né il Controller.
Suggerimento: il tema è una proprietà della View, annunciata come le proprietà del mondo; il menu la cambia, il pannello si ridisegna. Il Model non sa che esistono i temi.

Soluzione: progetto `sol-dxtr-02-c06`, package `it.unibs.pajc.redolfi.e09`.

Attenzione 6.7: Il colore nella pallina

La tentazione è dare alla pallina un campo `colore` impostato dal tema corrente: al cambio di tema le palline già lasciate restano del colore vecchio, e il Model ha un colore. Il colore è il tema che lo decide, al momento del disegno, dal materiale.

Risposte

Risposta 6.1. Con l'enum la classe `SuoniGlobali` viene inizializzata al primo accesso, cioè dentro `rimbalzo`, chiamato dal mondo dentro `stepNext`, chiamato dal timer: sul thread degli eventi, nel mezzo del primo rimbalzo, con un ritardo visibile (aprire la scheda audio richiede decine di millisecondi) e un fotogramma perso. Senza scheda audio il costruttore lancia `IllegalStateException`, che diventa un `ExceptionInInitializerError` nel listener del timer: il primo rimbalzo manda un'eccezione sulla console e, da lì in poi, ogni accesso a `SuoniGlobali` lancia `NoClassDefFoundError`. Con `AudioReale` la scheda si apre nel `main`, prima della finestra, sul thread principale; se manca, il `main` lo vede subito, lo dice, e passa `Audio.MUTO`: il gioco parte senza suono e senza eccezioni.

Risposta 6.2. Una classe (o una classe anonima nel test) che implementa `Audio` con due campi, `int chiamate` e `double ultimoVolume`, e un `toc` che li aggiorna; si passa `new Suoni(audioFinto)` al mondo e dopo `stepNext` si leggono i campi. Si chiama *test double* (un "sostituto" per il test). Con `SuoniGlobali Suoni` chiama una costante statica che non si può sostituire: il test aprirebbe la scheda audio, suonerebbe davvero, e non avrebbe modo di sapere che cosa è stato suonato.

Risposta 6.3. (a) Funziona: `daNome` converte in minuscolo. (b) `c[4]` non esiste: un'eccezione di indice fuori dai limiti (`ArrayIndexOutOfBoundsException`), con un messaggio che parla di indici e non di scenari. (c) `richiesto` lancia "la prima riga deve essere `mondo`". (d) Funziona ma è sbagliato: il secondo `mondo` ne crea uno nuovo e il primo, con le sue palline, viene perso in silenzio. (e) `daNome` lancia "materiale sconosciuto: vetro". I casi (b) e (d) meritano un controllo esplicito (numero di campi, `mondo` ripetuto); tutti meritano il numero di riga, che si ottiene contando le righe nel ciclo e avvolgendo le eccezioni in una `IllegalArgumentException` con "riga *n*: ..." come messaggio.

Risposta 6.4. Intrinseci: la restituzione, la soglia di scoppio, l'immagine della bolla, il colore con cui la gomma si disegna (nella View o in un tema, ma comunque per materiale, non per pallina). Estrinseci: la posizione, il fatto di essere ferma, il numero di rimbalzi fatti (è una storia della singola pallina). Il criterio: un dato è intrinseco se tutte le palline di quel materiale lo hanno uguale e non cambia mai; se può differire fra due palline di gomma, sta nella pallina.

Risposta 6.5. Ogni particella occupa il pool per 0,5 s: 400 particelle sostengono 800 accensioni al secondo, cioè circa 133 rimbalzi al secondo da 6 schegge. Oltre, `prendi` restituisce `null` e lo schizzo perde schegge: nessun errore e nessun rallentamento (il ciclo su 400 elementi è trascurabile), solo schizzi più poveri. Riciclando la più vecchia, gli schizzi recenti sarebbero sempre completi e quelli vecchi sparirebbero un po' prima del tempo, cosa che a occhio non si nota; il costo sarebbe trovare la più vecchia (un minimo sull'età, 400 confronti) o tenere le particelle in una coda ordinata per accensione, che rende `prendi` immediato: è la variante che i motori usano davvero.

Capitolo 7

Palestra

Questo capitolo non introduce niente di nuovo: mette alla prova ciò che i capitoli 2–6 hanno costruito, con otto programmi completi in difficoltà crescente. I primi due si fanno in un'ora e servono a fissare le regole; i tre in mezzo sono giochi piccoli ma veri; gli ultimi tre sono progetti da un pomeriggio ciascuno, e l'ultimo chiude il problema d'apertura del capitolo 1: Palline completa, montata per intero. Per ciascuno è detto che cosa allena, qual è l'errore tipico, e dove sta la soluzione nel repository del corso. Conviene farli nell'ordine, perché ognuno riusa pezzi del precedente, e conviene, prima di aprire la soluzione, aver scritto almeno il Model e il suo test: è l'unica parte che non si impara leggendo.

Problema 1: chi parla con chi

Allena: le frecce permesse del capitolo 2 e l'ordine degli eventi del capitolo 3. Si fa su carta, poi si verifica con un programma.

Esercizio 7.1: Tre architetture e una sequenza (⋆)

Tre colleghi hanno scritto tre versioni di un contatore con un pulsante e un'etichetta. Nella prima, la View (il pannello) tiene il numero in un campo, lo incrementa nel listener del pulsante e lo scrive nell'etichetta. Nella seconda, il Model `Contatore` tiene il numero e importa `javax.swing.JLabel` per aggiornare l'etichetta che riceve nel costruttore. Nella terza, il Controller riceve il click, chiama `contatore.incrementa()`, e il Model annuncia il cambiamento con un `PropertyChangeSupport` che l'etichetta ascolta. Per ciascuna versione dire quali regole della tabella del paragrafo 2.4 sono rispettate e quali no, e che cosa succede aggiungendo una seconda etichetta e un test senza finestra. Poi, per la terza versione, scrivere la sequenza esatta delle chiamate da quando l'utente preme il pulsante a quando l'etichetta mostra il numero nuovo, e verificarla con un programma che stampa una riga in ogni metodo.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e01`.

Attenzione 7.1: La sequenza "a senso"

La sequenza va scritta *prima* di eseguire il programma, e poi confrontata: chi la scrive dopo copia l'output. L'errore tipico è mettere `setText` prima di `firePropertyChange`, o far partire l'annuncio dal Controller invece che dal Model: nella terza versione il Controller non sa che esiste un'etichetta.

Problema 2: il contatore in tre versioni

Allena: i tre modi di tenere aggiornata una View (il polling, un Observer scritto a mano, `PropertyChangeSupport`) e il costo di ciascuno.

Esercizio 7.2: Polling, listener, proprietà (⋆)

Un Model `Contatore` che si incrementa da solo nel tempo (`stepNext(dt)` aggiunge un'unità ogni secondo di tempo ricevuto) e che si può azzerare. Tre View sullo stesso Model in tre finestre: la prima aggiornata dal Controller a ogni passo del timer (polling); la seconda che ascolta un `ContatoreListener` scritto a mano, con la lista dei listener nel Model; la terza che ascolta la proprietà `valore` con `PropertyChangeSupport`. Ogni View conta quante volte viene chiamato il proprio `setText` e lo mostra accanto al valore; dopo un minuto si confrontano i tre contatori. Un test senza finestra verifica che dopo 10 secondi di `stepNext` a passi di 16 ms il valore sia 10 e che i listener abbiano ricevuto esattamente dieci annunci.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e02`.

Attenzione 7.2: Tre volte lo stesso Model

Le tre View devono guardare *lo stesso* oggetto `Contatore`, non tre copie: se ciascuna finestra crea il suo, il confronto non dice niente. E il Model deve annunciare solo quando il valore cambia davvero, non a ogni passo: altrimenti la seconda e la terza View fanno tante chiamate quante la prima.

Problema 3: Pong

Allena: il motore del capitolo 4 in un gioco con due giocatori, i comandi continui dalla tastiera, il punteggio come View separata, gli stati.

Esercizio 7.3: Pong (⋆⋆)

Un campo visto dall'alto senza gravità, due racchette verticali ai lati mosse con W/S e con le frecce (più tasti premuti insieme: l'insieme dei tasti premuti del capitolo 5, letto a ogni passo), una pallina che rimbalza sulle sponde lunghe e sulle racchette con il motore a passo fisso (le racchette sono ostacoli rettangolari che si muovono: la loro velocità entra nell'urto come nel flipper del capitolo 4); un punto a chi fa uscire la pallina dal lato dell'avversario, e la pallina che riparte dal centro. Il Model annuncia `punto` e `rimbalzo`; il punteggio è una View a parte, due etichette grandi, che ascolta il Model; gli stati sono `Menu`, `InCorso`, `Pausa` e `Finita` (a 11 punti). Un test senza finestra lancia la pallina verso una racchetta ferma e verifica che torni indietro, poi la lancia verso un lato scoperto e verifica il punto.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e03`.

Attenzione 7.3: La racchetta che si muove a scatti

Se la racchetta si sposta solo nel `keyPressed`, si muove a scatti al ritmo della ripetizione automatica della tastiera, e con due tasti premuti il secondo blocca il primo. La tastiera aggiorna un insieme di tasti premuti; il passo del gioco legge l'insieme e muove le racchette di `velocità · dt`.

Problema 4: il gioco della vita, completo

Allena: MVC con gli eventi, gli stati, la fabbrica da file; la differenza fra un mondo che cambia a scatti (ridisegno su evento) e uno continuo.

Esercizio 7.4: Life (⋆⋆)

Il gioco della vita del capitolo 2, completo: il Model annuncia `generazione(int numero, int vive)` e la View si ridisegna *su evento*, non a ogni passo del timer; gli stati `Modifica` (il timer non scatta, i click invertono le celle, si può caricare un pattern), `InCorso` e `Pausa`; una `FabbricaPattern` che legge dalle risorse i pattern celebri in formato testo (righe di `.` e `O`: l'aliante, la navicella leggera, il cannone di Gosper) e li posa nella griglia nel punto cliccato; la velocità come proprietà legata a uno slider nei due versi; una riga di stato con la generazione e le celle vive, listener del Model.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e04`.

Attenzione 7.4: Il ridisegno a ogni tick

Chiamare `repaint` a ogni passo del timer, come nel ciclo di un'animazione, ridisegna sessanta volte al secondo una griglia che cambia cinque volte al secondo. Qui la View ascolta `generazione` e si ridisegna solo allora; il timer serve al Controller per decidere quando chiedere la generazione, e basta.

Problema 5: il biliardo con le buche

Allena: gli urti fra palline e l'attrito del capitolo 4, il turno come stato, i comandi registrati per rivedere un colpo.

Esercizio 7.5: Biliardo (⋆⋆⋆)

Il biliardo del capitolo 4 con sei buche agli angoli e a metà dei lati lunghi: una pallina il cui centro entra nel cerchio di una buca viene tolta dal mondo (evento `imbucata`); se è la bianca, torna al punto di partenza e il turno passa all'altro giocatore. Due giocatori a turni, con lo stato `TurnoDi(giocatore)` che aspetta il colpo, `InMoto` che aspetta che tutto si fermi, e `Finita` quando tutte le palline colorate sono imbucate; il punteggio per giocatore come View a parte. Il colpo è un comando `Colpo(Vec2 velocita)` registrato con il tempo del mondo: un pulsante "Rivedi" rifaà l'ultimo colpo da un mondo ripristinato allo stato di prima (Memento della lista delle palline), con la stessa fisica deterministica.
Suggerimento: la buca è un cerchio senza fisica: si controlla la distanza del centro di ogni pallina dal centro della buca alla fine di ogni passo fisso, prima delle notifiche.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e05`.

Attenzione 7.5: La buca che respinge

Una buca modellata come ostacolo circolare fa rimbalzare le palline invece di inghiottirle; e una buca controllata nel *sottopasso* invece che alla fine del passo può togliere una pallina a metà della risoluzione degli urti, lasciando l'altra pallina della coppia con un impulso a metà. Le palline si tolgono dopo che il passo è completo, come le entità morte del capitolo 5.

Problema 6: Mattoni

Allena: tutto il motore con gli ostacoli, i livelli da file (Factory), il punteggio e le vite come proprietà, gli stati e i comandi, le risorse passate.

Esercizio 7.6: Un Breakout (⋆⋆⋆)

Una racchetta in basso mossa con le frecce (comando continuo), una pallina che rimbalza su pareti, racchetta e mattoni; i mattoni sono ostacoli con un numero di colpi residui e un materiale (i mattoni di vetro si rompono al primo colpo, quelli di pietra al terzo, quelli d'acciaio mai), e un mattone rotto è tolto dal mondo a fine passo con un evento `mattoneRotto(mattone, punti)`; il punteggio e le vite sono proprietà del Model mostrate da una View a parte; i livelli sono file di testo fra le risorse (una riga per fila di mattoni, un carattere per mattone: `.` vuoto, `v` vetro, `p` pietra, `a` acciaio), costruiti da una fabbrica; gli stati `Menu`, `InCorso`, `Pausa`, `LivelloFinito` (che carica il successivo) e `Finita`; i suoni passati come `Audio` e listener degli eventi. La pallina parte dalla racchetta con la barra spaziatrice, e il punto in cui colpisce la racchetta decide l'angolo di rimbalzo (ai bordi più inclinato), come in ogni Breakout.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e06`.

Attenzione 7.6: La pallina che attraversa i mattoni

I mattoni sono sottili e la pallina è veloce: senza i sottopassi del capitolo 4 la pallina attraversa una fila intera a ogni partita. E il mattone colpito va tolto *a fine passo*: toglierlo dentro il ciclo degli ostacoli, mentre lo si sta scorrendo, è una `ConcurrentModificationException` o, peggio, un mattone saltato.

Problema 7: Asteroidi

Allena: la gerarchia delle entità con il Template Method, i renderer come Strategy, il pool dei proiettili, l'universo toroidale, le collisioni fra forme.

Esercizio 7.7: Asteroidi (⋆⋆⋆)

Una navicella al centro che ruota e accelera (le frecce: comandi continui) con l'inerzia e un leggero attrito, in un universo toroidale (chi esce da un lato rientra dall'altro); asteroidi grandi che vagano e che, colpiti da un proiettile, si spezzano in due medi, i medi in due piccoli, i piccoli spariscono; i proiettili da un pool, vivono due secondi; la navicella colpita da un asteroide perde una vita e riparte al centro con tre secondi di invulnerabilità (uno stato dell'entità, non del gioco). Le entità estendono un'`Entita` con il Template Method del capitolo 5; ogni tipo ha il suo renderer (Strategy scelta con uno `switch` su un `sealed`, o con una mappa per classe: motivare la scelta); le collisioni si decidono nel Model con i cerchi (raggio per tipo), e la View disegna le forme vere con `Path2D` e `AffineTransform`; il punteggio e le vite come proprietà; gli stati del gioco. *Suggerimento: l'universo toroidale si applica alle posizioni alla fine del passo (`x = (x + L) % L` con il resto positivo), e la collisione fra due entità vicine al bordo va controllata anche con la copia "dall'altra parte" se si vuole essere precisi; per cominciare si può ignorare.*

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e07`.

Attenzione 7.7: La navicella invulnerabile per sempre

L'invulnerabilità è uno stato con un tempo: se la si realizza con un booleano acceso alla rinascita e nessuno lo spegne, la navicella non muore più, e il gioco sembra funzionare. L'entità conta i secondi in `dopoIlMoto` e si spegne da sola; un test senza finestra fa scontrare la navicella con un asteroide a 0, 2 e 4 secondi dalla rinascita e verifica che solo l'ultimo scontro conti.

Problema 8: Palline completa

Allena: tutto, insieme. È il problema d'apertura del capitolo 1, montato per intero con due cose in più.

Esercizio 7.8: Il giocattolo del capitolo 1, fatto bene (⋆⋆⋆)

Rimontare Palline dalla versione del capitolo 6, scrivendo da capo (non copiando) il Model con i suoi test, la View, il Controller con gli stati e i comandi, l'audio passato e gli scenari da file, e aggiungere due funzioni. La prima è il *salvataggio dello scenario corrente*: un comando "Salva scenario" scrive in un file scelto con `JFileChooser` il mondo com'è adesso, nel formato della fabbrica (le mensole, le palline con posizione e materiale, la gravità), e il menu degli scenari lo può ricaricare: è la fabbrica al contrario, e la fabbrica deve leggere ciò che scrive. La seconda è una *seconda View* sullo stesso mondo, in un pannello sotto quello principale: il grafico dell'energia meccanica totale nel tempo (`energia()` del capitolo 4), campionata una volta ogni decimo di secondo di tempo del mondo, che scorre da destra a sinistra e mostra i gradini a ogni rimbalzo e il calo con le bolle che scoppiano. La View legge il mondo e ascolta i suoi eventi; non tiene altro che la sua storia di campioni.

Soluzione: progetto `sol-dxtr-02-c07`, package `it.unibs.pajc.redolfi.e08`.

Attenzione 7.8: La seconda View che guida il tempo

La View dell'energia ha bisogno di campionare ogni decimo di secondo di *tempo del mondo*: non ha un timer suo (ne avrebbe uno sfasato) né chiede al Controller di chiamarla (il Controller non deve conoscerla). Ascolta il mondo, che annuncia un evento `passo` o espone `tempo()`, e campiona quando il tempo supera il prossimo decimo. Se Palline completa gira con il tempo misurato nel posto sbagliato, questo grafico lo mostra subito.

Capitolo 8

Dove andare adesso

Palline è un giocattolo, e i pattern che la tengono in piedi sono quelli con cui si scrivono programmi molto più grandi. Questo capitolo indica, per ciascuna direzione in cui Palline potrebbe crescere, qual è il passo successivo e dove si legge; per ognuna c'è un assaggio di che cosa cambia, non una trattazione.

8.1 Quando la gerarchia non basta: Entity-Component-System

Il Template Method di `Entita` regge finché le entità sono *specie*: una pallina, una scheggia. Nei giochi grandi le entità si combinano in modi che una gerarchia non descrive: un asteroide che è anche un nemico che spara, un proiettile che ha una luce e un suono, un personaggio che perde la capacità di volare. La risposta è l'*Entity-Component-System*: un'entità è solo un identificatore, le sue proprietà sono *componenti* indipendenti (posizione, velocità, forma, salute, arma), e i *sistemi* sono funzioni che a ogni passo operano su tutte le entità che hanno certi componenti (il sistema del moto su chi ha posizione e velocità, il sistema delle collisioni su chi ha posizione e forma). È la Strategy e il Template Method portati all'estremo: nessuna ereditarietà, solo composizione di dati e funzioni. Nystrom lo descrive nel capitolo *Component* del suo libro, e la versione Java più usata è *Artemis*; ha senso quando i tipi di entità sono decine, non prima.

8.2 Il ciclo fuori dal thread degli eventi

Il ciclo di Palline gira sul thread degli eventi con un `javax.swing.Timer`: è la scelta giusta finché un passo dura pochi millisecondi. Con mille palline e gli urti a coppie, o con un'intelligenza artificiale che pensa, il passo diventa lungo e l'interfaccia si inceppa. Il passo successivo è un *thread di simulazione*: un ciclo `while` su un thread proprio (o uno `ScheduledExecutorService`) che fa avanzare il mondo a passo fisso, e una View che a ogni fotogramma legge uno *stato immutabile* pubblicato dal mondo (una copia delle posizioni), oppure riceve gli eventi con `invokeLater`. Le regole sono quelle di *Thread e concorrenza*: il mondo è confinato nel suo thread, ciò che passa fra i due è immutabile o protetto, e i comandi dell'utente entrano nel mondo attraverso una coda, così che il pattern Command del capitolo 5 diventi anche il modo in cui i due thread si parlano. L'interpolazione fra l'ultimo stato e il precedente, citata nel capitolo 4, si fa qui: la View conosce due stati e il tempo rimasto nell'accumulatore, e disegna in mezzo.

8.3 Motori fisici e strutture spaziali

Il motore del capitolo 4 fa palline e rettangoli fissi. Un motore vero fa *corpi rigidi* di qualunque forma, con rotazione, attrito di contatto, vincoli (giunti, corde, molle), e risolve gli urti con metodi iterativi che gestiscono le pile di oggetti senza che tremino. In Java i due più usati sono *JBox2D*, la traduzione di Box2D (il motore di Angry Birds), e *dyn4j*, scritto in Java da zero; entrambi espongono un mondo con `step(dt)` e dei corpi con posizione e velocità, e si integrano con l'architettura di Palline sostituendo l'interno di `Mondo.stepNext`. Il concetto che vale la pena capire prima è la *broad phase*: con migliaia di oggetti la verifica a coppie è impossibile, e il mondo si divide in una griglia di celle (o in un *quadtree*, celle che si suddividono dove ci sono più oggetti) per confrontare solo i vicini; il problema 4.8 con mille palline è il posto giusto per provarci. Per la fisica degli urti il riferimento è Millington; per le collisioni Ericson; per l'integrazione e il passo fisso gli articoli di Fiedler, brevi e con il codice.

8.4 Il gioco in rete

Due giocatori su due computer con lo stesso mondo è un problema che Palline ha già mezzo risolto. La simulazione è deterministica e i comandi sono oggetti: due computer che partono dallo stesso scenario e si scambiano i comandi con il tempo del mondo in cui vanno eseguiti simulano esattamente lo stesso mondo, senza mai trasmettere le posizioni. È il modello *lockstep* dei giochi di strategia, e il nastro del capitolo 5 è già il protocollo; ciò che manca è il trasporto (socket TCP, oggetti serializzati o righe di testo) e la gestione del ritardo (un comando ricevuto in ritardo non può essere eseguito nel passato: si esegue con un ritardo fisso concordato, e il gioco attende chi è indietro). I giochi d'azione fanno diversamente, con un server che simula e i client che prevedono e correggono; per Palline il lockstep basta, e la dispensa sulla rete dà gli strumenti per il trasporto.

8.5 I test del Model, come abitudine

Ogni capitolo ha avuto un test sul mondo e nessuno sulla View. È la proporzione giusta, e vale la pena renderla un'abitudine: il Model si scrive *insieme* al test, che diventa il primo utilizzatore del suo codice e spesso ne decide l'interfaccia (un `Mondo` che si può far avanzare senza finestra, un `Audio` che si può sostituire). JUnit 5 ha ciò che serve per i Model di un gioco: test parametrici per provare la stessa fisica con molti valori, `assertAll` per verificare più cose in un colpo, e i *test double* (il finto `Audio` del capitolo 6) scritti a mano o con *Mockito*. La View non si testa con JUnit: si guarda, e per le regressioni si confrontano le immagini prodotte come fa il programma che genera le schermate di questa dispensa.

8.6 Oltre Swing

Swing è la libreria di questa dispensa perché è nel JDK e perché un pannello con `paintComponent` è tutto ciò che un gioco 2D semplice chiede. Per un gioco con molte immagini, animazioni fluide e suono, *libGDX* è la libreria Java più usata: ha il ciclo del gioco a passo fisso, il rendering con la scheda grafica, Box2D incluso, e gira anche su Android; la sua architettura (`Game`, `Screen`, `Stage`, `Actor`) è fatta degli stessi pattern di Palline con altri nomi. *JavaFX* è l'altra libreria grafica di Java, con un *scene graph* (un Composite di nodi grafici) e le *proprietà osservabili* (un `PropertyChangeSupport` integrato nel linguaggio delle View): chi la incontra riconosce l'MVC e l'Observer di questa dispensa in ogni pagina della sua documentazione.

Nelle altre dispense

Interfacce grafiche con Swing è la dispensa gemella di questa: tutto ciò che riguarda il disegno, le coordinate, i componenti e i loro Model, che qui è stato dato per noto. *Thread e concorrenza* è il riferimento per il ciclo del gioco fuori dal thread degli eventi, per `invokeLater` e per i pool di thread, che sono un Object Pool. *Le interfacce in Java moderno* spiega i `sealed` e lo `switch` con i pattern, su cui si regge il confronto fra Strategy e `switch` del capitolo 5. *Dal nome alla funzione* è il motivo per cui i comandi e i listener di questa dispensa sono lambda, e *Java Fast Forward* quello per cui gli eventi sono record.

Riferimenti

- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. Il libro dei pattern: ventitré schede, con il problema, la struttura e le conseguenze di ciascuno. I nomi usati qui sono i suoi.
- R. Nystrom, *Game Programming Patterns*, Genever Benning, 2014; gratuito in rete (`gameprogrammingpatterns.com`). I pattern del libro dei pattern visti da chi fa giochi, più quelli propri dei giochi (Game Loop, Update Method, Component, Object Pool, Spatial Partition). Il libro da leggere dopo questa dispensa.
- G. Fiedler, *Fix Your Timestep!* e *Integration Basics*, `gafferongames.com`. Il passo fisso con l'accumulatore e il confronto fra integratori, in due articoli con il codice.
- I. Millington, *Game Physics Engine Development*, 2a ed., CRC Press, 2010. Un motore fisico costruito da zero, capitolo per capitolo.
- C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2004. Tutte le geometrie degli urti, dal punto più vicino di un rettangolo alle griglie e agli alberi.
- J. Bloch, *Effective Java*, 3a ed., Addison-Wesley, 2018. Item 3 (il singleton con l'enum), 17 (l'immutabilità), 18 (composizione contro ereditarietà), 20 (interfacce contro classi astratte).
- Javadoc di `java.beans` (`PropertyChangeSupport`), `javax.swing.event` (`EventListenerList`), `java.util.function` (`Supplier`, `Consumer`) e `javax.sound.sampled`.